



Faculty of Computers &
Artificial Intelligence



Benha University

Alzheimer's Detection with Generative AI

Artificial intelligence, department

Team members

1. Ahmed Ashraf Ahmed Alsayed (Leader).
2. Ahmed Hussien Shokry Mohammed.
3. Ahmed Elsayed Ahmed Hassan.
4. Ahmed Elsayed Mahmoud Arafat.
5. Abdelrahman Abdo Hassan Ahmed.
6. Hani Yasser ElMetwaly Sorour Ahmed.
7. Radwa Mohamed Said Abd El- Shafi.
8. Shrouk Salama Mahmoud Mohamed.
9. Nada Adel Ismael Hussainen Shahin.

Under Supervision of

Dr. Sara Sweidan.

Eng. Sahar Mohamed.

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

"وَيَسْأَلُونَكَ عَنِ الرُّوحِ قُلِ الرُّوحُ مِنْ أَمْرِ رَبِّي وَمَا أُوتِيتُمْ مِنَ الْعِلْمِ إِلَّا قَلِيلًا"

صدق الله العظيم

Acknowledgment

The success of any project will not happen without the help of many people around us. First, we thank God for giving us the ability to complete our project, we thank our family, friends, doctors, and instructors that help us to complete the project successfully, we want to thank Specially **Dr. Sara Swidan** for his encouragement, guidance, patience, and his advice for us that have helped us to help us to achieve the success in our project.

In addition, many thanks go to **Eng. Sahar Mohamed and Eng. Ahmed Ashraf** for their inspirational thoughts, valuable guidance, and assistance throughout this work and for their patience. Finally, we are very grateful to the many people who helped us with contributions to complete this project. , We thank every person in Banha Faculty of Computer and artificial intelligence who may have helped in the tiniest ways possible in this project as well as along the duration of the educational time in college., We deeply thank our parents and siblings for the support and encouragement along the four years of college time.

Table Of Contents

Chapter 1→ Introduction & Background	10
1. Introduction	10
2. Problem Definition	14
3. Scope Initiation	17
4. Summary.....	18
Chapter 2 → Analysis and Design	19
1. Introduction	19
2. User and System Requirements.....	21
2.1 User Requirements.....	21
2.2 System Requirements	21
2.2.1 Functional Requirements	22
2.2.2 Non – Functional Requirements	23
3. System Design	24
3.1 System Architecture	24
3.2 Use Cases Diagram	25
3.3 Activity Diagrams.....	26
3.3.1 Login Diagram	26
3.3.2 User Diagram.....	27
3.3.3 Doctor Diagram	28
3.4 Class Diagrams	29
3.5 Sequence Diagrams.....	30
3.5.1 User Diagram.....	30
3.5.2 Doctor Diagram	31
3.6 Gantt chart.....	32
3.7 Data Flow Diagram (DFD)	33
3.7.1 Context Diagram (Level 0)	33
3.7.2 DFD (Level 1)	34
3.8 State Diagram	35
3.9 Entity Relationship Diagram (ERD)	36

Table Of Contents

4. Technologies and Tools.....	37
5. Summary.....	37
Chapter 3 → Alzheimer's Model (4 class of Images) Analysis & Visualization.....	38
1. Libraries	38
2. Dataset Gathering	38
3. Read Data	39
4.1 Data preprocessing	40
4.2 Analyzing & Visualization	40
4.3 Split Data	40
4.5 Data Augmentation	43
5. Model training	45
5.1 CNN	46
5.1.1 CNN build steps.	46
5.1.2 Model Architecture.....	47
5.1.4 Training the model.....	48
5.1.5 Model Performance	48
5.1.6 Evaluate the Model.....	49
5.1.7 Model score	50
5.1.8 Model Evaluation.....	50
5.1.8.1 Confusion Matrix.....	50
5.1.8.2 Classification Report.....	50
5.2 VGG16	51
5.2.1 Model Architecture.....	51
5.2.2 Model Scratch	52
5.2.3 Model Performance	53
5.2.4 Evaluate the Model.....	54
5.2.5 Model Score	55
5.2.6.1 Confusion Matrix.....	55
5.2.6.2 Classification Report.....	55
5.3 DenseNet201	56
5.3.1 Model Architecture.....	56

Table Of Contents

5.3.2 Model Scratch	57
5.3.3 Model Performance	58
5.3.4 Evaluate the Model.....	59
5.3.5 Model Score	60
5.3.6 Model Evaluation.....	60
5.3.6.1 Confusion Matrix.....	60
5.3.6.2 Classification Report.....	60
5.4 ResNet152V2	61
5.4.1 Model Architecture.....	61
5.4.2 Model Scratch	62
5.4.3 Model Performance	63
5.4.4 Evaluate the Model.....	64
5.4.5 Model Score	65
5.4.6 Model Evaluation.....	65
5.4.6.1 Confusion Matrix.....	65
5.4.6.2 Classification Report.....	65
5.5 MobileNetV2	66
5.5.1 Model Architecture.....	66
5.5.2 Model Scratch	67
5.5.3 Model Performance	68
5.5.4 Evaluate the Model.....	69
5.5.5 Model Score	70
5.5.6 Model Evaluation.....	70
5.5.6.1 Confusion Matrix.....	70
5.5.6.2 Classification Report.....	70
5.6 Model predictions	71
5.7 Summary.....	72
Chapter 4 → UI Design	73
Chapter 5 → Generative AI Chatbot	81
5.1 Introduction	81
5.2 Libraries	81

Table Of Contents

5.2 Load Data	81
5.3 Split Data	82
5.4 Model training	82
5.4.1 Transformer	83
5.4.1.1 Text Vectorization	83
5.4.1.2 Positional Embedding	84
5.4.1.3 Scaled Dot-Product Attention	84
5.4.1.4 Causal Masking.....	85
5.4.1.5 Multi-Head Attention	86
5.4.1.6 Encoder	87
5.4.1.7 Decoder.....	87
5.4.1.8 Transformer Block for Encoder and Decoder	88
5.4.1.9 Training the Transformer	89
5.4.1.10 Model Evaluation.....	91
5.4.2 Pre-Trained GPT-2	93
5.4.2.1 Helper Function for GPT-2	93
5.4.2.2 Training GPT-2.....	95
5.4.2.3 Evaluation GPT-2.....	96
5.4.2.4 Generation Using Custom GPT-2	96
<i>References</i>	<i>98</i>

List Of Figure

Figure (1) System Architecture.....	24
Figure (2) Use Cases Diagram.....	25
Figure (3) Login Diagram	26
Figure (4) User Diagram.....	27
Figure (5) Doctor Diagram.....	28
Figure (6) Class Diagrams	29
Figure (7) User Sequence Diagram.....	30
Figure (8) Doctor Sequence Diagram	31
Figure (9) Gantt chart	32
Figure (10) Context Diagram (Level 0)	33
Figure (11) DFD (Level 1)	34
Figure (12) State Diagram	35
Figure (13) ERD	36
Figure (14) Machine Learning Process	38
Figure (15) Train Data	41
Figure (16) Test Data.....	42
Figure (17) Validation Data	42
Figure (18) Data Visualization	44
Figure (19) CNN Model.....	46
Figure (20) CNN Model Scratch	47
Figure (21) CNN Evaluation	49
Figure (22) CNN Confusion Matrix	50
Figure (23) VGG16 Model	51
Figure (24) VGG16 Model Scratch	52
Figure (25) VGG16 Evaluation	54
Figure (26) VGG16 Confusion Matrix	55
Figure (27) DenseNet201 Model	56
Figure (28) DenseNet201 Model Scratch	57
Figure (29) DenseNet201 Evaluation	59
Figure (30) DenseNet201 Confusion Matrix	60
Figure (31) ResNet152V2 Model	61
Figure (32) ResNet152V2 Model Scratch	62
Figure (33) ResNet152V2 Evaluation	64
Figure (34) ResNet152V2 Confusion Matrix	65
Figure (35) MobileNetV2 Model	66
Figure (36) Model Scratch	67
Figure (37) MobileNetV2 Evaluation.....	69
Figure (38) MobileNetV2 Confusion Matrix	70
Figure (39) Model predictions.....	71
Figure (40) Models Summary	72
Figure (41) Transformer Data Split.....	82
Figure (42) Transformer Block	88

Chapter 1→ Introduction & Background

Figure (43) Training the Transformer	90
Figure (44) Training GPT-2	95

Chapter 1→ Introduction & Background

1.Introduction

Alzheimer's disease (AD), named after the German psychiatrist and neuropathologist **Alois Alzheimer** who first described it in 1906, is a progressive neurodegenerative disorder characterized by cognitive decline and memory impairment. It is the most common cause of dementia, contributing to approximately 60-70% of all cases.

The disease typically begins with **subtle memory problems**, such as difficulty remembering recent events or conversations, and progresses over time, affecting various cognitive functions including language, reasoning, and judgment. As the condition advances, individuals may experience disorientation, mood swings, changes in behavior, and difficulty performing everyday tasks. In the later stages, individuals may lose the ability to recognize loved ones, communicate effectively, and care for themselves.

The underlying cause of Alzheimer's disease is not fully understood, but it is believed to involve a complex interplay of genetic, environmental, and lifestyle factors. Pathologically, Alzheimer's is characterized by the accumulation of abnormal protein deposits in the brain, namely beta-amyloid plaques and tau tangles, which lead to the destruction of nerve cells and disruption of neural communication.

Currently, there is **no cure for Alzheimer's disease**, and available treatments aim to alleviate symptoms and slow down disease progression. Research into potential treatments and preventive strategies continues, driven by the urgent need to address the growing impact of Alzheimer's on individuals, families, and society as a whole.

1.1 Background

A major cause of **Alzheimer's disease** has not yet been identified, but there are some factors that increase the risk of developing it, including the following

- **Age**, where the risk of developing the disease increases with age, as most people with this disease are over **65 years old**.
- **Family history**, the risk of Alzheimer's increases if a person's family member has the disease.
- **Genes**, where some genes have been linked to causing Alzheimer's disease.
- Other factors, such as head injuries, heart health and related diseases, increase the risk of Alzheimer's disease in people with health conditions that cause damage to the heart or blood vessels, such as strokes, high blood pressure, diabetes, and others.

The course of the disease is divided into **three stages**, with a progressive pattern of cognitive and functional impairment.

Early stage: Mild symptoms.

The early stage of [Alzheimer's disease](#) begins with mild symptoms such as memory loss and mental confusion. During this period, a person may have difficulty remembering newly acquired information, and may make mistakes in puzzles and games in which they were previously successful. Sometimes they may lose some objects in their environment or at home and have difficulty finding them.

These symptoms, *which do **not affect a person's daily life***, are usually noticed by their inner circle. At this stage, the person can still do their job in daily life and live a normal life. However, more severe symptoms may appear if the disease is not diagnosed during this period. Family doctors or neurologists are usually the first to be visited because of mild symptoms.

The diagnosis of the disease in the early stage is usually made using a series of tests and imaging methods. These tests include **cognitive tests**, **neuropsychological tests**, and **imaging tests** such as magnetic resonance imaging (**MRI**).

Middle stage: Symptoms that affect daily life.

The middle stage of Alzheimer's disease is characterized by memory loss and severe loss of cognitive abilities. At this stage, symptoms that ***affect a person's daily life begin*** to appear and they may lose their independence. A person may forget the names of people they know, have trouble remembering places in their immediate surroundings, and have difficulty performing simple tasks they've done before.

Cognitive and emotional difficulties in a person increase in the middle stage. A person may have serious difficulties understanding and communication and may have difficulty understanding speech. Changes in personality and behavior can be observed due to changes occurring in a person's mind. For example, a person may be moody, irritable, or anxious.

In the intermediate stage, the disease is diagnosed and medical support is sought. Treatment is carried out to slow the progression of the disease and a specific care plan is prepared for the patient. Patients during this stage need more help and their caregivers need a plan as well.

Advanced stage: the condition that needs care.

The advanced stage of Alzheimer's disease leads to a ***person being completely dependent on others***. At this stage, he develops severe symptoms such as severe memory loss, language impairment, and walking difficulties. A person becomes having difficulty performing activities of daily living and needs full-time care and support.

As the symptoms of the disease increase in the advanced stage, personal and behavioral changes increase significantly. Patients may become more anxious or afraid. Speech and communication skills may also be affected, and the patient may be completely dependent on caregivers for daily activities and personal care.

Because the patient is now completely dependent on others, home care may not be possible, and the patient may need to be transferred to a specialized care center for care. These care centers provide specialized services provided by healthcare professionals, specialists in daily activities and personal care of patients.

As of my *last update* in **January 2022**, global statistics for Alzheimer's disease may vary based on different sources and methodologies used for data collection. However, I can provide some general information:

Prevalence: Alzheimer's disease and other dementias are significant public health concerns globally, with **millions** of people affected. According to estimates from the **World Health Organization (WHO)**, there were approximately **50 million** people worldwide living with dementia **in 2019**, with Alzheimer's disease being the most common cause of dementia. This number is expected to increase significantly in the coming years due to aging populations and other factors.

Future Projections: With population aging and increasing life expectancy, the prevalence of Alzheimer's disease is projected to rise substantially in the coming decades. Some projections suggest that the number of people living with dementia worldwide could reach **152 million by 2050** if no effective interventions are implemented to prevent or slow the progression of the disease.

2. Problem Definition

Alzheimer's disease entails progressive cognitive decline, memory loss, and daily function impairment, imposing emotional, financial burdens, and global healthcare challenges, necessitating early detection, effective treatment, and support.

2.1 The Importance of This Problem

The problem of **Alzheimer's disease** is of paramount importance due to its devastating impact on individuals and society. Alzheimer's patients face progressive cognitive decline, impacting memory, reasoning, and daily functioning. As the disease advances, patients require increasing levels of care, often placing strain on families and caregivers. Alzheimer's disease is the leading cause of dementia, affecting millions worldwide. It poses significant challenges to healthcare systems, requiring specialized resources for diagnosis, treatment, and support.

The emotional toll on patients and their loved ones is profound, as cherished memories and relationships fade away. Addressing Alzheimer's is essential for preserving quality of life and dignity for affected individuals. Research into prevention, early detection, and effective treatments is critical to combatting this growing public health crisis. Supporting Alzheimer's patients and their families is crucial for ensuring compassionate care and support throughout the disease journey.

2.2 The Current Solutions

This is a sample of some applications that help **Alzheimer's patients**:

- **Memory Stimulation Apps:**
 - **My Reef 3D:** Interactive game with virtual fish to improve memory.
 - **Reminiscence:** Displays pictures from the past to stimulate memories.

- **Lumosity:** Training games to enhance memory and cognitive skills.
- **Communication Apps:**
 - **Timeless:** User-friendly app for remembering events and staying connected with family and friends.
 - **Talking Point:** Communication platform for Alzheimer's patients and their families.
 - **Ava:** Writing and speech assistance app to aid communication.
- **Daily Assistance Apps:**
 - **Caregiver Smart:** Smart assistant for caregivers of Alzheimer's patients.
 - **Medisafe:** App to remind patients to take their medication.
 - **WanderSafe:** GPS tracking system to prevent patients from wandering off.
- **Diagnostic Apps:**
 - **SimBrain:** Brain imaging analysis tool for early detection of Alzheimer's signs.
 - **Akili Interactive:** Interactive games to assess cognitive abilities.
 - **Cogito:** Digital tests for diagnosing Alzheimer's disease.
- **Other Apps:**
 - **HappyNeuron:** Training games to stimulate cognitive functions.
 - **BrainHQ:** Games and activities to improve memory and focus.
 - **Pzizz:** Music and soundscapes to enhance sleep.

2.3 Project Objectives

The primary objective of this project is to leverage state-of-the-art **Deep Learning** methodologies for the purpose of medical image analysis, focusing on the precise detection of **Alzheimer's disease**. By employing advanced neural network architectures, we aim to develop a robust diagnostic tool capable of accurately identifying the presence of Alzheimer's pathology within medical imaging data.

In addition to the deployment of Deep Learning models, we propose the integration of a Generative model component into our framework. This Generative model serves a dual purpose: firstly, to corroborate the findings of disease detection through the synthesis of supplementary visual data, thus enhancing the reliability of diagnostic outcomes. Secondly, the Generative model serves to mitigate patient anxiety and stress associated with medical examinations by providing personalized, visually coherent representations of diagnostic results.

The utilization of cutting-edge Deep Learning techniques underscores our commitment to achieving high levels of accuracy and reliability in disease detection within medical imaging contexts. Through rigorous model training and validation processes, we endeavor to establish a robust and clinically viable tool for Alzheimer's diagnosis, with the potential for broader applications in medical image analysis.

Furthermore, the incorporation of a **Generative model** component reflects our holistic approach to addressing not only diagnostic accuracy but also patient-centered care. By prioritizing the reduction of patient stress and anxiety through innovative technological solutions, we aim to enhance the overall patient experience within the realm of medical imaging diagnostics.

The interdisciplinary nature of this project necessitates collaboration between experts in Deep Learning, medical imaging, and healthcare. Through close collaboration and knowledge exchange, we aim to

leverage the collective expertise of diverse stakeholders to drive meaningful advancements in Alzheimer's disease detection and patient care.

Our project adheres to the highest standards of ethical conduct and data privacy, ensuring that patient confidentiality and consent are paramount throughout all stages of research and development. By upholding these principles, we demonstrate our commitment to responsible innovation and the well-being of patients and individuals involved in our study.

3.Scope Initiation

Our project's scope focuses on assisting individuals **aged 50 to 85** who are in **the early** and **middle stages** of Alzheimer's disease. With a primary aim to support both patients and healthcare providers, our initiative serves as a crucial assistant to doctors in diagnosing and managing the disease.

Through innovative technologies and tailored interventions, we aim to help patients refresh their memory and enhance cognitive function, thereby improving their quality of life. By targeting this specific demographic, we acknowledge the critical need for early intervention and support in Alzheimer's care.

Our scope encompasses the development of assistive tools and applications that aid healthcare professionals in accurate diagnosis and personalized treatment planning. Additionally, we aim to empower patients by providing them with accessible resources and support systems to navigate their journey with Alzheimer's disease.

By focusing on the early and middle stages of the disease, we aim to intervene at a crucial juncture where timely assistance can significantly impact disease progression and patient outcomes. Through collaborative efforts with medical experts and caregivers, we aspire to optimize care delivery and enhance patient well-being.

Our scope initiation emphasizes a holistic approach to Alzheimer's disease management, encompassing medical, psychological, and social aspects of care. By addressing the needs of both patients and healthcare providers, we aim to foster a supportive ecosystem that promotes early detection, effective intervention, and improved outcomes for individuals living with Alzheimer's disease.

4.Summary

Now we have finished the first chapter of our project, which helped us understand and know all the information on which the project is based, starting with a clear and concise introduction to **Alzheimer's disease (AD)**, its progressive nature, and its crippling effects on memory and thought. It emphasizes the significant impact on millions globally, highlighting the lack of a cure and the urgency for solutions.

The chapter acknowledges the existing efforts in supporting patients and caregivers, outlining various applications that assist with memory stimulation, communication, and daily tasks. However, it recognizes the limitations of these solutions and paves the way for our innovative project.

This section introduces our ambitious project utilizing cutting-edge Deep Learning for precise AD detection through medical image analysis. It goes beyond mere diagnosis, incorporating a Generative AI component to create personalized visualizations for patient comfort and understanding.

Our focus isn't solely on technology; it's about people. We target the crucial early and middle stages, empowering both patients and healthcare providers with personalized tools and plans. Collaboration and ethical considerations are paramount as we strive for a holistic approach encompassing medical, psychological, and social aspects of care.

Chapter 2 → Analysis and Design

1. Introduction

Analysis is defined as the process of identifying problems and organizing the facts and details of a system. Analysis can also be described as the careful breakdown of a system into its organized components or parts. This process helps you to know the ins and outs of the system. It is important to know what works with what, what makes something work or fail, and what can work independently.

Importance of analysis:

Analysis is important because it provides a means to the solutions in the system through the various tasks involved in performing the analysis **graphs**. Through these various tasks, which makes it easy to implement the project because it serves as a map of everything that is involved in the project and the overall quality of the system can be easily modified or improved, and the occurrence of errors can be reduced in the end.

There are a set of **procedures and steps** to create this analysis summarized as follows:

1. Determinations of requirements: We prepare **user** and system **requirements** and have 2 types:

® Functional requirements

® Non - functional requirements

2. Stake holders: Determine the people who benefit or are interested in the project such as:

® Doctors

® Users (Patients)

3. System Design: It consists of a set of Diagrams that describe the entire project and a working mechanism such as:

® System Architecture

® Use Cases

® Activity Diagram

® Class Diagram

® Sequence Diagrams

® Database Design

4. Application of the system

5. System evaluation (as we will see in next chapter)

6. Modify the system (as we will see in next chapter)

2. User and System Requirements

2.1 User Requirements

User requirements are just what the name implies. They are requirements set by the **end user**. These requirements express how an application should perform in terms of the functions to be done. User requirements provide information that serves as the basis for further specification and design.

User Requirements
contains a model that detection Alzheimer's disease
contains intelligent chat for Alzheimer's.
Provides a lot of tests besides chatting to diagnose the patient's condition.
Provides a list of histories containing all test results.
Be easy, simple, and understandable for all age groups.
Provide the possibility communicate with a specialized doctor if necessary.

2.2 System Requirements

The requirements that specify criteria that can be used to judge that the system operates as expected.

2.2.1 Functional Requirements

Requirements	Description
User authentication	An application for users to log in and register, either through an email and password combination.
Alzheimer's detection interface	Interface for users for Alzheimer's classification. This should be intuitive and easy to use.
Chat interface	A chat interface for users to communicate with the chatbot. This should be intuitive and easy to use.
Natural Language Processing (NLP) and understanding	A chatbot must be able to understand user input and respond to it in a conversational manner, and chatbot must be able to track the context of a conversation and provide relevant responses.
Personalized recommendations	Based on the user's assessment, the chatbot should be able to provide personalized recommendations for self-care, therapy, and other mental health resources.
Notification system	The system should have a notification system to alert users about scheduled assessments, follow-ups, or important information related to Alzheimer's detection and care.

2.2.2 Non – Functional Requirements

Requirements	Description
Response Time	The system should provide quick responses to user inputs, with a maximum response time of X seconds for both the chat interface and Alzheimer's detection interface.
Usability	for both the generative model and Alzheimer's detection model must be user-friendly and provide a smooth and intuitive experience for users.
Maintainability	for both the generative model and Alzheimer's detection model must be easy to maintain and upgrade as necessary.
Reliability	The application should be reliable and available, so users can access support whenever they need it.
Scalability	Design the system to handle a growing user base by ensuring scalability. The generative and detection models should be scalable to accommodate larger datasets and evolving requirements.
Security	Ensure that all sensitive data, especially health-related information, is encrypted.

3. System Design

It consists of a set of Diagrams that describe the entire project and a working mechanism.

3.1 System Architecture

A **system architecture** is the Conceptual model that defines the structure, behavior, and more views of a system. An architecture description is a formal description and representation of a system, organized in a way that supports reasoning about the structures and behaviors of the system.

- Diagram in **Figure (1)** describes the structure of the system and all the functions.

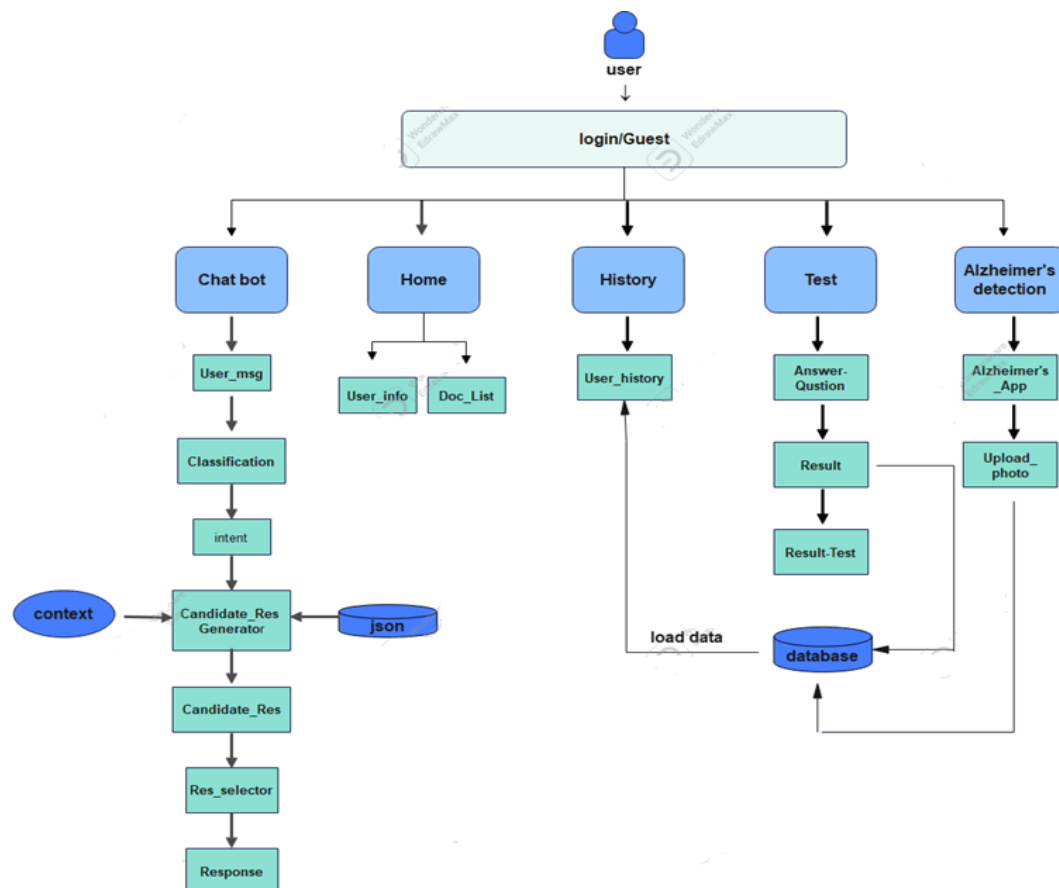


Figure (1) System Architecture

3.2 Use Cases Diagram

The Use Case in **Figure (2)** contains three actors **User**, **Doctor**, and **AI Doctor**, each actor has some of the process he performs, such as that the User can login and chose a doctor and the talk with the Chabot. And that the Doctor can login and be responsible for choosing the date for patients.

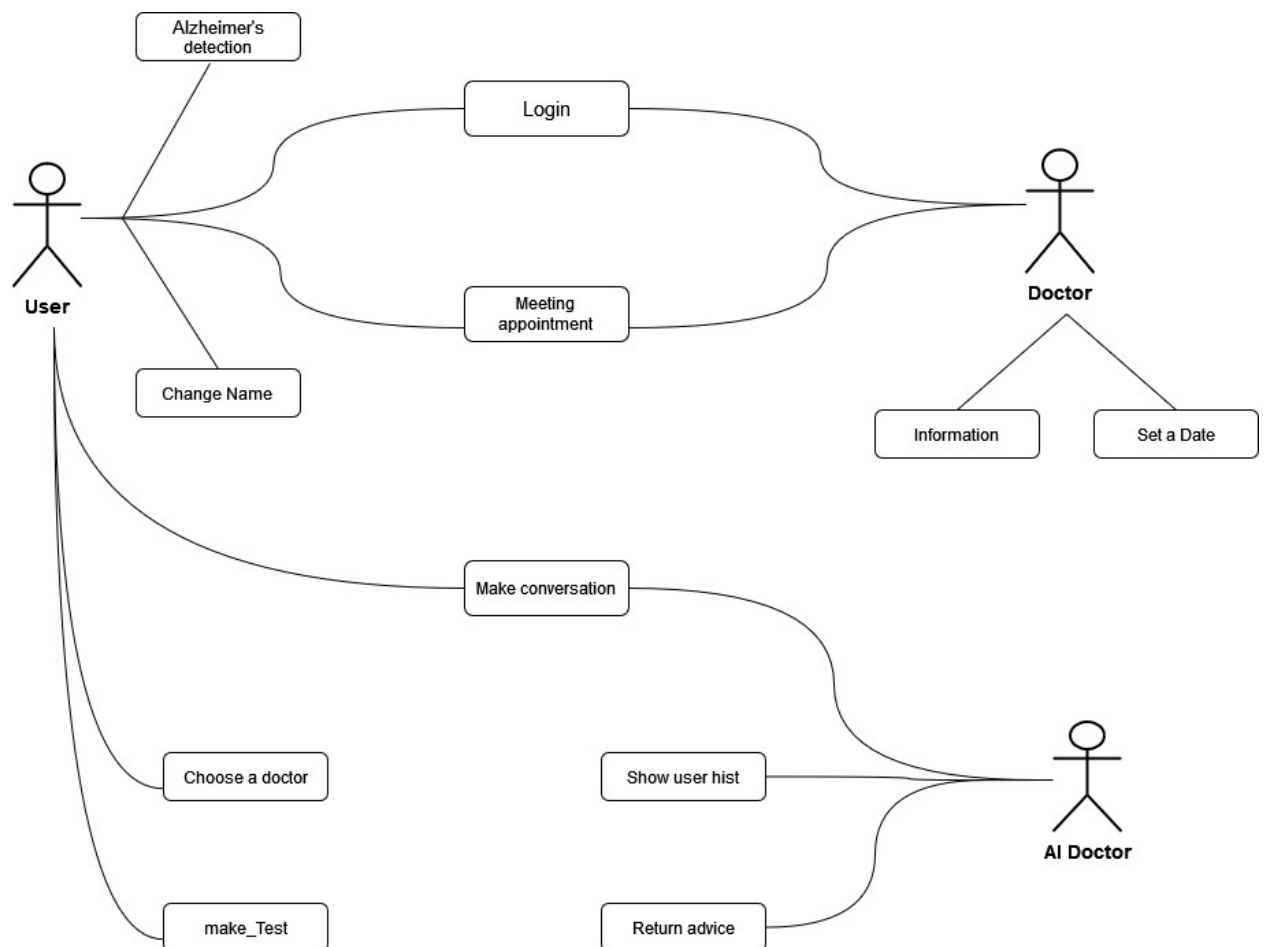


Figure (2) Use Cases Diagram

3.3 Activity Diagrams

The Activity Diagram in **Figure (3)**, **Figure (4)**, and **Figure (5)** explains the operations that are carried out on each process, and I see the validation, the powers, and how the system will work, In the case of success, one thing is done, and in case of failure, another thing is done.

3.3.1 Login Diagram

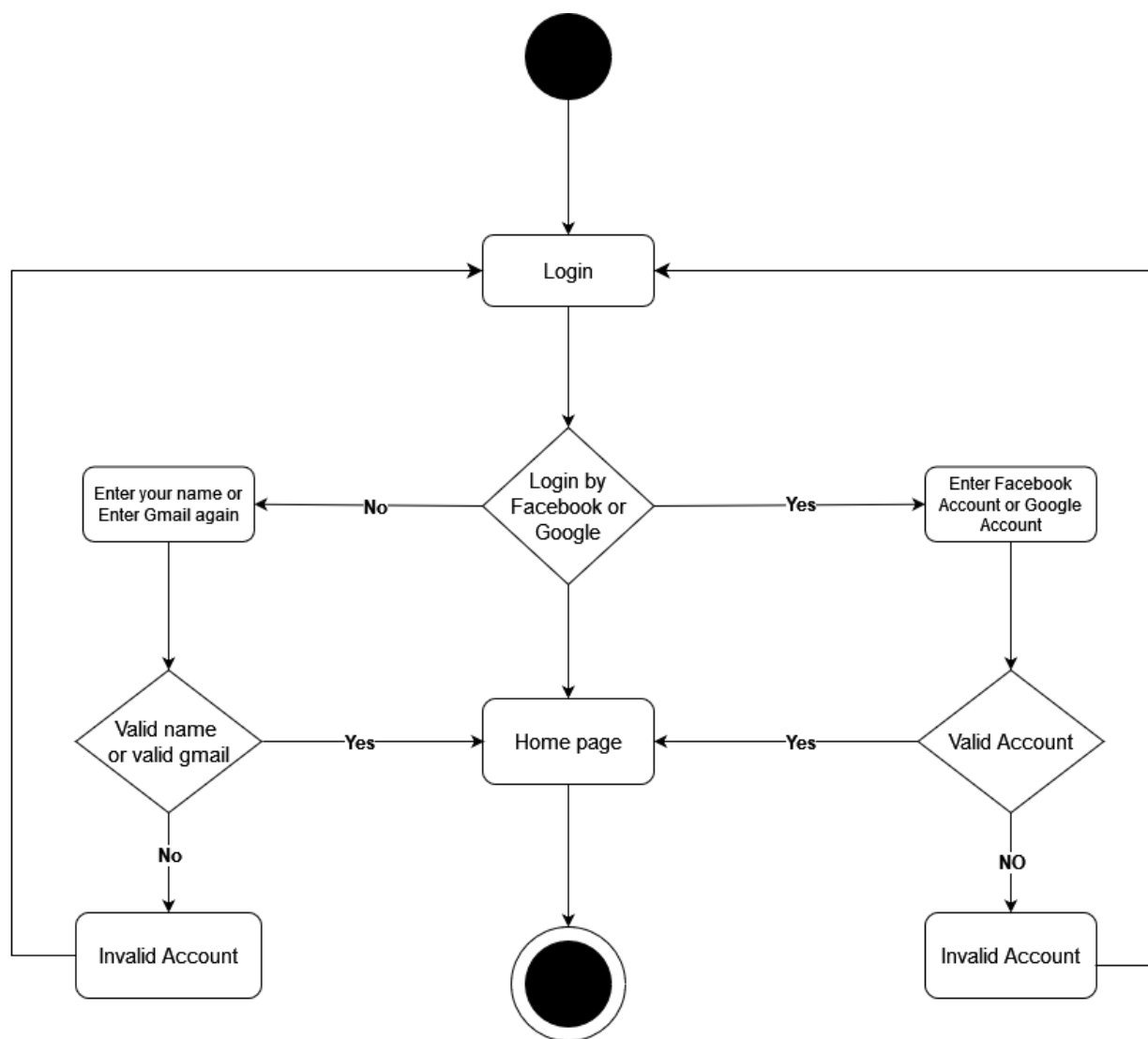


Figure (3) Login Diagram

3.3.2 User Diagram

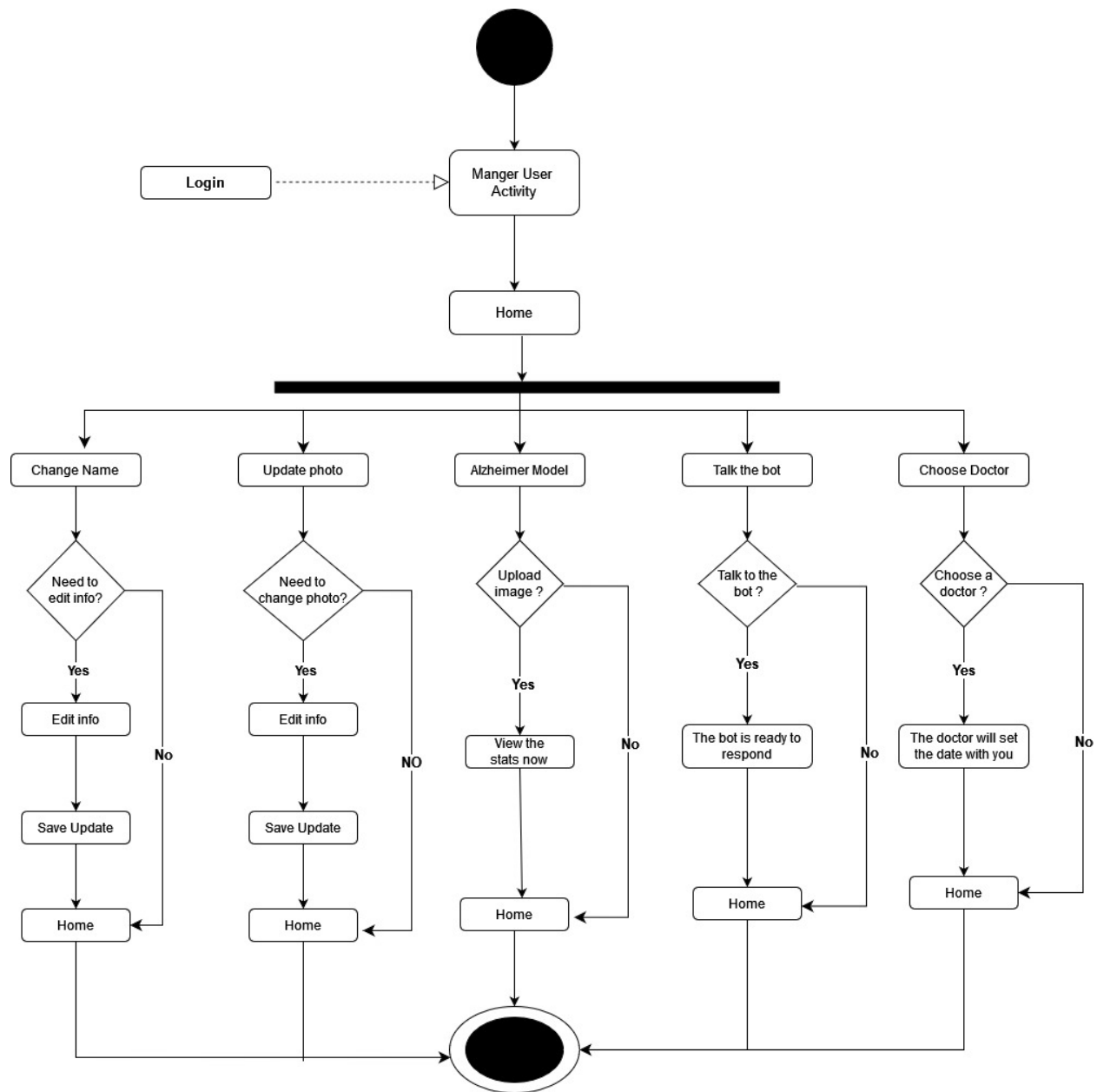


Figure (4) User Diagram

3.3.3 Doctor Diagram

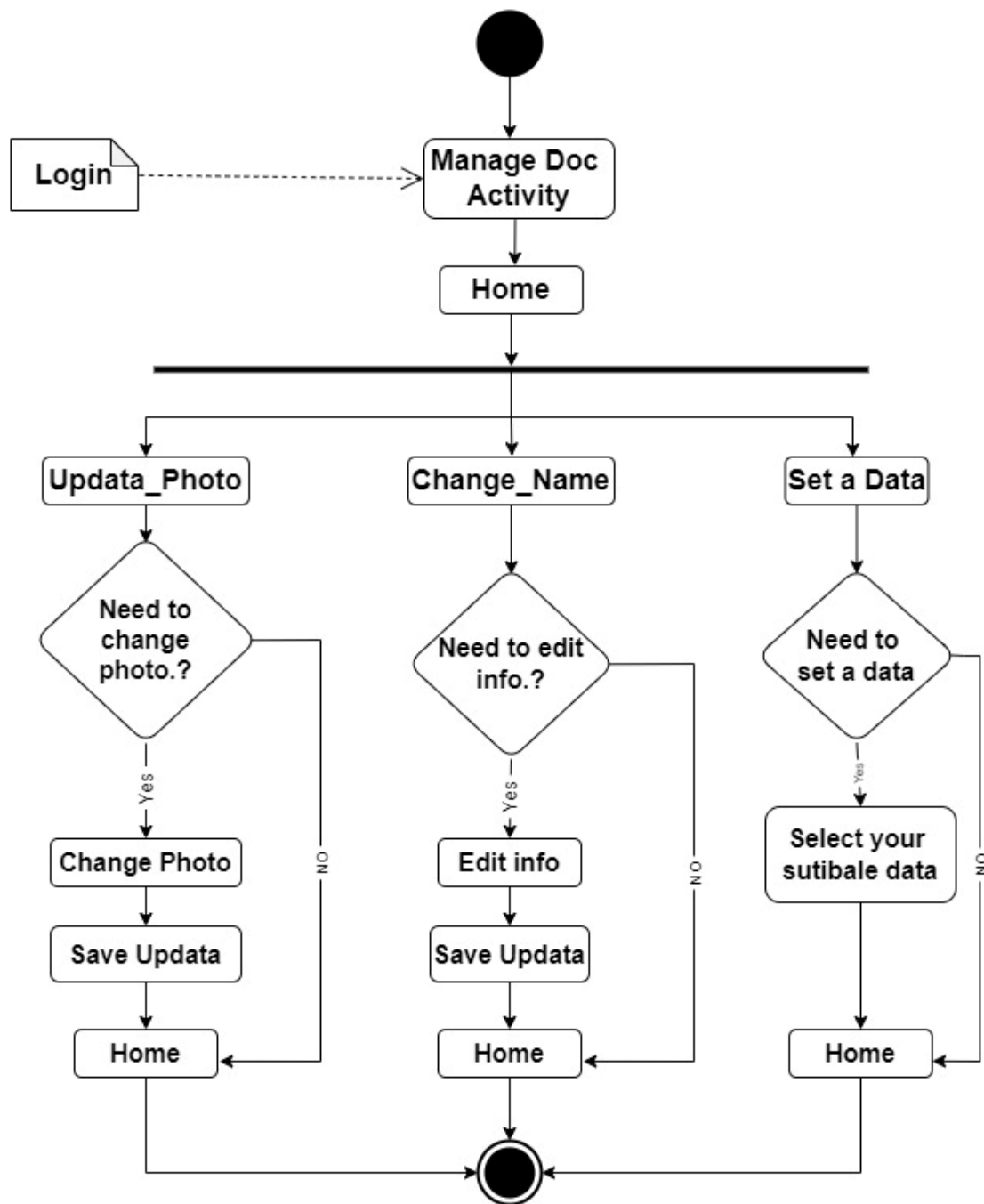


Figure (5) Doctor Diagram

3.4 Class Diagrams

The Class Diagram in **Figure (6)** is the one that describes the classes that the database, is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (and methods), and the relationships among objects.

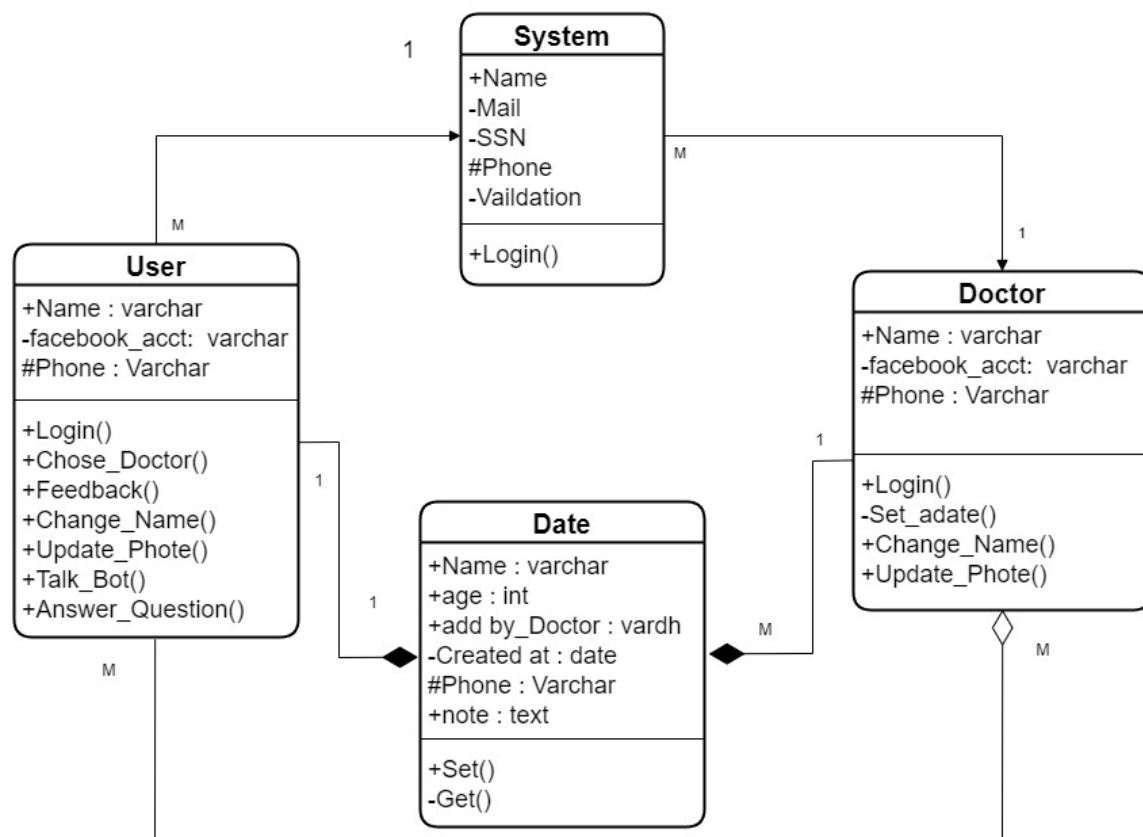


Figure (6) Class Diagrams

3.5 Sequence Diagrams

Sequence diagram, in **Figure (7)**, **Figure (8)**, which is that I see processes behind each other for each action process that happens to it, and this will have a complete and comprehensive description of any activity that occurs in the system. They capture the interaction between objects in the context of a collaboration. Sequence Diagrams are time focus and they show the order of the interaction visually by using the vertical axis of the diagram to represent time what messages are sent and when.

3.5.1 User Diagram

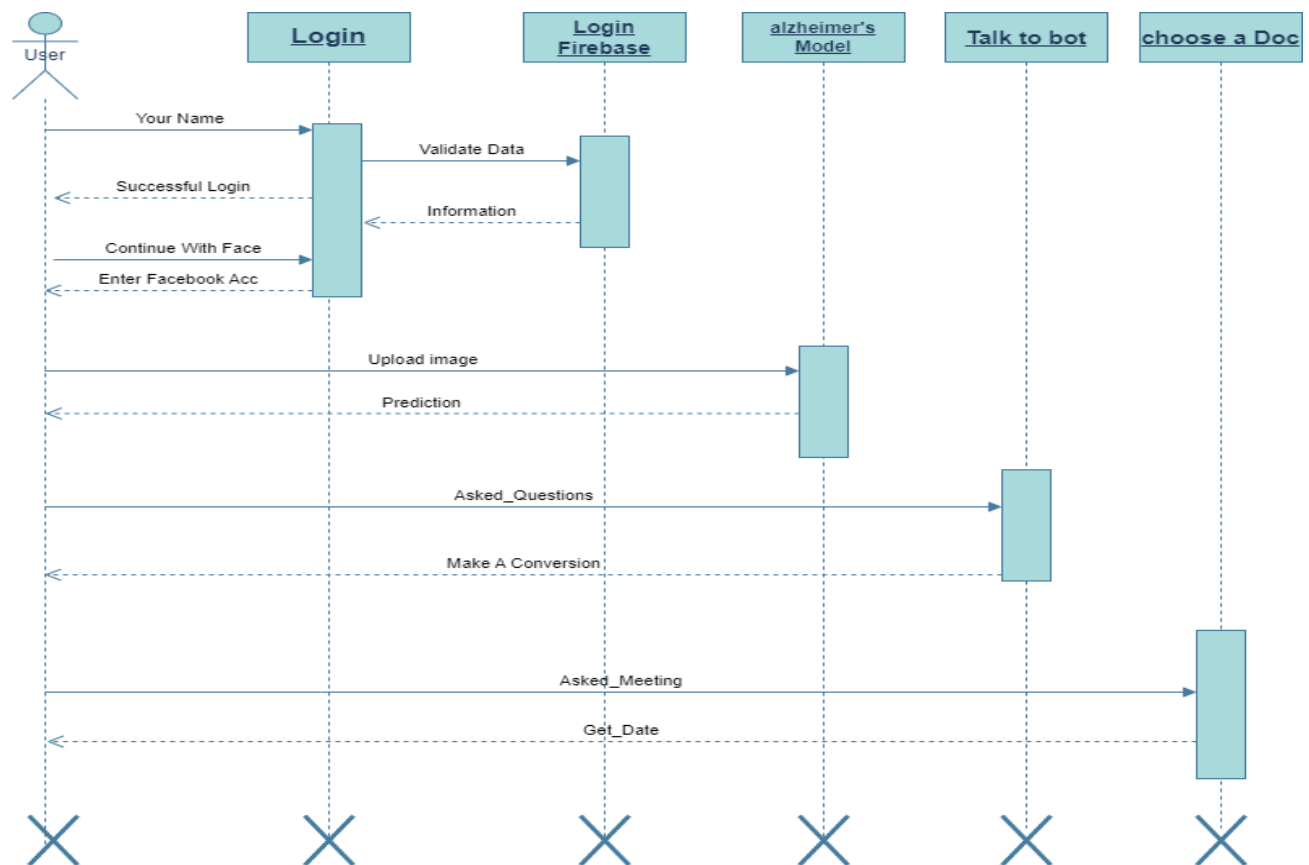


Figure (7) User Sequence Diagram

3.5.2 Doctor Diagram

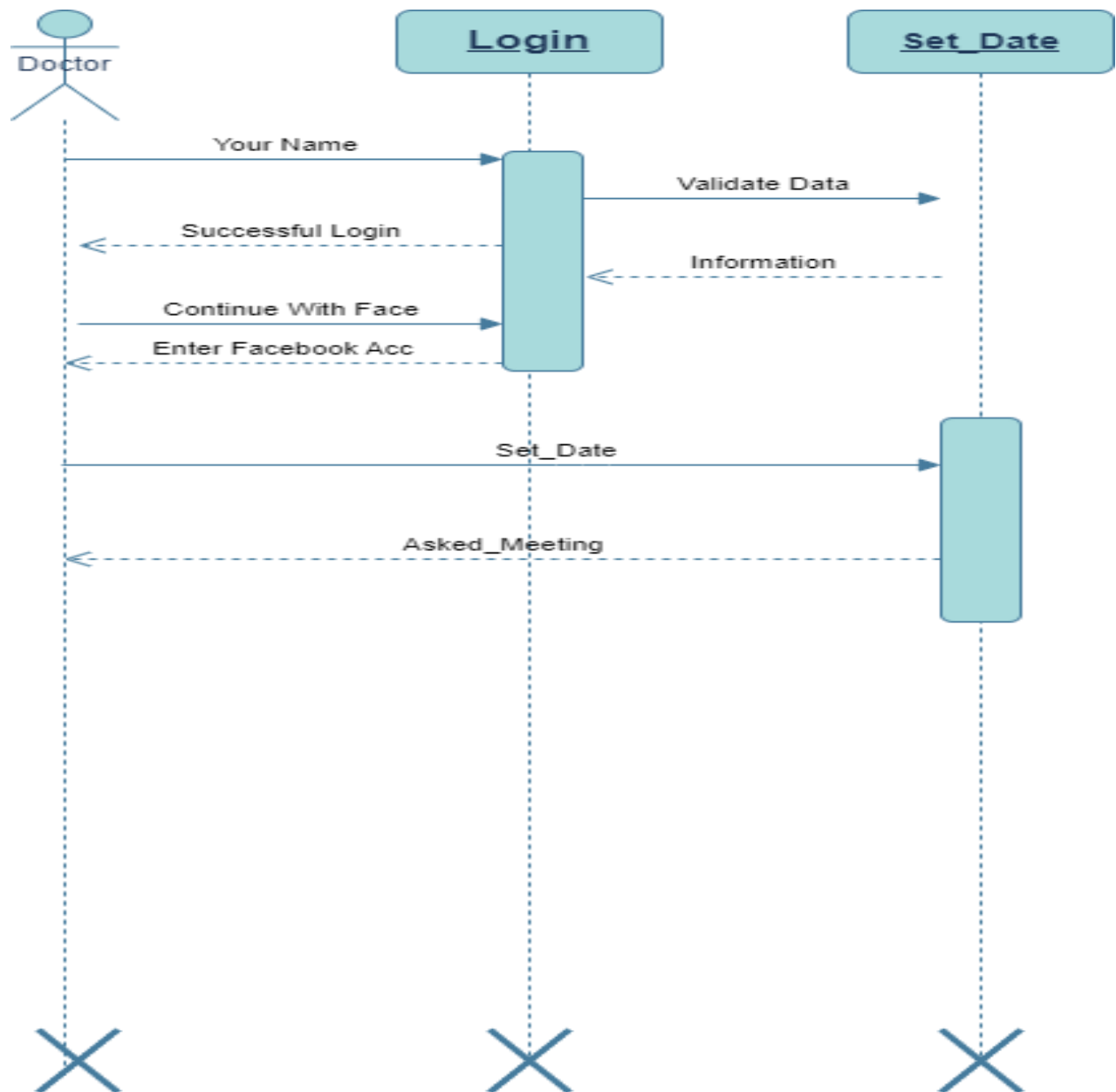


Figure (8) Doctor Sequence Diagram

3.6 Gantt chart

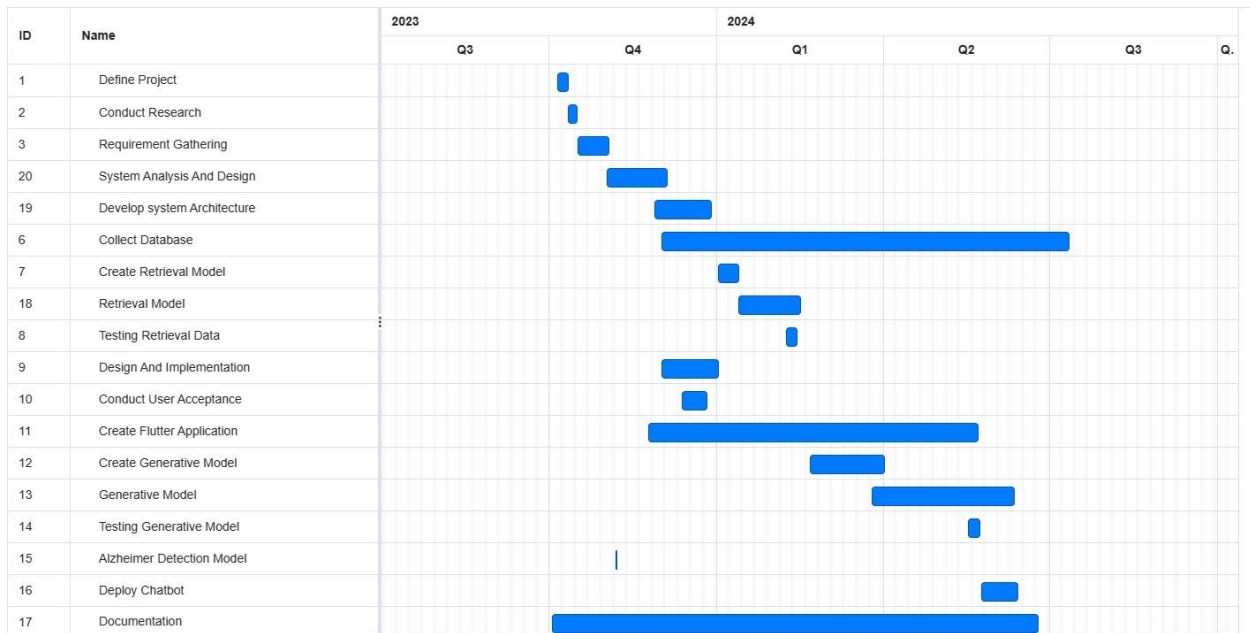


Figure (9) Gantt chart

3.7 Data Flow Diagram (DFD)

Can be divided into logical and physical. The logical data flow diagram describes a flow of data through a system to perform certain functionality of a business. The physical data flow diagram describes the implementation of the logical data flow.

3.7.1 Context Diagram (Level 0)

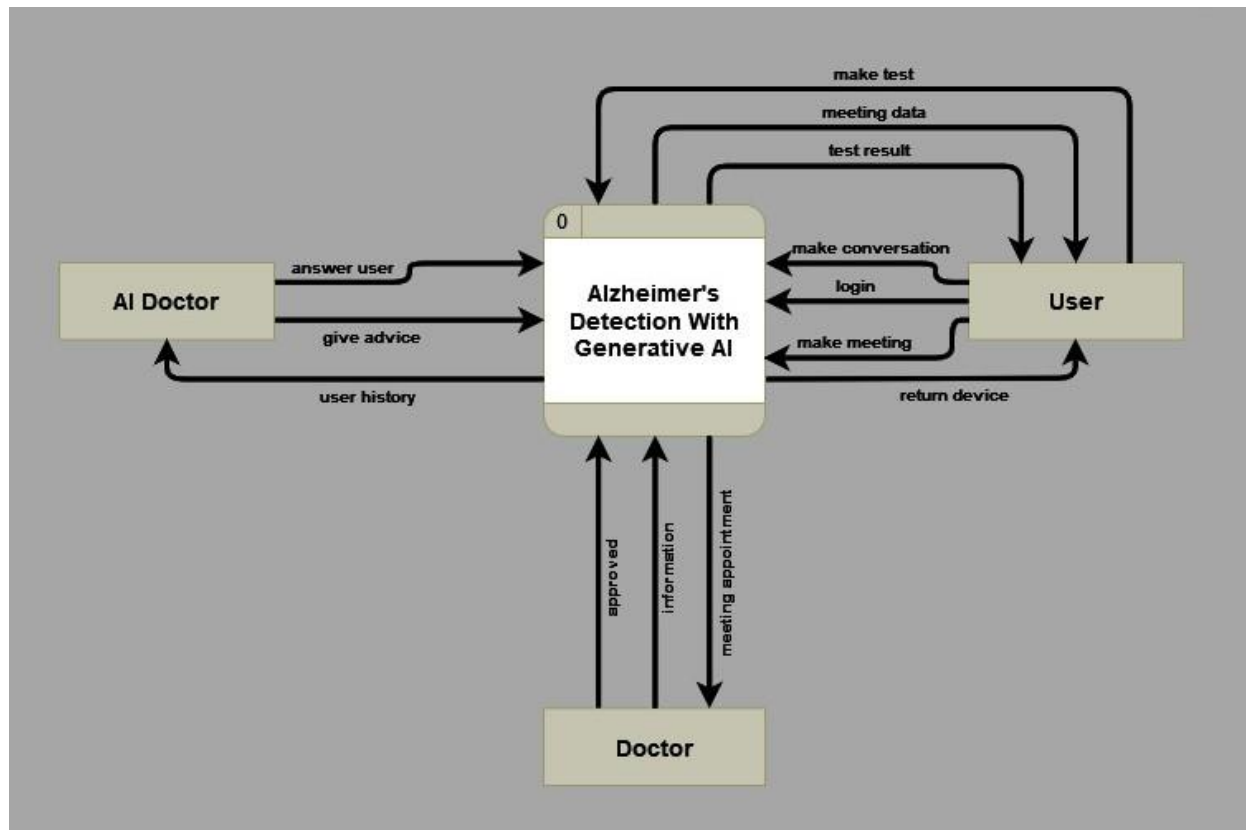


Figure (10) Context Diagram (Level 0)

"Data flow diagrams (DFDs) serve as visual representations depicting the intricate flow of data within a business information system. In the context of an Alzheimer's detection model empowered by generative AI, DFDs outline the processes involved in transferring, analyzing, and interpreting data, from initial input through various stages, ultimately culminating in the identification and reporting of potential markers for Alzheimer's disease."

3.7.2 DFD (Level 1)

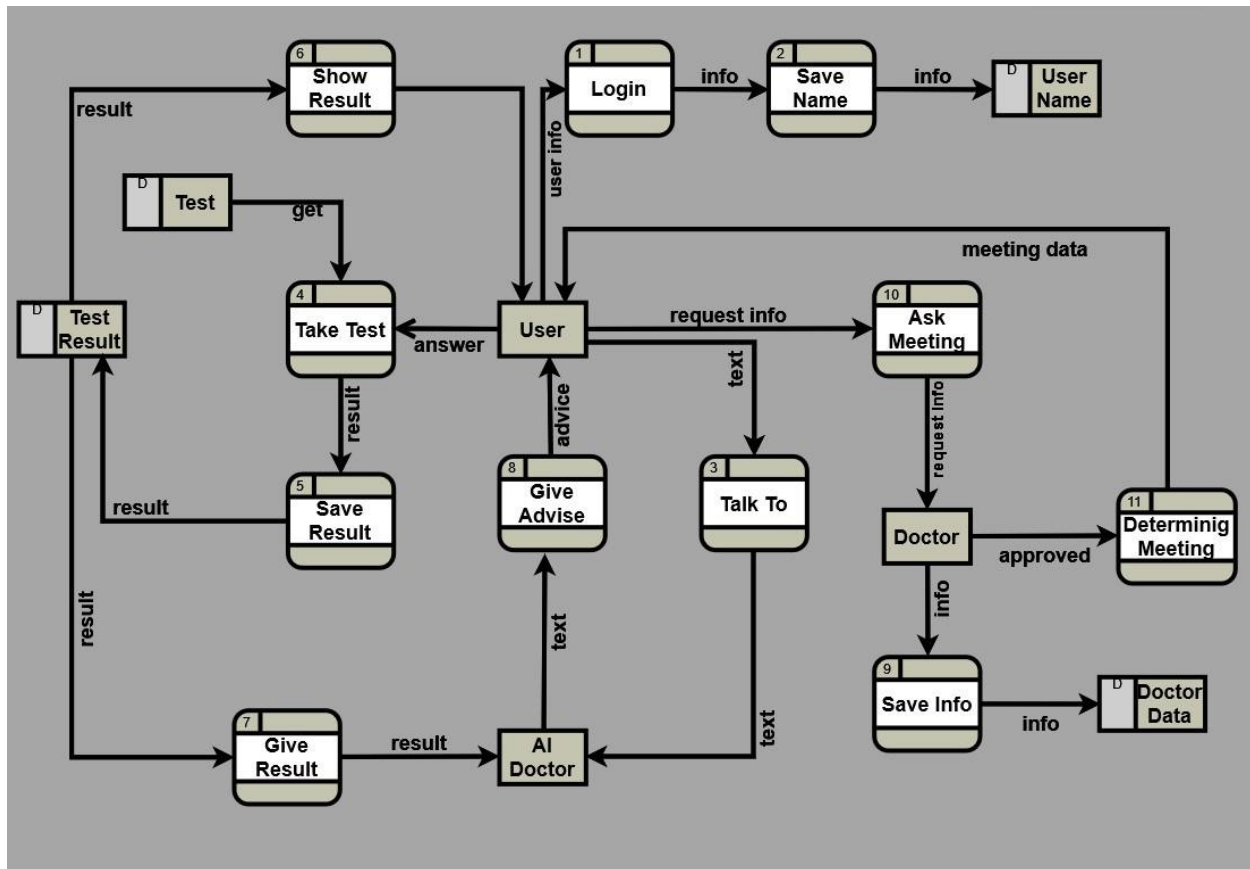


Figure (11) DFD (Level 1)

3.8 State Diagram

State Diagrams in **Figure (12)** is a type of diagram used in computer science and related fields to describe the behavior of systems. State diagrams require that the system described is composed of a finite number of states; sometimes, this is indeed the case, while at other times this is a reasonable abstraction. Many forms of state diagrams exist, which differ slightly and have different semantics.

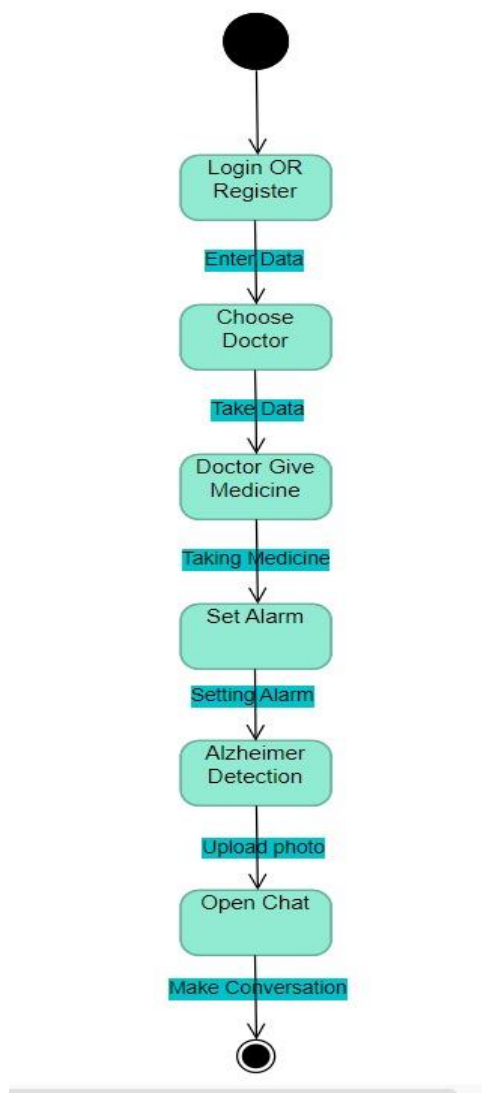


Figure (12) State Diagram

3.9 Entity Relationship Diagram (ERD)

ERD (Entity-Relationship Diagram) in **Figure (13)** is a visual representation of entities and their relationships to each other. It is used to describe the data and relationships between entities in a database or application. An ERD represents Users, Tests questions, Answer Tests, Topic, and Results entities. These entities have relationships with each other such as a User taking a test question, or a User having many Results. The ERD also shows the attributes and data types of each entity, such as the User's name and email.

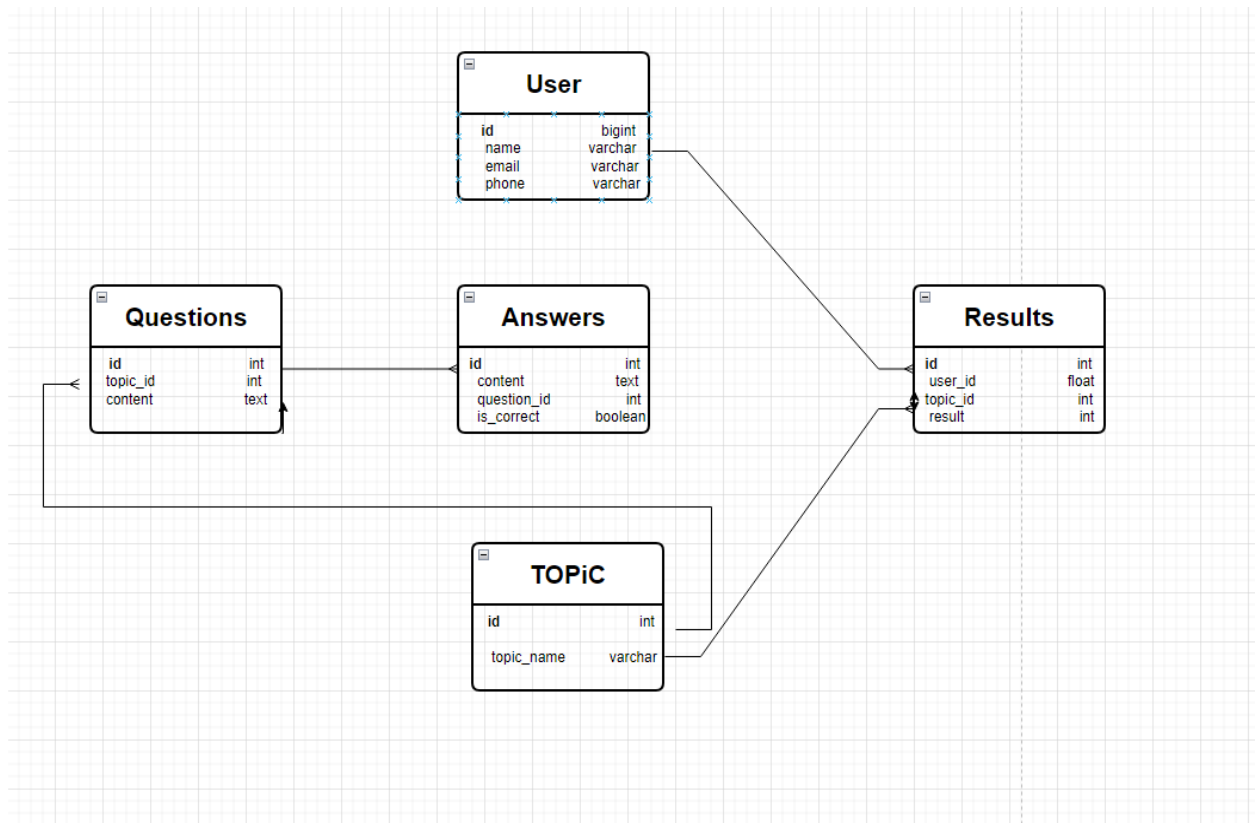


Figure (13) ERD

4. Technologies and Tools

Tools
Programming language (Python)
Machine Learning (ML)
Deep Learning (DL)
Natural Language Processing (NLP)
Natural Language Understanding (NLU)
Mobile Development
Flask
Firebase and local database

5. Summary

In this chapter We talk about the complete analysis of all the functions that the application performs, and this analysis is represented in a set of diagrams that in turn make it easier for us to understand and implement the project and the ease of modification to it. Each diagram is different from the rest and represents the functions and how the application performs them in a sequential manner. We also know who the owners are the interest of that application, which is the doctor, the user, and the enabler of the developer because he has an interest because of the return on him from using the application. One of the most important parts in this chapter is knowing the requirements of this project, which is represented in the required functions that the project or application does not exist except in their presence and non-required functions. Also, we have become fully aware of the requirements of both the user and the system requirements as well.

Chapter 3 → Alzheimer's Model (4 class of Images) Analysis & Visualization

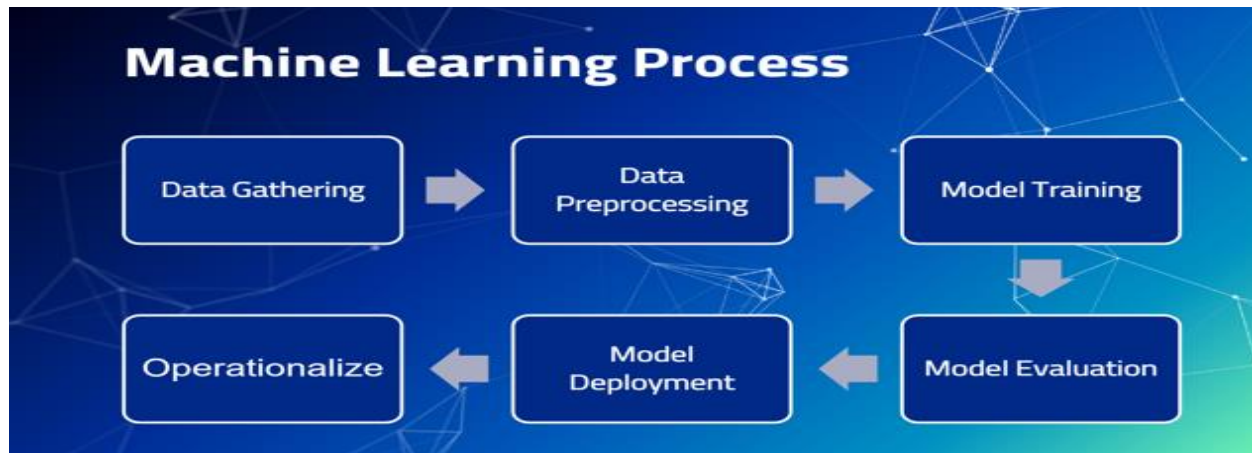


Figure (14) Machine Learning Process

1. Libraries

```
In [1]: import os
import cv2
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from keras.preprocessing.image import ImageDataGenerator
import keras
from keras.callbacks import EarlyStopping, ModelCheckpoint
import tensorflow as tf
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
from tqdm import tqdm
from imblearn.over_sampling import SMOTE
```

2. Dataset Gathering

-This data show four classes: -

- 1- Alzheimer's disease
- 2- Cognitively normal
- 3- Early mild cognitive impairment
- 4- Late mild cognitive impairment

-The data consists of two columns: -

- images
- labels

3. Read Data

```
]:
images = []
labels = []
for subfolder in tqdm(os.listdir('/kaggle/input/fdata-adni-dataset')):
    subfolder_path = os.path.join('/kaggle/input/fdata-adni-dataset', subfolder)
    for folder in os.listdir(subfolder_path):
        subfolder_path2 = os.path.join(subfolder_path, folder)
        for image_filename in os.listdir(subfolder_path2):
            image_path = os.path.join(subfolder_path2, image_filename)
            images.append(image_path)
            labels.append(folder)
df = pd.DataFrame({'image': images, 'label': labels})
df
```

-Show Data Frame: -

A Data Frame is a two-dimensional, tabular data structure commonly used in data analysis and statistics.

-Found 33984 validated image filenames belonging to 4 classes.

-We have in our data 33984 rows and 2 columns as shown in the figure below.

Out[2]:

	image	label
0	/kaggle/input/fdata-adni-dataset/AugmentedAlzh...	LMCI
1	/kaggle/input/fdata-adni-dataset/AugmentedAlzh...	LMCI
2	/kaggle/input/fdata-adni-dataset/AugmentedAlzh...	LMCI
3	/kaggle/input/fdata-adni-dataset/AugmentedAlzh...	LMCI
4	/kaggle/input/fdata-adni-dataset/AugmentedAlzh...	LMCI
...	...	...
33979	/kaggle/input/fdata-adni-dataset/AugmentedAlzh...	CN
33980	/kaggle/input/fdata-adni-dataset/AugmentedAlzh...	CN
33981	/kaggle/input/fdata-adni-dataset/AugmentedAlzh...	CN
33982	/kaggle/input/fdata-adni-dataset/AugmentedAlzh...	CN
33983	/kaggle/input/fdata-adni-dataset/AugmentedAlzh...	CN

33984 rows × 2 columns

4. Visualization & Analysis of Dataset (data preprocessing)

4.1 Data preprocessing

Data preprocessing is a crucial step in the data analysis and machine learning pipeline. It involves cleaning and transforming raw data into a format that is suitable for analysis or model training.

4.2 Analyzing & Visualization

- The Number of Samples for Each Class:

```
In [5]: # Number of photos in each class
df['label'].value_counts()

Out[5]:
label
Early mild cognitive impairment    9680
Late mild cognitive impairment     8960
Alzheimer_s disease                8960
Cognitively normal                 6464
Name: count, dtype: int64

In [6]: #Visualizing class distribution using count plot and pie chart
plt.figure(figsize=(30, 15))

# Count Plot
plt.subplot(1, 2, 1)
ax = sns.countplot(x='label', data=df, palette='Set1')
ax.set_xlabel("Class", fontsize=20)
ax.set_ylabel("Count", fontsize=20)
plt.title('The Number Of Samples For Each Class Train Set', fontsize=20)
plt.grid(True)
plt.xticks(rotation=45)

# Pie Chart
plt.subplot(1, 2, 2)
plt.pie(df['label'].value_counts(), autopct='%1.2f%%', labels=df['label'].value_counts().index, radius=1)
plt.title('The Ratio Of Samples For Each Class Data Set', fontsize=20)

plt.show()
```

4.3 Split Data

- 80% train
- 10% test
- 10% validation

```
In [7]: X_train, X_test1, y_train, y_test1 = train_test_split(df['image'], df['label'], test_size=0.2, random_state=42, shuffle=True, stratify=df['label'])
X_val, X_test, y_val, y_test = train_test_split(X_test1, y_test1, test_size=0.5, random_state=42, shuffle=True, stratify=y_test1)
df_train = pd.DataFrame({'image': X_train, 'label': y_train})
df_test = pd.DataFrame({'image': X_test, 'label': y_test})
df_val = pd.DataFrame({'image': X_val, 'label': y_val})
```

4.3.1 In Train

```
In [8]: pd.DataFrame(y_train).value_counts()

Out[8]:
label
Early mild cognitive impairment    7680
Alzheimer_s disease               7168
Late mild cognitive impairment    7168
Cognitively normal               5171
Name: count, dtype: int64
```

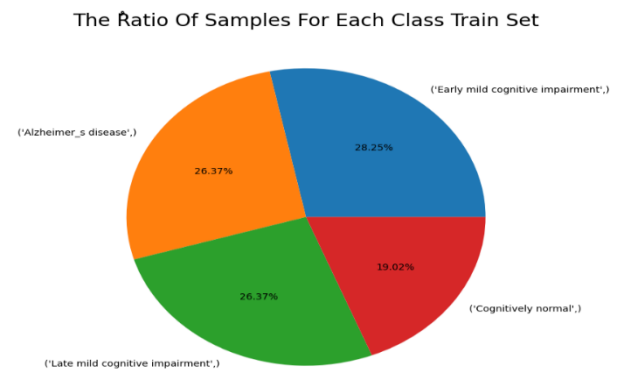
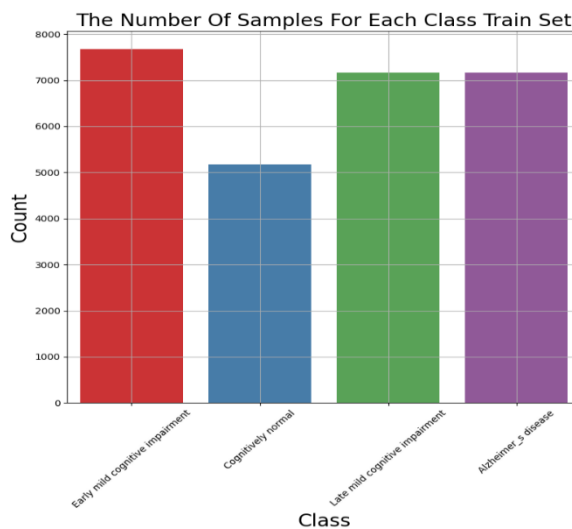


Figure (15) Train Data

4.3.2 In Test

```
In [10]: pd.DataFrame(y_test).value_counts()

Out[10]:
label
Early mild cognitive impairment    960
Alzheimer_s disease               896
Late mild cognitive impairment    896
Cognitively normal               647
Name: count, dtype: int64
```

Chapter 3 → Alzheimer's Model (4 class of Images) Analysis & Visualization

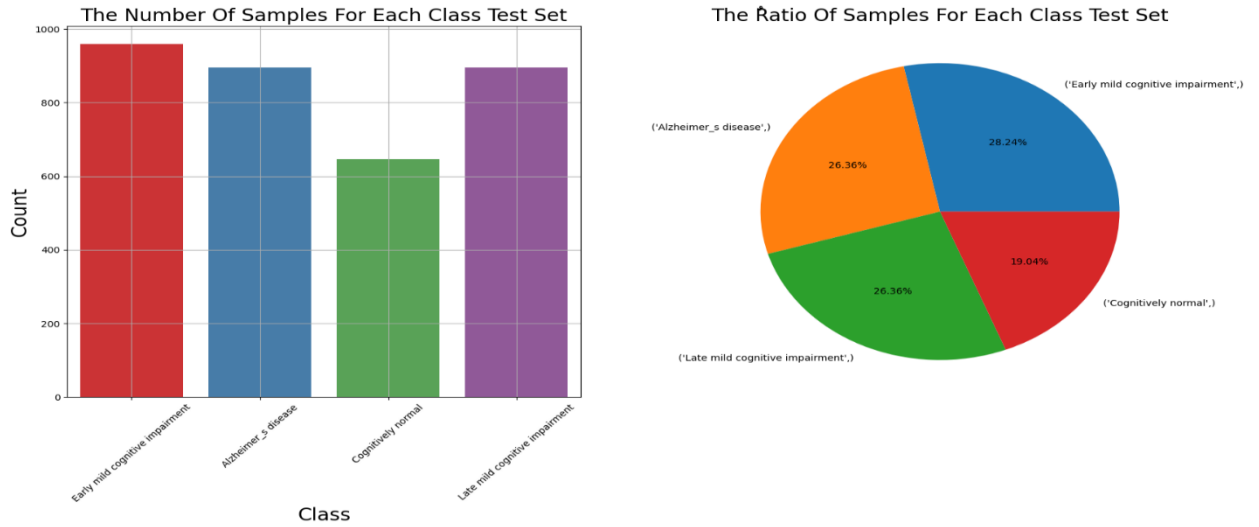


Figure (16) Test Data

4.3.3 In Validation

```
In [12]: pd.DataFrame(y_val).value_counts()

Out[12]:
label
Early mild cognitive impairment    960
Alzheimer_s disease              896
Late mild cognitive impairment    896
Cognitively normal               646
Name: count, dtype: int64
```

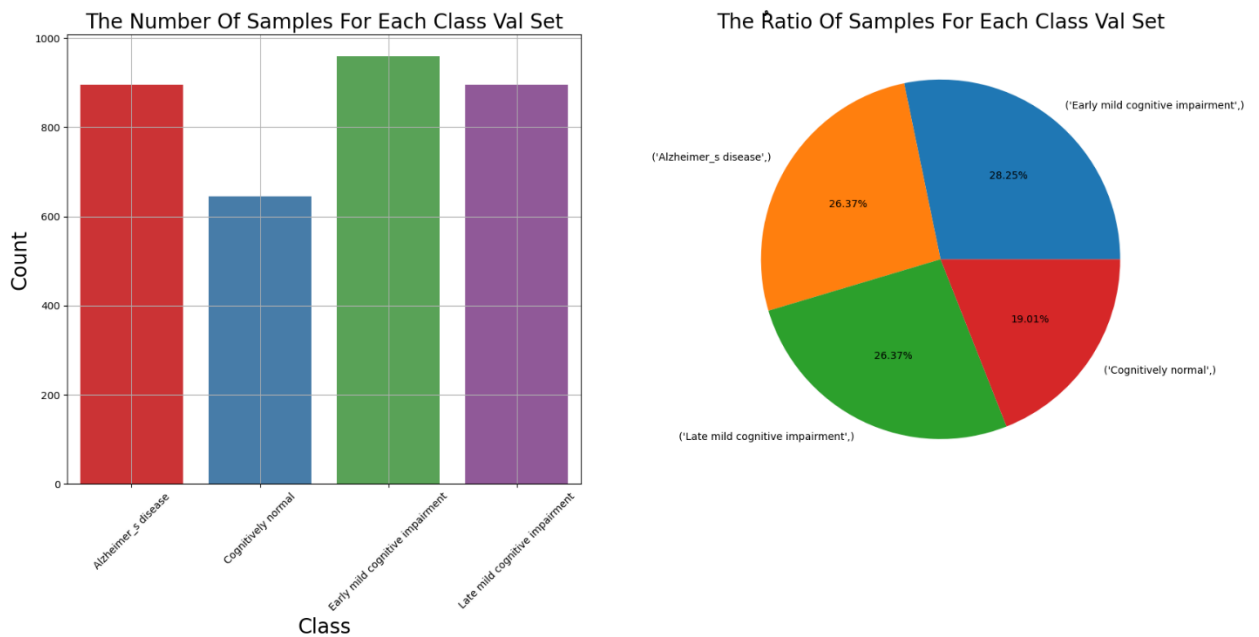


Figure (17) Validation Data

4.5 Data Augmentation

Data augmentation is a technique commonly used in machine learning and deep learning to artificially increase the size of a training dataset by applying various transformations to the existing data. Common image augmentations include:

- Rotation
- Flip (horizontal and vertical)
- Zooming
- Scaling
- Cropping (etc.)

```
image_size = (224,224)
batch_size = 32
datagen = ImageDataGenerator(
    preprocessing_function= tf.keras.applications.resnet.preprocess_input,
    rescale=1./255,
    horizontal_flip=True
)
train_generator = datagen.flow_from_dataframe(
    df_train,
    x_col='image',
    y_col='label',
    target_size=image_size,
    batch_size=batch_size,
    class_mode='categorical',
    shuffle=True
)
test_generator = datagen.flow_from_dataframe(
    df_test,
    x_col='image',
    y_col='label',
    target_size=image_size,
    batch_size=batch_size,
    class_mode='categorical',
    shuffle=False
)
val_generator = datagen.flow_from_dataframe(
    df_val,
    x_col='image',
    y_col='label',
    target_size=image_size,
    batch_size=batch_size,
    class_mode='categorical',
    shuffle=True
)
```

Now we show four classes an image: -

```
In [15]:
NUM_IMAGES_PER_CLASS = 8
NUM_CATEGORIES=4
fig = plt.figure(1, figsize=(15, 15))
grid = ImageGrid(fig, 111, nrows_ncols=(NUM_CATEGORIES, NUM_IMAGES_PER_CLASS), axes_pad=0.05)
i = 0

for category_id, category in enumerate(class_num):
    for filepath in df['image'][df['label'] == category][:NUM_IMAGES_PER_CLASS]:
        ax = grid[i]
        img = cv2.imread(filepath)
        img=cv2.resize(img, (224, 224))
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        ax.imshow(img / 255)
        ax.axis('off')

    if i % NUM_IMAGES_PER_CLASS == NUM_IMAGES_PER_CLASS - 1:
        ax.text(250, 80, category, verticalalignment='center', size=15)

    i += 1

plt.show()
```

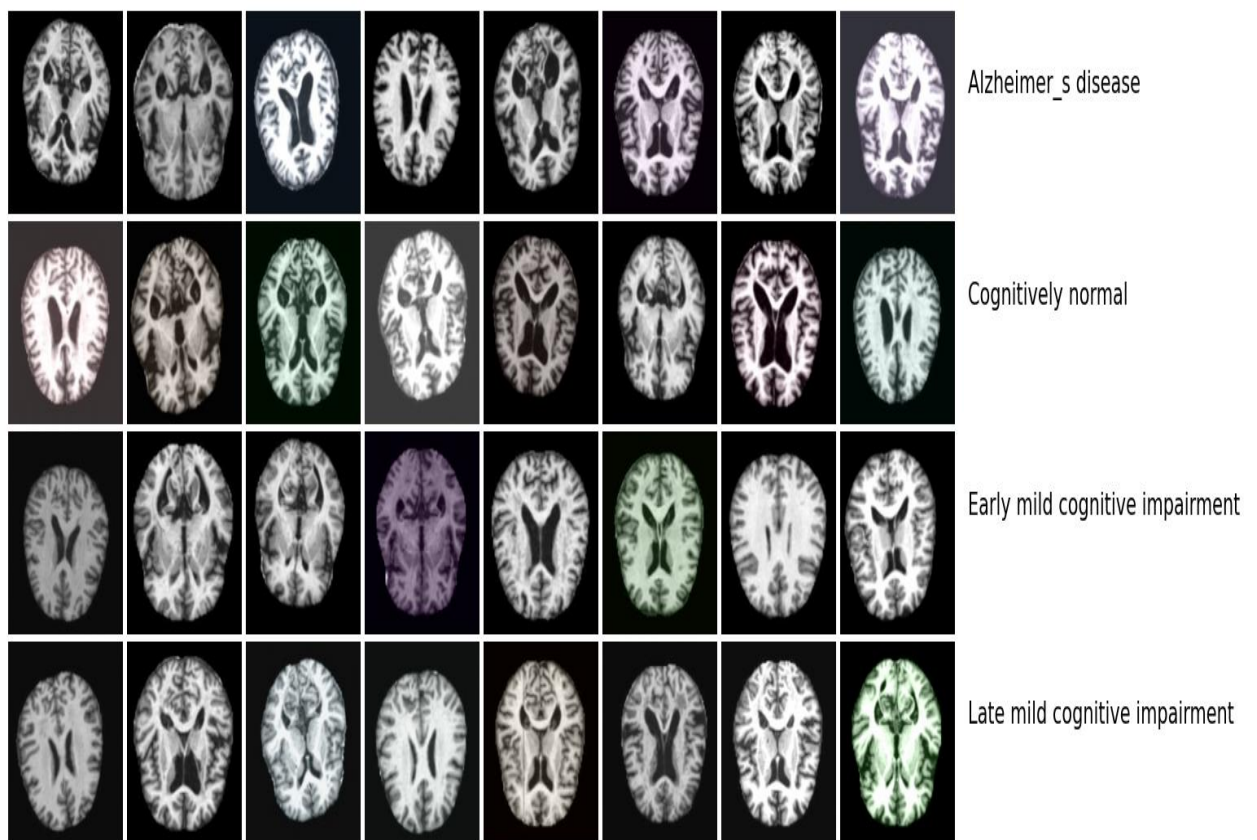


Figure (18) Data Visualization

5. Model training

Convolutional Neural Networks (CNNs) revolutionize medical imaging, offering swift and accurate disease diagnosis, even in emergency situations lacking human expertise. By analyzing X-ray images, CNNs efficiently detect and classify diseases like Alzheimer's, reducing diagnostic errors and saving valuable time. Traditional diagnostic methods, often time-consuming and exhausting for professionals, are replaced by CNNs, enhancing efficiency and precision in medical image classification. This advancement in artificial intelligence, particularly in deep learning algorithms like CNNs, transforms disease diagnosis by enabling rapid and precise identification of abnormalities, ultimately improving patient outcomes, and saving lives.

We Use Four Models

- Convolutional Neural Network (CNN).
- ResNet152V2
- VGG16
- DenseNet201
- MobileNetV2

5.1 CNN

A convolutional neural network (CNN) is a powerful type of deep learning for image processing often used in computer vision that includes image and video recognition. CNN consists of a set of layers, namely, the input layer, the output layer, and a hidden layer consisting of multiple convolutional layers, pooling layers, and fully connected layers. These layers will be explained in the following:

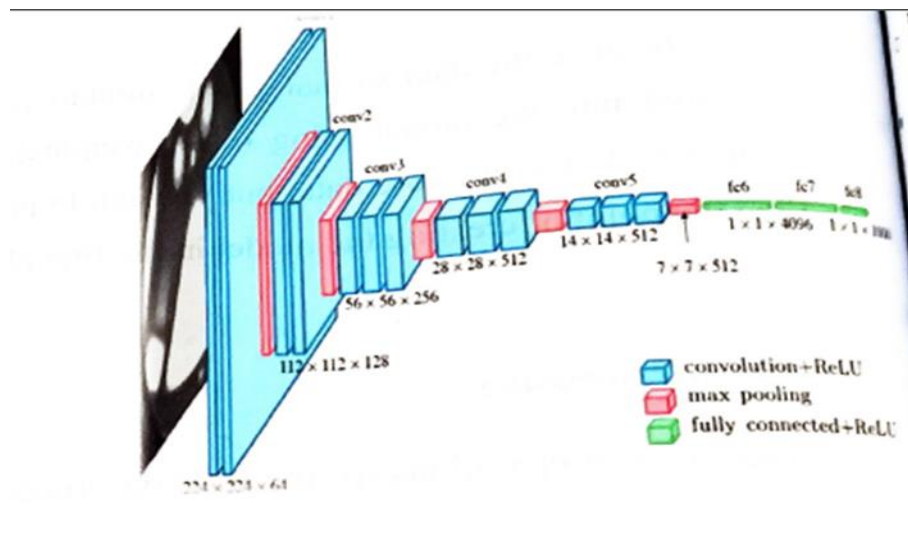


Figure (19) CNN Model

5.1.1 CNN build steps.

® Using Keres Libraries

1. Initialize the CNN
2. Add a convolutional layer.
3. Pooling operation
4. Add more convolutional layers.
5. Flattening operation
6. Fully connected layer
7. Output layer (SoftMax)

5.1.2 Model Architecture.

```
In [8]:
model=keras.models.Sequential()
model.add(keras.layers.Conv2D(32, kernel_size=(3,3), strides=2, padding='same', activation='relu', input_shape=(224, 224, 3)))
model.add(keras.layers.MaxPool2D(pool_size=(2,2), strides=2, padding='same'))
model.add(keras.layers.Conv2D(64, kernel_size=(3,3), strides=2, activation='relu', padding='same'))
model.add(keras.layers.MaxPool2D((2,2), 2, padding='same'))
model.add(keras.layers.Conv2D(128, kernel_size=(3,3), strides=2, activation='relu', padding='same'))
model.add(keras.layers.MaxPool2D((2,2), 2, padding='same'))
model.add(keras.layers.Flatten())
model.add(keras.layers.Dense(1024, activation='relu'))
model.add(keras.layers.Dropout(0.3))
model.add(keras.layers.Dense(4, activation='softmax'))
model.summary()
```

5.1.3 Model Scratch.

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 88, 88, 32)	896
max_pooling2d (MaxPooling2D)	(None, 44, 44, 32)	0
conv2d_1 (Conv2D)	(None, 22, 22, 64)	18496
max_pooling2d_1 (MaxPooling2D)	(None, 11, 11, 64)	0
conv2d_2 (Conv2D)	(None, 6, 6, 128)	73856
max_pooling2d_2 (MaxPooling2D)	(None, 3, 3, 128)	0
flatten (Flatten)	(None, 1152)	0
dense (Dense)	(None, 1024)	1180672
dropout (Dropout)	(None, 1024)	0
dense_1 (Dense)	(None, 4)	4100

Total params: 1278020 (4.88 MB)
 Trainable params: 1278020 (4.88 MB)
 Non-trainable params: 0 (0.00 Byte)

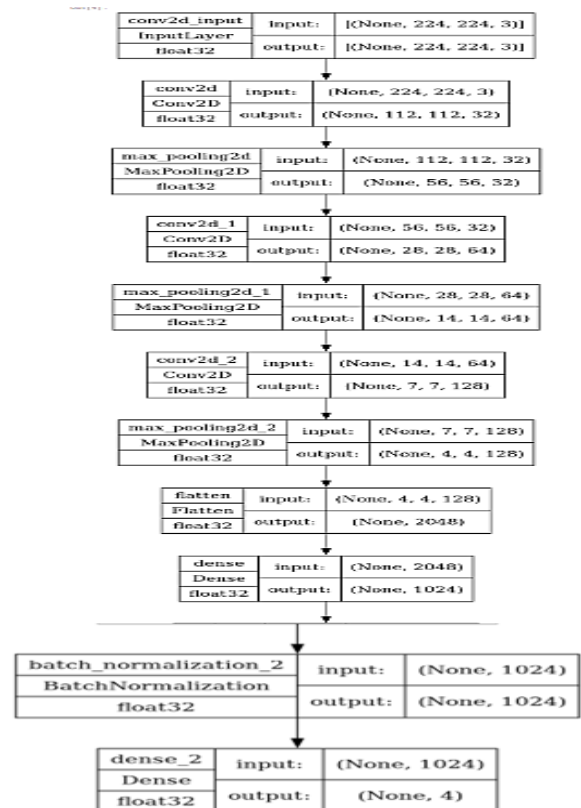


Figure (20) CNN Model Scratch

5.1.4 Training the model.

Now, let's discuss training and evaluation of the Convolutional neural network. We will be discussing this section in 3 steps:

1. The Compile models.
2. Fit model on training set
3. Evaluate Result

5.1.5 Model Performance

We have 50 Epoch.

```
850/850 [=====] - 73s 86ms/step - loss: 0.7385 - accuracy: 0.6588 - val_loss: 0.6849 - val_
accuracy: 0.6995
Epoch 3/50
850/850 [=====] - 68s 80ms/step - loss: 0.6337 - accuracy: 0.7149 - val_loss: 0.6012 - val_
accuracy: 0.7345
Epoch 4/50
850/850 [=====] - 71s 83ms/step - loss: 0.5548 - accuracy: 0.7515 - val_loss: 0.5623 - val_
accuracy: 0.7575
Epoch 5/50
850/850 [=====] - 74s 86ms/step - loss: 0.5002 - accuracy: 0.7830 - val_loss: 0.5378 - val_
accuracy: 0.7719
Epoch 6/50
850/850 [=====] - 72s 85ms/step - loss: 0.4455 - accuracy: 0.8042 - val_loss: 0.4769 - val_
accuracy: 0.7949
Epoch 7/50
850/850 [=====] - 72s 84ms/step - loss: 0.4167 - accuracy: 0.8197 - val_loss: 0.4767 - val_
accuracy: 0.7887
Epoch 8/50
850/850 [=====] - 72s 85ms/step - loss: 0.3778 - accuracy: 0.8392 - val_loss: 0.4674 - val_
accuracy: 0.8014
Epoch 9/50
850/850 [=====] - 74s 87ms/step - loss: 0.3498 - accuracy: 0.8517 - val_loss: 0.4790 - val_
accuracy: 0.7946
Epoch 10/50
850/850 [=====] - 74s 86ms/step - loss: 0.3273 - accuracy: 0.8543 - val_loss: 0.4322 - val_
accuracy: 0.8181
Epoch 11/50
850/850 [=====] - 73s 86ms/step - loss: 0.3045 - accuracy: 0.8723 - val_loss: 0.4047 - val_
accuracy: 0.8467
Epoch 12/50
850/850 [=====] - 74s 87ms/step - loss: 0.2208 - accuracy: 0.9119 - val_loss: 0.3948 - val_
accuracy: 0.8467
Epoch 13/50
850/850 [=====] - 70s 83ms/step - loss: 0.2063 - accuracy: 0.9167 - val_loss: 0.4000 - val_
accuracy: 0.8514
Epoch 14/50
850/850 [=====] - 75s 89ms/step - loss: 0.2026 - accuracy: 0.9186 - val_loss: 0.3865 - val_
accuracy: 0.8543
Epoch 15/50
850/850 [=====] - 74s 87ms/step - loss: 0.1843 - accuracy: 0.9258 - val_loss: 0.4060 - val_
accuracy: 0.8579
Epoch 16/50
850/850 [=====] - 70s 82ms/step - loss: 0.1794 - accuracy: 0.9274 - val_loss: 0.4363 - val_
accuracy: 0.8499
Epoch 17/50
850/850 [=====] - 70s 82ms/step - loss: 0.1712 - accuracy: 0.9330 - val_loss: 0.4489 - val_
accuracy: 0.8529
Epoch 18/50
850/850 [=====] - 71s 84ms/step - loss: 0.1635 - accuracy: 0.9365 - val_loss: 0.4096 - val_
accuracy: 0.8593
Epoch 19/50
850/850 [=====] - 71s 84ms/step - loss: 0.1587 - accuracy: 0.9392 - val_loss: 0.4623 - val_
accuracy: 0.8576
Epoch 20/50
850/850 [=====] - 72s 85ms/step - loss: 0.1558 - accuracy: 0.9401 - val_loss: 0.4210 - val_
accuracy: 0.8573
Epoch 21/50
850/850 [=====] - 69s 81ms/step - loss: 0.1465 - accuracy: 0.9429 - val_loss: 0.4885 - val_
accuracy: 0.8573
```

5.1.6 Evaluate the Model

Out[11]:

	loss	accuracy	val_loss	val_accuracy
0	1.032727	0.512083	0.801055	0.623896
1	0.738479	0.657962	0.684912	0.699529
2	0.633742	0.714900	0.601222	0.734550
3	0.554841	0.751499	0.562320	0.757504
4	0.500154	0.782984	0.537781	0.771925
5	0.445505	0.804245	0.476933	0.794879
6	0.416744	0.819656	0.476716	0.788699
7	0.377834	0.839225	0.467358	0.801354
8	0.349776	0.851694	0.478952	0.794585
9	0.327270	0.864310	0.432190	0.818128
10	0.304531	0.872255	0.404680	0.828723
11	0.286358	0.880899	0.405497	0.839612
12	0.268824	0.888513	0.402413	0.836374
13	0.254610	0.895833	0.405649	0.831960
14	0.241516	0.901056	0.436632	0.832254
15	0.232143	0.905175	0.384417	0.847263
16	0.220755	0.911943	0.394776	0.846675
17	0.206254	0.916725	0.399999	0.851383
18	0.202556	0.918601	0.386534	0.854326
19	0.184269	0.925773	0.405992	0.857858
20	0.179424	0.927429	0.436312	0.849912
21	0.171233	0.932983	0.448881	0.852855
22	0.163545	0.936514	0.409629	0.859329
23	0.158747	0.939199	0.462341	0.857563
24	0.155844	0.940082	0.421049	0.857269
25	0.146466	0.942877	0.488467	0.857269

```
In [12]: plt.figure(figsize=(15,10))
plt.subplot(1,2,1)
plt.plot(hist_['loss'],label='Train_Loss')
plt.plot(hist_['val_loss'],label='Validation_Loss')
plt.title('Train_Loss & Validation_Loss',fontsize=20)
plt.legend()
plt.subplot(1,2,2)
plt.plot(hist_['accuracy'],label='Train_Accuracy')
plt.plot(hist_['val_accuracy'],label='Validation_Accuracy')
plt.title('Train_Accuracy & Validation_Accuracy',fontsize=20)
plt.legend()
plt.show()
```

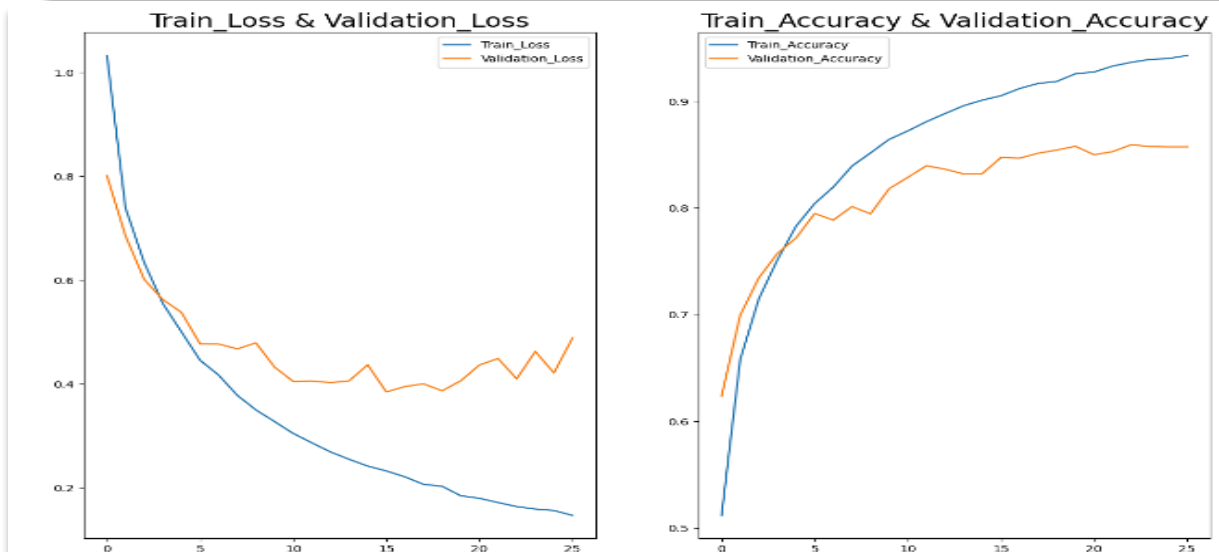


Figure (21) CNN Evaluation

5.1.7 Model score

```
In [13]: score, acc= model.evaluate(test_generator)
print('Test Loss =', score)
print('Test Accuracy =', acc)

107/107 [=====] - 30s 276ms/step - loss: 0.0745 - accuracy: 0.9791
Test Loss = 0.07452037185430527
Test Accuracy = 0.9791114926338196
```

5.1.8 Model Evaluation

5.1.8.1 Confusion Matrix

```
In [10]: CM = confusion_matrix(y_test,y_pred)
CM_percent = CM.astype('float') / CM.sum(axis=1)[:, np.newaxis]
sns.heatmap(CM_percent,fmt='g',center = True,cbar=False,annot=True,cmap='Blues',xticklabels=class_num,
            yticklabels=class_num)
CM

Out[16]:
array([[775,  5,  46,  70],
       [ 12, 632,  0,  3],
       [ 43,  3, 829, 85],
       [ 96,  5, 133, 662]])
```

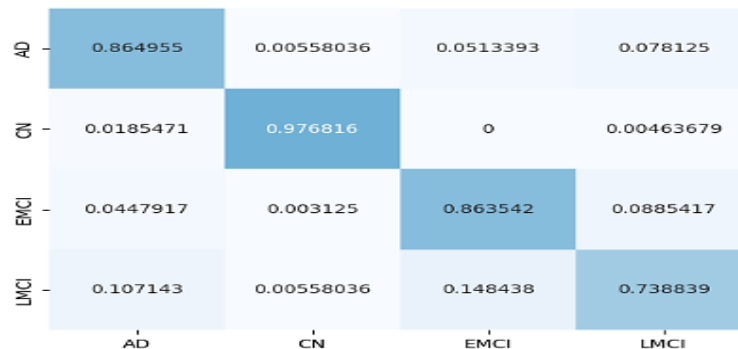


Figure (22) CNN Confusion Matrix

5.1.8.2 Classification Report

```
In [17]: ClassificationReport = classification_report(y_test,y_pred,target_names=class_num)
print('Classification Report is : ', ClassificationReport)

Classification Report is :
              precision    recall  f1-score   support

      AD              0.84         0.86         0.85         896
      CN              0.98         0.98         0.98         647
      EMCI             0.82         0.86         0.84         960
      LMCI             0.81         0.74         0.77         896

 accuracy              0.86
 macro avg              0.86
 weighted avg           0.85
```

5.2 VGG16

The VGG-16 model in machine learning refers to convolutional neural network (CNN) architecture that was proposed by the Visual Geometry Group (VGG) at the University of Oxford. It was introduced in the paper titled "Very Deep Convolutional Networks for Large-Scale Image Recognition" by Karen Simonyan and Andrew Zisserman in 2014.

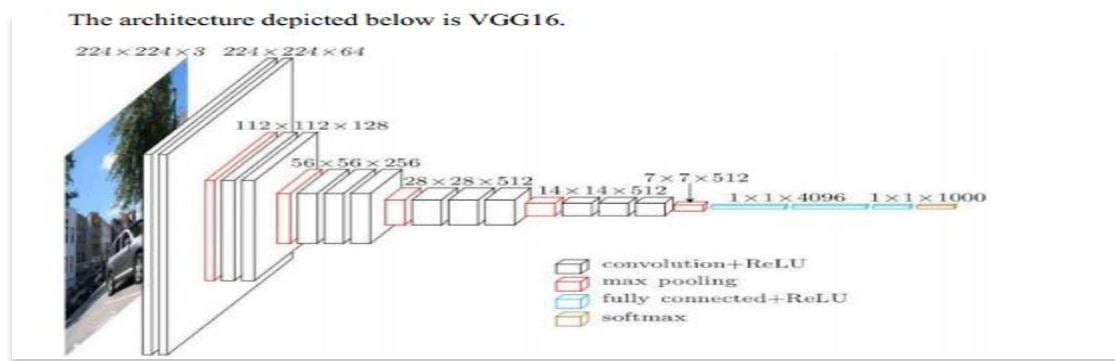


Figure (23) VGG16 Model

5.2.1 Model Architecture

```
In [8]: model = keras.models.Sequential()
model.add(tf.keras.applications.VGG16(input_shape=(224,224,3),include_top=False, pooling
='avg', weights='imagenet'))
model.add(keras.layers.Flatten())
model.add(keras.layers.BatchNormalization())
model.add(keras.layers.Dense(2048, activation='relu'))
model.add(keras.layers.BatchNormalization())
model.add(keras.layers.Dense(1024, activation='relu'))
model.add(keras.layers.BatchNormalization())
model.add(keras.layers.Dense(4, activation='softmax'))
model.layers[0].trainable = False
model.summary()
```

5.2.2 Model Scratch

Model: "sequential"

Layer (type)	Output Shape	Param #
=====		
vgg16 (Functional)	(None, 512)	14714688
flatten (Flatten)	(None, 512)	0
batch_normalization (Batch Normalization)	(None, 512)	2048
dense (Dense)	(None, 2048)	1050624
batch_normalization_1 (Batch Normalization)	(None, 2048)	8192
dense_1 (Dense)	(None, 1024)	2098176
batch_normalization_2 (Batch Normalization)	(None, 1024)	4096
dense_2 (Dense)	(None, 4)	4100
=====		
Total params: 17881924 (68.21 MB)		
Trainable params: 3160068 (12.05 MB)		
Non-trainable params: 14721856 (56.16 MB)		
=====		

Out[9]:

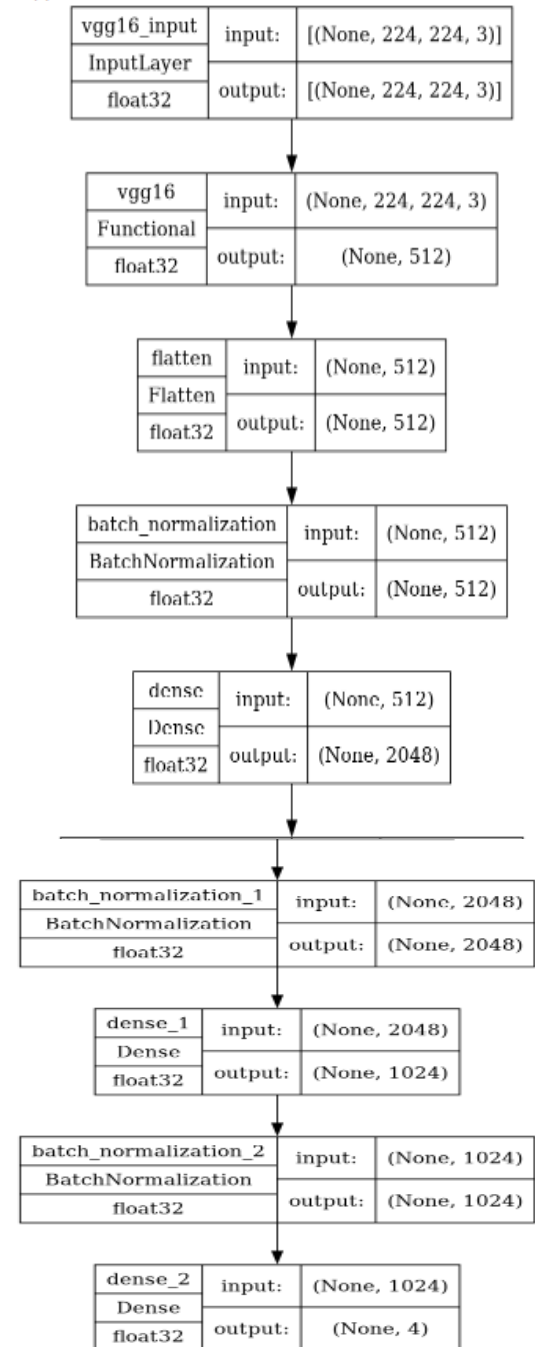


Figure (24) VGG16 Model Scratch

5.2.3 Model Performance

```

858/858 [-----] - 238s 254ms/step - loss: 0.7884 - accuracy: 0.7373 - val_loss: 0.3747 - val_accu
racy: 0.8473
Epoch 2/50
858/858 [-----] - 138s 153ms/step - loss: 0.3778 - accuracy: 0.8516 - val_loss: 0.2816 - val_accu
racy: 0.8932
Epoch 3/50
858/858 [-----] - 138s 153ms/step - loss: 0.2863 - accuracy: 0.8884 - val_loss: 0.2465 - val_accu
racy: 0.9152
Epoch 4/50
858/858 [-----] - 138s 153ms/step - loss: 0.2247 - accuracy: 0.9131 - val_loss: 0.1745 - val_accu
racy: 0.9335
Epoch 5/50
858/858 [-----] - 138s 152ms/step - loss: 0.1877 - accuracy: 0.9282 - val_loss: 0.1714 - val_accu
racy: 0.9358
Epoch 6/50
858/858 [-----] - 138s 153ms/step - loss: 0.1616 - accuracy: 0.9398 - val_loss: 0.1442 - val_accu
racy: 0.9497
Epoch 7/50
858/858 [-----] - 138s 153ms/step - loss: 0.1462 - accuracy: 0.9454 - val_loss: 0.1436 - val_accu
racy: 0.9517
Epoch 8/50
858/858 [-----] - 138s 153ms/step - loss: 0.1278 - accuracy: 0.9545 - val_loss: 0.1318 - val_accu
racy: 0.9564
Epoch 9/50
858/858 [-----] - 138s 152ms/step - loss: 0.1096 - accuracy: 0.9599 - val_loss: 0.1075 - val_accu
racy: 0.9628
Epoch 10/50
858/858 [-----] - 138s 153ms/step - loss: 0.1083 - accuracy: 0.9685 - val_loss: 0.1405 - val_accu
racy: 0.9482
Epoch 11/50
858/858 [-----] - 138s 152ms/step - loss: 0.0979 - accuracy: 0.9658 - val_loss: 0.1142 - val_accu
racy: 0.9626
Epoch 12/50
858/858 [-----] - 138s 153ms/step - loss: 0.0866 - accuracy: 0.9688 - val_loss: 0.1428 - val_accu
racy: 0.9597
Epoch 13/50
858/858 [-----] - 138s 152ms/step - loss: 0.0757 - accuracy: 0.9722 - val_loss: 0.0899 - val_accu
racy: 0.9706
Epoch 14/50
858/858 [-----] - 138s 152ms/step - loss: 0.0763 - accuracy: 0.9731 - val_loss: 0.1122 - val_accu
racy: 0.9708
Epoch 15/50
858/858 [-----] - 138s 152ms/step - loss: 0.0776 - accuracy: 0.9776 - val_loss: 0.1122 - val_accu
racy: 0.9776
Epoch 16/50
858/858 [-----] - 138s 152ms/step - loss: 0.0332 - accuracy: 0.9884 - val_loss: 0.0812 - val_accu
racy: 0.9791
Epoch 17/50
858/858 [-----] - 138s 152ms/step - loss: 0.0316 - accuracy: 0.9896 - val_loss: 0.0747 - val_accu
racy: 0.9758
Epoch 18/50
858/858 [-----] - 138s 153ms/step - loss: 0.0324 - accuracy: 0.9892 - val_loss: 0.0663 - val_accu
racy: 0.9828
Epoch 19/50
858/858 [-----] - 129s 152ms/step - loss: 0.0357 - accuracy: 0.9875 - val_loss: 0.0841 - val_accu
racy: 0.9753
Epoch 20/50
858/858 [-----] - 138s 153ms/step - loss: 0.0382 - accuracy: 0.9898 - val_loss: 0.0842 - val_accu
racy: 0.9803
Epoch 21/50
858/858 [-----] - 129s 152ms/step - loss: 0.0317 - accuracy: 0.9898 - val_loss: 0.0857 - val_accu
racy: 0.9808
Epoch 22/50
858/858 [-----] - 138s 153ms/step - loss: 0.0291 - accuracy: 0.9899 - val_loss: 0.0896 - val_accu
racy: 0.9778
Epoch 23/50
858/858 [-----] - 138s 153ms/step - loss: 0.0313 - accuracy: 0.9894 - val_loss: 0.0895 - val_accu
racy: 0.9791
Epoch 24/50
858/858 [-----] - 138s 152ms/step - loss: 0.0292 - accuracy: 0.9898 - val_loss: 0.0837 - val_accu
racy: 0.9818
Epoch 25/50
858/858 [-----] - 138s 153ms/step - loss: 0.0318 - accuracy: 0.9894 - val_loss: 0.1037 - val_accu
racy: 0.9788
Epoch 26/50
858/858 [-----] - 138s 153ms/step - loss: 0.0249 - accuracy: 0.9918 - val_loss: 0.0925 - val_accu
racy: 0.9794
Epoch 27/50
858/858 [-----] - 129s 152ms/step - loss: 0.0241 - accuracy: 0.9922 - val_loss: 0.0878 - val_accu
racy: 0.9778
Epoch 28/50
858/858 [-----] - 138s 153ms/step - loss: 0.0244 - accuracy: 0.9918 - val_loss: 0.0768 - val_accu
racy: 0.9829

```

5.2.4 Evaluate the Model

	loss	accuracy	val_loss	val_accuracy
0	0.700399	0.737301	0.374563	0.847263
1	0.377806	0.851583	0.281569	0.893172
2	0.286302	0.888439	0.246466	0.915244
3	0.224869	0.913120	0.174475	0.933490
4	0.167654	0.928201	0.171355	0.935845
5	0.161577	0.939751	0.144185	0.948676
6	0.146167	0.945376	0.143591	0.951736
7	0.127783	0.954464	0.131771	0.956445
8	0.109632	0.959907	0.107532	0.962036
9	0.106262	0.960496	0.140545	0.948205
10	0.097865	0.965020	0.114174	0.962625
11	0.086649	0.968809	0.142003	0.959652
12	0.075654	0.972229	0.089908	0.970571
13	0.076305	0.973149	0.112246	0.969952
14	0.070567	0.975356	0.096150	0.972042
15	0.068139	0.975356	0.113815	0.967039
16	0.061190	0.979108	0.091450	0.969666
17	0.056974	0.980910	0.102189	0.970571
18	0.056869	0.980873	0.082467	0.974955
19	0.056197	0.979512	0.116774	0.970865
20	0.052802	0.981903	0.092420	0.972925
21	0.046672	0.984625	0.092007	0.970865
22	0.042050	0.985802	0.094290	0.976751
23	0.045404	0.984551	0.084220	0.975868
24	0.050570	0.982823	0.076973	0.979105
25	0.039517	0.986317	0.070495	0.978222
26	0.036713	0.987751	0.071278	0.979105
27	0.036511	0.987347	0.083378	0.976457
28	0.042438	0.985545	0.086250	0.977634
29	0.033236	0.988414	0.081169	0.979105
30	0.031573	0.989554	0.074747	0.974955
31	0.032444	0.989223	0.066305	0.982048
32	0.035683	0.987457	0.084150	0.975280
33	0.030223	0.989811	0.084179	0.980283
34	0.031663	0.989811	0.085743	0.979958
35	0.029111	0.989922	0.080586	0.977045
36	0.031333	0.989443	0.089526	0.979105
37	0.029209	0.989848	0.083676	0.981754
38	0.031012	0.989407	0.103732	0.978811
39	0.024932	0.991798	0.092475	0.979400
40	0.024108	0.992239	0.087834	0.977045
41	0.024437	0.991025	0.075954	0.982931

```
In [12]: plt.figure(figsize=(15,10))
plt.subplot(1,2,1)
plt.plot(hist_['loss'],label='Train_Loss')
plt.plot(hist_['val_loss'],label='Validation_Loss')
plt.title('Train_Loss & Validation_Loss',fontsize=20)
plt.legend()
plt.subplot(1,2,2)
plt.plot(hist_['accuracy'],label='Train_Accuracy')
plt.plot(hist_['val_accuracy'],label='Validation_Accuracy')
plt.title('Train_Accuracy & Validation_Accuracy',fontsize=20)
plt.legend()
plt.show()
```

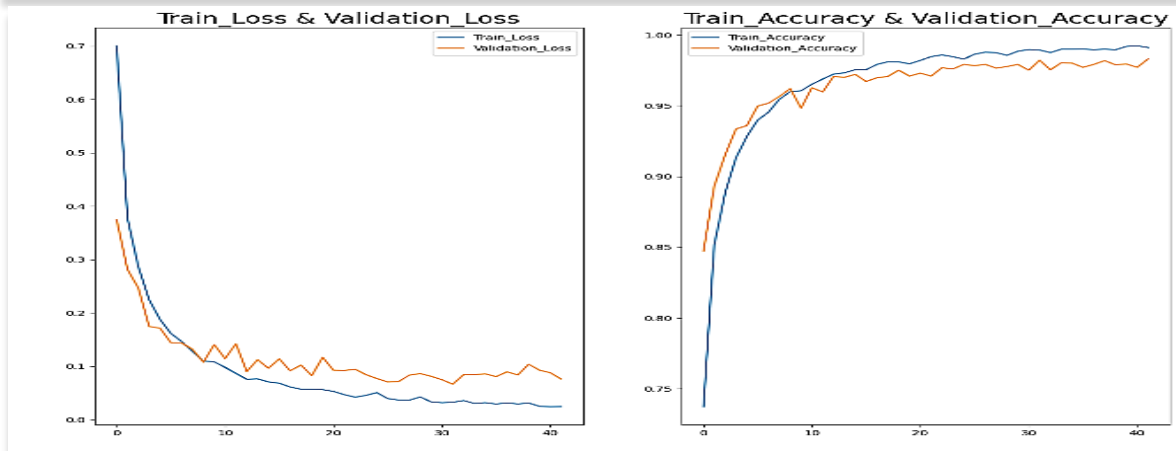


Figure (25) VGG16 Evaluation

5.2.5 Model Score

```
In [13]: score, acc= model.evaluate(test_generator)
print('Test Loss =', score)
print('Test Accuracy =', acc)

107/107 [=====] - 30s 276ms/step - loss: 0.0745 - accuracy: 0.9791
Test Loss = 0.07452037185430527
Test Accuracy = 0.9791114926338196
```

5.2.6 Model Evaluation

5.2.6.1 Confusion Matrix

```
In [16]: CM = confusion_matrix(y_test,y_pred)
CM_percent = CM.astype('float') / CM.sum(axis=1)[:, np.newaxis]
sns.heatmap(CM_percent,fmt='g',center = True,cbar=False,annot=True,cmap='Blues',xticklabels=class_num,
            yticklabels=class_num)
CM

Out[16]:
array([[889,  0,  4,  3],
       [ 0, 647,  0,  0],
       [ 6,  0, 936, 18],
       [ 8,  2, 23, 863]])
```

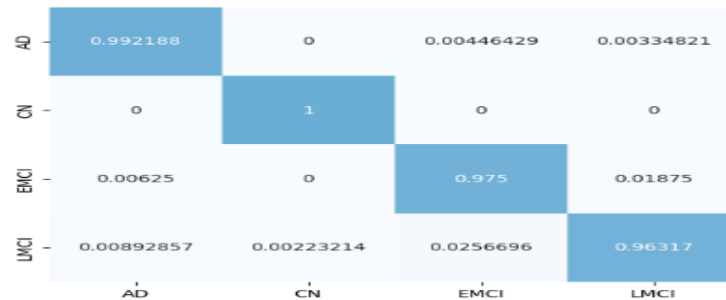


Figure (26) VGG16 Confusion Matrix

5.2.6.2 Classification Report

```
In [17]: ClassificationReport = classification_report(y_test,y_pred,target_names=class_num)
print('Classification Report is : ', ClassificationReport)
```

```
Classification Report is :
              precision    recall  f1-score   support

      AD          0.98      0.99      0.99         896
      CN          1.00      1.00      1.00         647
      EMCI         0.97      0.97      0.97         960
      LMCI         0.98      0.96      0.97         896

   accuracy          0.98
  macro avg          0.98
 weighted avg          0.98
```

5.3 DenseNet201

DenseNet-201 is a convolutional neural network that is 201 layers deep and is a flexible architecture applicable to a variety of computer vision applications including picture classification, object identification, and semantic segmentation. Among the most prevalent uses of DenseNet are NLP: Used in translation, sentiment analysis, and text generation.

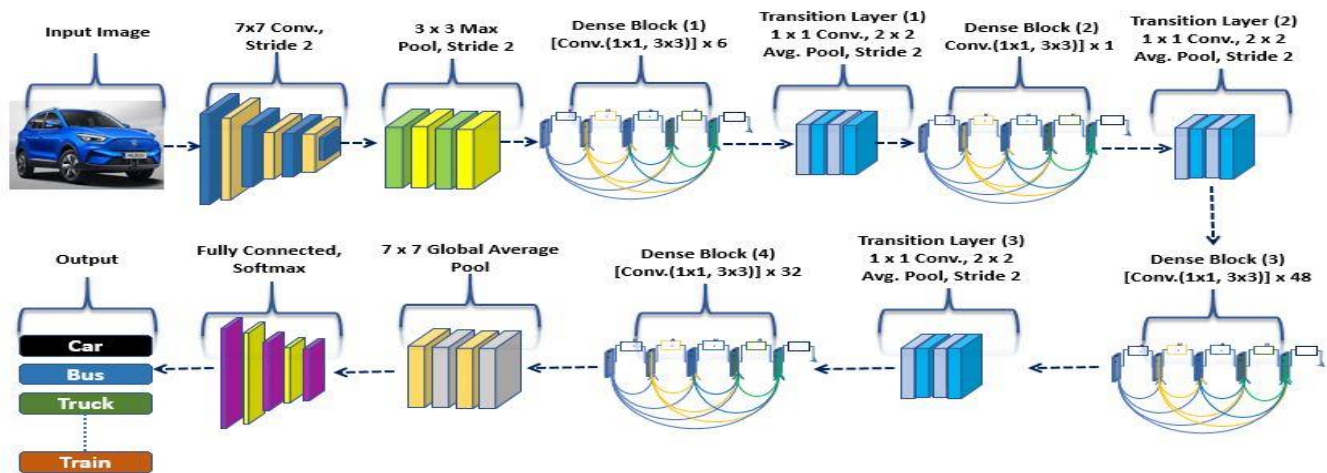


Figure (27) DenseNet201 Model

5.3.1 Model Architecture

```
In [8]: model = keras.models.Sequential()
model.add(tf.keras.applications.DenseNet201(input_shape=(224,224,3), include_top=False, pooling='avg', weights='imagenet'))
model.add(keras.layers.Flatten())
model.add(keras.layers.BatchNormalization())
model.add(keras.layers.Dense(2048, activation='relu'))
model.add(keras.layers.BatchNormalization())
model.add(keras.layers.Dense(1024, activation='relu'))
model.add(keras.layers.BatchNormalization())
model.add(keras.layers.Dense(4, activation='softmax'))
model.layers[0].trainable = False
model.summary()
```

5.3.2 Model Scratch

Model: "sequential"

Layer (type)	Output Shape	Param #
=====		
densenet201 (Functional)	(None, 1920)	18321984
flatten (Flatten)	(None, 1920)	0
batch_normalization (Batch Normalization)	(None, 1920)	7680
dense (Dense)	(None, 2048)	3934208
batch_normalization_1 (Batch Normalization)	(None, 2048)	8192
dense_1 (Dense)	(None, 1024)	2098176
batch_normalization_2 (Batch Normalization)	(None, 1024)	4096
dense_2 (Dense)	(None, 4)	4100
=====		

Total params: 24378436 (93.00 MB)

Trainable params: 6046468 (23.07 MB)

Non-trainable params: 18331968 (69.93 MB)

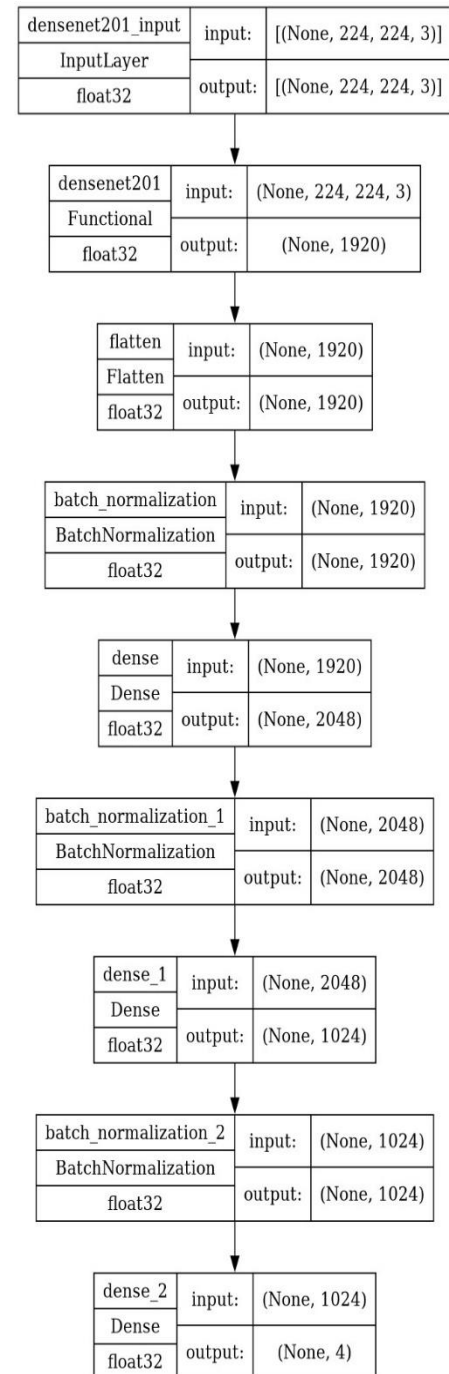


Figure (28) DenseNet201 Model Scratch

5.3.3 Model Performance

```

Epoch 1/50
850/850 [=====] - ETA: 0s - loss: 0.6654 - accuracy: 0.7512
850/850 [=====] - 229s 242ms/step - loss: 0.6654 - accuracy:
0.7512 - val_loss: 0.3861 - val_accuracy: 0.8473
Epoch 2/50
850/850 [=====] - 147s 172ms/step - loss: 0.3605 - accuracy:
0.8589 - val_loss: 0.2757 - val_accuracy: 0.8870
Epoch 3/50
850/850 [=====] - 145s 170ms/step - loss: 0.2589 - accuracy:
0.9023 - val_loss: 0.2830 - val_accuracy: 0.8999
Epoch 4/50
850/850 [=====] - 147s 172ms/step - loss: 0.2018 - accuracy:
0.9244 - val_loss: 0.1749 - val_accuracy: 0.9341
Epoch 5/50
850/850 [=====] - 146s 172ms/step - loss: 0.1734 - accuracy:
0.9354 - val_loss: 0.1710 - val_accuracy: 0.9403
Epoch 6/50
850/850 [=====] - 145s 170ms/step - loss: 0.1426 - accuracy:
0.9485 - val_loss: 0.1768 - val_accuracy: 0.9406
Epoch 7/50
850/850 [=====] - 147s 172ms/step - loss: 0.1218 - accuracy:
0.9554 - val_loss: 0.1261 - val_accuracy: 0.9538
Epoch 8/50
850/850 [=====] - 145s 170ms/step - loss: 0.1075 - accuracy:
0.9612 - val_loss: 0.1285 - val_accuracy: 0.9567
Epoch 9/50
850/850 [=====] - 145s 170ms/step - loss: 0.0934 - accuracy:
0.9655 - val_loss: 0.1439 - val_accuracy: 0.9550
Epoch 10/50
850/850 [=====] - 147s 173ms/step - loss: 0.0890 - accuracy:
0.9682 - val_loss: 0.0985 - val_accuracy: 0.9644
Epoch 11/50
850/850 [=====] - 145s 170ms/step - loss: 0.0782 - accuracy:
0.9725 - val_loss: 0.1579 - val_accuracy: 0.9544
Epoch 12/50
850/850 [=====] - 145s 170ms/step - loss: 0.0717 - accuracy:
0.9744 - val_loss: 0.1298 - val_accuracy: 0.9641
Epoch 13/50
850/850 [=====] - 145s 170ms/step - loss: 0.0653 - accuracy:
0.9777 - val_loss: 0.1408 - val_accuracy: 0.9614
Epoch 14/50
850/850 [=====] - 145s 170ms/step - loss: 0.0615 - accuracy:
0.9779 - val_loss: 0.1285 - val_accuracy: 0.9609
Epoch 15/50
850/850 [=====] - 147s 172ms/step - loss: 0.0586 - accuracy:
0.9796 - val_loss: 0.0966 - val_accuracy: 0.9712
Epoch 16/50
850/850 [=====] - 145s 171ms/step - loss: 0.0544 - accuracy:
0.9811 - val_loss: 0.1089 - val_accuracy: 0.9717
Epoch 17/50
850/850 [=====] - 145s 170ms/step - loss: 0.0437 - accuracy:
0.9848 - val_loss: 0.1350 - val_accuracy: 0.9641
Epoch 18/50
850/850 [=====] - 145s 171ms/step - loss: 0.0484 - accuracy:
0.9842 - val_loss: 0.1153 - val_accuracy: 0.9717
Epoch 19/50
850/850 [=====] - 145s 170ms/step - loss: 0.0483 - accuracy:
0.9839 - val_loss: 0.1191 - val_accuracy: 0.9685
Epoch 20/50
850/850 [=====] - 145s 171ms/step - loss: 0.0445 - accuracy:
0.9847 - val_loss: 0.1079 - val_accuracy: 0.9720
Epoch 21/50
850/850 [=====] - 146s 172ms/step - loss: 0.0443 - accuracy:
0.9848 - val_loss: 0.0814 - val_accuracy: 0.9732
Epoch 22/50
850/850 [=====] - 145s 171ms/step - loss: 0.0391 - accuracy:
0.9870 - val_loss: 0.1138 - val_accuracy: 0.9670
Epoch 23/50
850/850 [=====] - 145s 171ms/step - loss: 0.0400 - accuracy:
0.9867 - val_loss: 0.1114 - val_accuracy: 0.9729
Epoch 24/50
850/850 [=====] - 145s 170ms/step - loss: 0.0346 - accuracy:
0.9881 - val_loss: 0.1183 - val_accuracy: 0.9682
Epoch 25/50
850/850 [=====] - 145s 170ms/step - loss: 0.0355 - accuracy:
0.9889 - val_loss: 0.0866 - val_accuracy: 0.9715
Epoch 26/50
850/850 [=====] - 145s 171ms/step - loss: 0.0342 - accuracy:
0.9891 - val_loss: 0.0891 - val_accuracy: 0.9738
Epoch 27/50
850/850 [=====] - 147s 173ms/step - loss: 0.0316 - accuracy:
0.9891 - val_loss: 0.0660 - val_accuracy: 0.9797
Epoch 28/50
850/850 [=====] - 145s 170ms/step - loss: 0.0319 - accuracy:
0.9886 - val_loss: 0.1114 - val_accuracy: 0.9688
Epoch 29/50
850/850 [=====] - 145s 170ms/step - loss: 0.0306 - accuracy:
0.9897 - val_loss: 0.0860 - val_accuracy: 0.9744
Epoch 30/50
850/850 [=====] - 145s 170ms/step - loss: 0.0304 - accuracy:
0.9900 - val_loss: 0.0816 - val_accuracy: 0.9794
Epoch 31/50
850/850 [=====] - 145s 170ms/step - loss: 0.0319 - accuracy:
0.9889 - val_loss: 0.0879 - val_accuracy: 0.9756
Epoch 32/50
850/850 [=====] - 145s 170ms/step - loss: 0.0262 - accuracy:
0.9912 - val_loss: 0.0961 - val_accuracy: 0.9768
Epoch 33/50
850/850 [=====] - 145s 170ms/step - loss: 0.0286 - accuracy:
0.9901 - val_loss: 0.0725 - val_accuracy: 0.9803
Epoch 34/50
850/850 [=====] - 145s 170ms/step - loss: 0.0237 - accuracy:
0.9916 - val_loss: 0.1418 - val_accuracy: 0.9691
Epoch 35/50
850/850 [=====] - 145s 170ms/step - loss: 0.0256 - accuracy:
0.9912 - val_loss: 0.1107 - val_accuracy: 0.9744
Epoch 36/50
850/850 [=====] - 145s 170ms/step - loss: 0.0295 - accuracy:
0.9904 - val_loss: 0.0949 - val_accuracy: 0.9756
Epoch 37/50
850/850 [=====] - 145s 171ms/step - loss: 0.0232 - accuracy:
0.9925 - val_loss: 0.0740 - val_accuracy: 0.9803

```

5.3.4 Evaluate the Model

	loss	accuracy	val_loss	val_accuracy
0	0.665427	0.751205	0.386106	0.847263
1	0.360529	0.858903	0.275746	0.886992
2	0.258857	0.902306	0.282974	0.899941
3	0.201800	0.924376	0.174850	0.934079
4	0.173359	0.935374	0.171018	0.940259
5	0.142625	0.948542	0.176811	0.940553
6	0.121810	0.955383	0.126143	0.953796
7	0.107524	0.961158	0.128504	0.956739
8	0.093430	0.965535	0.143940	0.954974
9	0.088980	0.968183	0.098547	0.964391
10	0.078237	0.972524	0.157921	0.954385
11	0.071673	0.974436	0.129756	0.964097
12	0.065287	0.977673	0.140764	0.961448
13	0.061454	0.977931	0.128473	0.960859
14	0.058630	0.979623	0.096632	0.971160
15	0.054387	0.981131	0.108872	0.971748
16	0.043681	0.984809	0.134965	0.964097
17	0.048449	0.984220	0.115340	0.971748
18	0.048347	0.983926	0.119119	0.968511
19	0.044450	0.984699	0.107915	0.972042
20	0.044299	0.984846	0.081372	0.973220
21	0.039103	0.987016	0.113774	0.967039
22	0.040017	0.986722	0.111442	0.972925
23	0.034622	0.988119	0.118251	0.968217
24	0.035525	0.988892	0.086557	0.971454
25	0.034155	0.989076	0.089102	0.973808
26	0.031600	0.989112	0.066022	0.979694
27	0.031872	0.988634	0.111413	0.968805
28	0.030649	0.989738	0.085973	0.974397
29	0.030410	0.989958	0.081561	0.979400
30	0.031926	0.988892	0.087941	0.975574
31	0.026192	0.991209	0.096095	0.976751
32	0.028599	0.990106	0.072498	0.980283
33	0.023676	0.991577	0.141832	0.969099
34	0.025612	0.991209	0.110727	0.974397
35	0.029514	0.990363	0.094890	0.975574
36	0.023183	0.992533	0.074017	0.980283

In [12]:

```
plt.figure(figsize=(15,10))
plt.subplot(1,2,1)
plt.plot(hist_['loss'],label='Train_Loss')
plt.plot(hist_['val_loss'],label='Validation_Loss')
plt.title('Train_Loss & Validation_Loss',fontsize=20)
plt.legend()
plt.subplot(1,2,2)
plt.plot(hist_['accuracy'],label='Train_Accuracy')
plt.plot(hist_['val_accuracy'],label='Validation_Accuracy')
plt.title('Train_Accuracy & Validation_Accuracy',fontsize=20)
plt.legend()
plt.show()
```

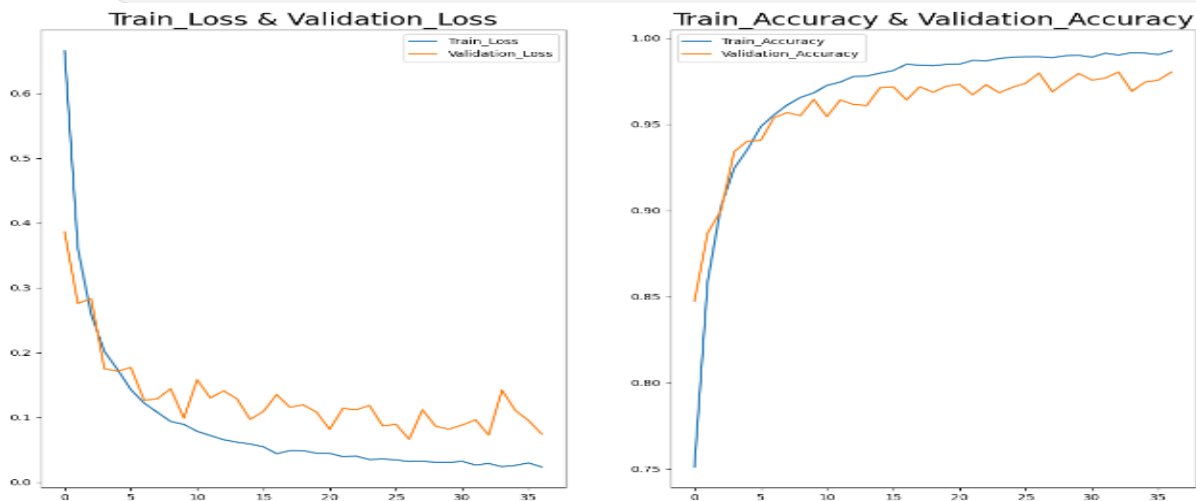


Figure (29) DenseNet201 Evaluation

5.3.5 Model Score

```
In [13]: score, acc= model.evaluate(test_generator)
print('Test Loss =', score)
print('Test Accuracy =', acc)

107/107 [=====] - 22s 208ms/step - loss: 0.07
68 - accuracy: 0.9785
Test Loss = 0.07676248997449875
Test Accuracy = 0.9785230755805969
```

5.3.6 Model Evaluation

5.3.6.1 Confusion Matrix

```
[16]: 1 CM = confusion_matrix(y_test,y_pred)
2 CM_percent = CM.astype('float') / CM.sum(axis=1)[:, np.newaxis]
3 sns.heatmap(CM_percent,fmt='g',center = True,cbar=False,annot=True,cmap='Blues',xticklabels=class_
4 CM

t[16]: array([[889,  0,  2,  5],
 [ 0, 646,  0,  1],
 [ 9,  1, 931, 19],
 [13,  0, 18, 865]])
```

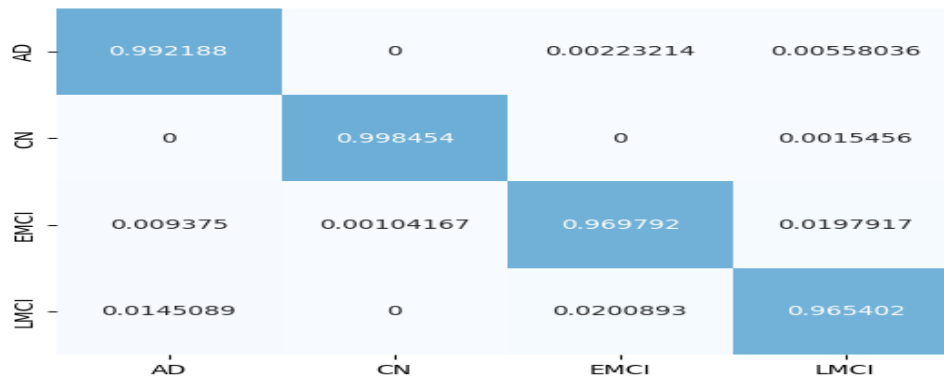


Figure (30) DenseNet201 Confusion Matrix

5.3.6.2 Classification Report

```
|: 1 ClassificationReport = classification_report(y_test,y_pred,target_names=class_num)
2 print('Classification Report is : ', ClassificationReport)
```

Classification Report is :

		precision	recall	f1-score	support
AD	0.98	0.99	0.98	0.98	896
CN	1.00	1.00	1.00	1.00	647
EMCI	0.98	0.97	0.97	0.97	960
LMCI	0.97	0.97	0.97	0.97	896
accuracy			0.98	0.98	3399
macro avg	0.98	0.98	0.98	0.98	3399
weighted avg	0.98	0.98	0.98	0.98	3399

5.4 ResNet152V2

ResNet-v2 is a convolutional neural architecture that builds on the Inception family of architectures but incorporates residual connections (replacing the filter concatenation stage of the Inception architecture). Source: Inception-v4, Inception-ResNet and the Impact of Residual Connections on Learning.

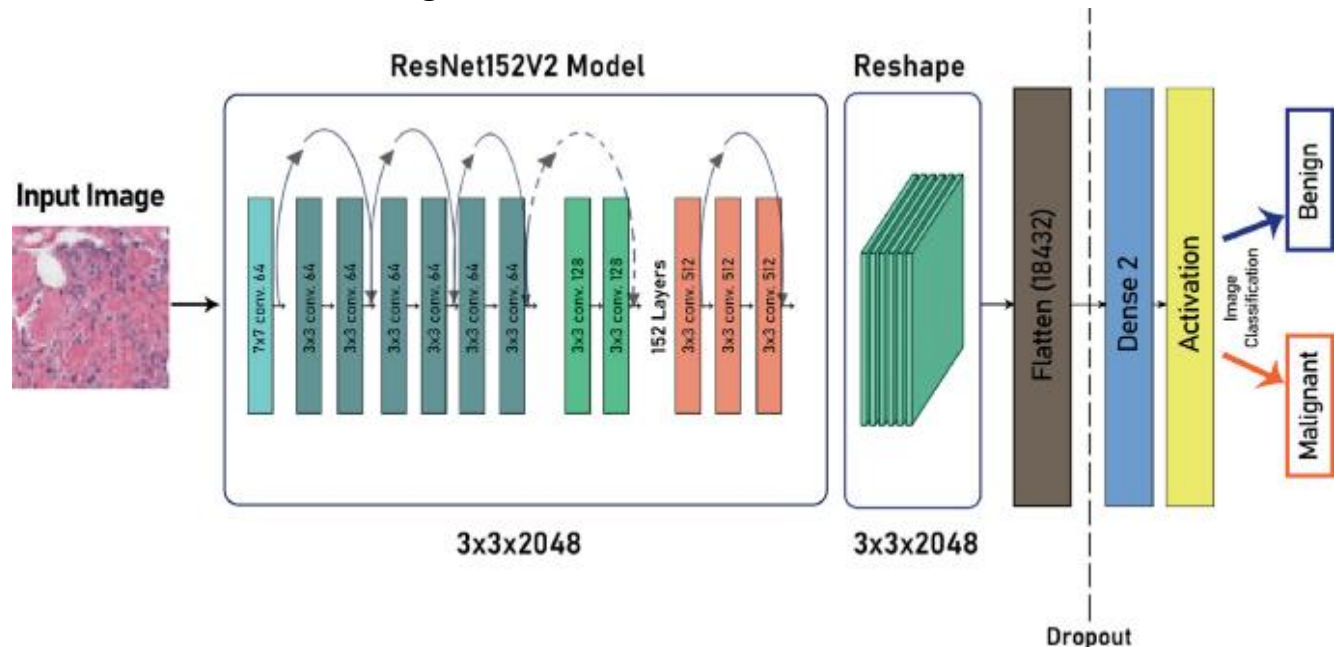


Figure (31) ResNet152V2 Model

5.4.1 Model Architecture

```
In [8]:
model = keras.models.Sequential()
model.add(tf.keras.applications.ResNet152V2(input_shape=(224,224,3),include_top=False, pooling='avg', weights='imagenet'))
model.add(keras.layers.Flatten())
model.add(keras.layers.BatchNormalization())
model.add(keras.layers.Dense(2048, activation='relu'))
model.add(keras.layers.BatchNormalization())
model.add(keras.layers.Dense(1024, activation='relu'))
model.add(keras.layers.BatchNormalization())
model.add(keras.layers.Dense(4, activation='softmax'))
model.layers[0].trainable = False
model.summary()
```

5.4.2 Model Scratch

Model: "sequential"

Layer (type)	Output Shape	Param #
=====		
resnet152v2 (Functional)	(None, 2048)	58331648
flatten (Flatten)	(None, 2048)	0
batch_normalization (Batch Normalization)	(None, 2048)	8192
dense (Dense)	(None, 2048)	4196352
batch_normalization_1 (Batch Normalization)	(None, 2048)	8192
dense_1 (Dense)	(None, 1024)	2098176
batch_normalization_2 (Batch Normalization)	(None, 1024)	4096
dense_2 (Dense)	(None, 4)	4100
=====		

Total params: 64650756 (246.62 MB)

Trainable params: 6308868 (24.07 MB)

Non-trainable params: 58341888 (222.56 MB)

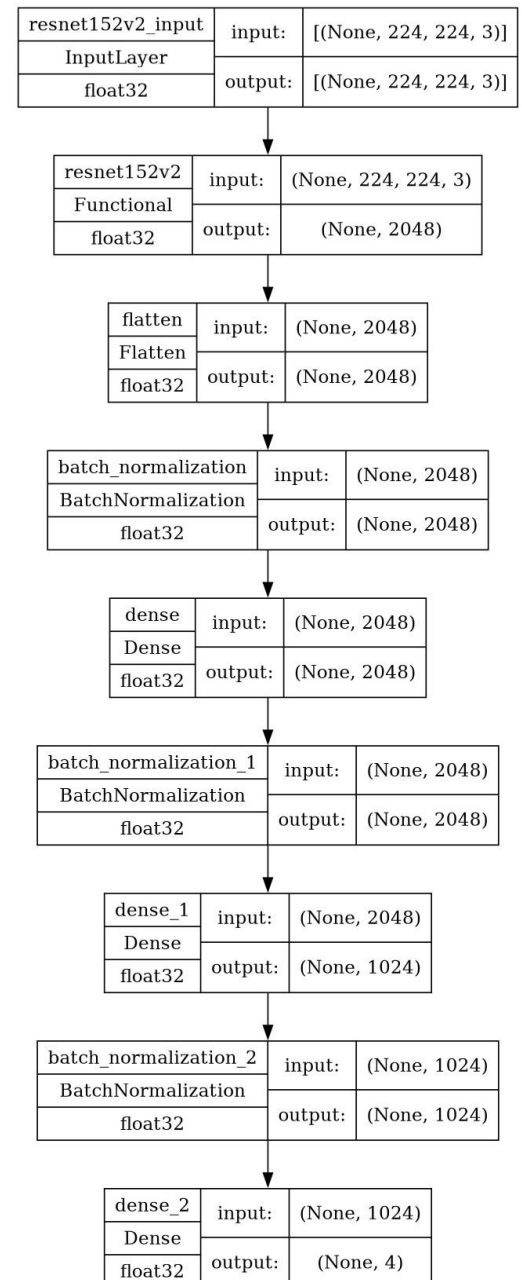


Figure (32) ResNet152V2 Model Scratch

5.4.3 Model Performance

```
Epoch 1/50
850/850 [=====] - ETA: 0s - loss: 0.8286 - accuracy: 0.6719
850/850 [=====] - 243s 266ms/step - loss: 0.8286 - accuracy:
0.6719 - val_loss: 0.5417 - val_accuracy: 0.7640
Epoch 2/50
850/850 [=====] - 228s 268ms/step - loss: 0.5395 - accuracy:
0.7789 - val_loss: 0.5368 - val_accuracy: 0.7787
Epoch 3/50
850/850 [=====] - 229s 269ms/step - loss: 0.4374 - accuracy:
0.8250 - val_loss: 0.3801 - val_accuracy: 0.8402
Epoch 4/50
850/850 [=====] - 228s 269ms/step - loss: 0.3592 - accuracy:
0.8574 - val_loss: 0.3693 - val_accuracy: 0.8523
Epoch 5/50
850/850 [=====] - 228s 269ms/step - loss: 0.3118 - accuracy:
0.8758 - val_loss: 0.3410 - val_accuracy: 0.8634
Epoch 6/50
850/850 [=====] - 228s 269ms/step - loss: 0.2793 - accuracy:
0.8918 - val_loss: 0.3264 - val_accuracy: 0.8811
Epoch 7/50
850/850 [=====] - 228s 269ms/step - loss: 0.2325 - accuracy:
0.9107 - val_loss: 0.2901 - val_accuracy: 0.8902
Epoch 8/50
850/850 [=====] - 228s 268ms/step - loss: 0.2072 - accuracy:
0.9232 - val_loss: 0.2632 - val_accuracy: 0.9035
Epoch 9/50
850/850 [=====] - 226s 266ms/step - loss: 0.1880 - accuracy:
0.9298 - val_loss: 0.2954 - val_accuracy: 0.8917
Epoch 10/50
850/850 [=====] - 227s 266ms/step - loss: 0.1643 - accuracy:
0.9384 - val_loss: 0.2886 - val_accuracy: 0.9079
Epoch 11/50
850/850 [=====] - 227s 267ms/step - loss: 0.1467 - accuracy:
0.9461 - val_loss: 0.2702 - val_accuracy: 0.9105
Epoch 12/50
850/850 [=====] - 226s 266ms/step - loss: 0.1320 - accuracy:
0.9522 - val_loss: 0.2696 - val_accuracy: 0.9167
Epoch 13/50
850/850 [=====] - 228s 268ms/step - loss: 0.1247 - accuracy:
0.9550 - val_loss: 0.2439 - val_accuracy: 0.9197
Epoch 14/50
850/850 [=====] - 227s 267ms/step - loss: 0.1174 - accuracy:
0.9560 - val_loss: 0.2531 - val_accuracy: 0.9208
Epoch 15/50
850/850 [=====] - 228s 268ms/step - loss: 0.1026 - accuracy:
0.9628 - val_loss: 0.2395 - val_accuracy: 0.9205
Epoch 16/50
850/850 [=====] - 227s 267ms/step - loss: 0.0991 - accuracy:
0.9640 - val_loss: 0.2847 - val_accuracy: 0.9164
Epoch 17/50
850/850 [=====] - 226s 266ms/step - loss: 0.0936 - accuracy:
0.9651 - val_loss: 0.3143 - val_accuracy: 0.9079
Epoch 18/50
850/850 [=====] - 227s 267ms/step - loss: 0.0835 - accuracy:
0.9711 - val_loss: 0.2753 - val_accuracy: 0.9232
Epoch 19/50
850/850 [=====] - 227s 266ms/step - loss: 0.0744 - accuracy:
0.9731 - val_loss: 0.2457 - val_accuracy: 0.9285
Epoch 20/50
850/850 [=====] - 226s 266ms/step - loss: 0.0768 - accuracy:
0.9719 - val_loss: 0.2483 - val_accuracy: 0.9308
Epoch 21/50
850/850 [=====] - 227s 267ms/step - loss: 0.0718 - accuracy:
0.9743 - val_loss: 0.2447 - val_accuracy: 0.9288
Epoch 22/50
850/850 [=====] - 226s 266ms/step - loss: 0.0658 - accuracy:
0.9764 - val_loss: 0.2687 - val_accuracy: 0.9223
Epoch 23/50
850/850 [=====] - 227s 267ms/step - loss: 0.0672 - accuracy:
0.9757 - val_loss: 0.2780 - val_accuracy: 0.9238
Epoch 24/50
850/850 [=====] - 227s 267ms/step - loss: 0.0552 - accuracy:
0.9810 - val_loss: 0.2488 - val_accuracy: 0.9258
Epoch 25/50
850/850 [=====] - 227s 267ms/step - loss: 0.0569 - accuracy:
0.9811 - val_loss: 0.2986 - val_accuracy: 0.9244
```

5.4.4 Evaluate the Model

	loss	accuracy	val_loss	val_accuracy
0	0.828621	0.671902	0.541669	0.763979
1	0.539499	0.778938	0.536810	0.778693
2	0.437374	0.825027	0.380056	0.840200
3	0.359183	0.857432	0.369309	0.852266
4	0.311762	0.875823	0.340961	0.863449
5	0.279274	0.891823	0.326399	0.881107
6	0.232536	0.910729	0.290103	0.890230
7	0.207184	0.923199	0.263188	0.903473
8	0.187984	0.929819	0.295401	0.891701
9	0.164344	0.938353	0.288586	0.907887
10	0.146749	0.946077	0.270237	0.910536
11	0.131968	0.952220	0.269567	0.916716
12	0.124709	0.954978	0.243933	0.919659
13	0.117402	0.955972	0.253141	0.920836
14	0.102575	0.962813	0.239520	0.920541
15	0.099074	0.964027	0.284699	0.916421
16	0.093611	0.965130	0.314296	0.907887
17	0.083496	0.971089	0.275284	0.923190
18	0.074386	0.973149	0.245684	0.928487
19	0.076776	0.971935	0.248300	0.930842
20	0.071751	0.974252	0.244694	0.928782
21	0.065769	0.976386	0.268717	0.922307
22	0.067214	0.975687	0.277995	0.923779
23	0.055155	0.981020	0.248840	0.925839
24	0.056905	0.981131	0.298553	0.924367

```
In [12]: plt.figure(figsize=(15,10))
plt.subplot(1,2,1)
plt.plot(hist_['loss'],label='Train_Loss')
plt.plot(hist_['val_loss'],label='Validation_Loss')
plt.title('Train_Loss & Validation_Loss',fontsize=20)
plt.legend()
plt.subplot(1,2,2)
plt.plot(hist_['accuracy'],label='Train_Accuracy')
plt.plot(hist_['val_accuracy'],label='Validation_Accuracy')
plt.title('Train_Accuracy & Validation_Accuracy',fontsize=20)
plt.legend()
plt.show()
```

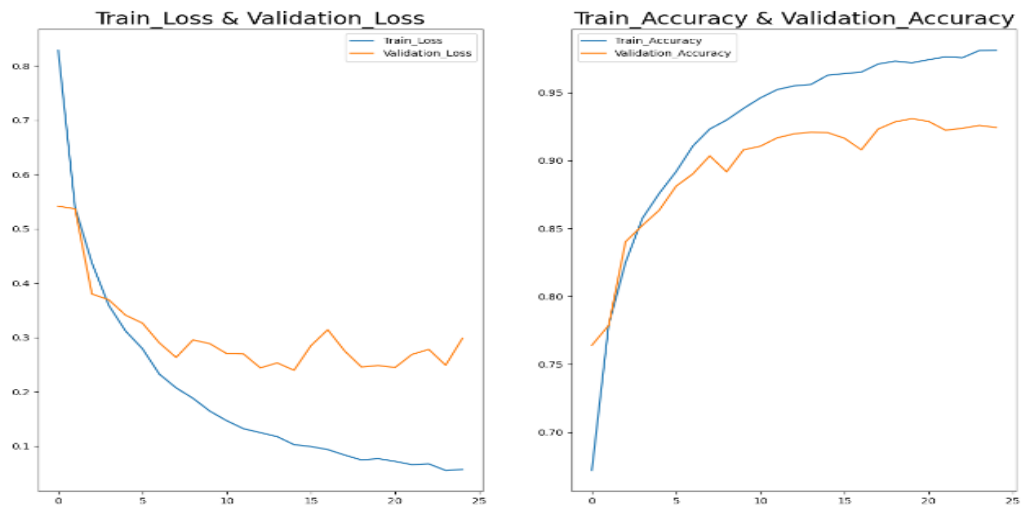


Figure (33) ResNet152V2 Evaluation

5.4.5 Model Score

```

:
score, acc= model.evaluate(test_generator)
print('Test Loss =', score)
print('Test Accuracy =', acc)

107/107 [=====] - 27s 247ms/step - loss: 0.26
27 - accuracy: 0.9167
Test Loss = 0.2627180218696594
Test Accuracy = 0.9167402386665344

```

5.4.6 Model Evaluation

5.4.6.1 Confusion Matrix

```

[16]: 1 CM = confusion_matrix(y_test,y_pred)
      2 CM_percent = CM.astype('float') / CM.sum(axis=1)[:, np.newaxis]
      3 sns.heatmap(CM_percent,fmt='g',center = True,cbar=False,annot=True,cmap='Blues',xticklabels=class_
      4 CM

[16]: array([[835,  2, 28, 31],
            [ 0, 646,  1,  0],
            [22,  0, 886, 52],
            [39,  0, 106, 751]])

```

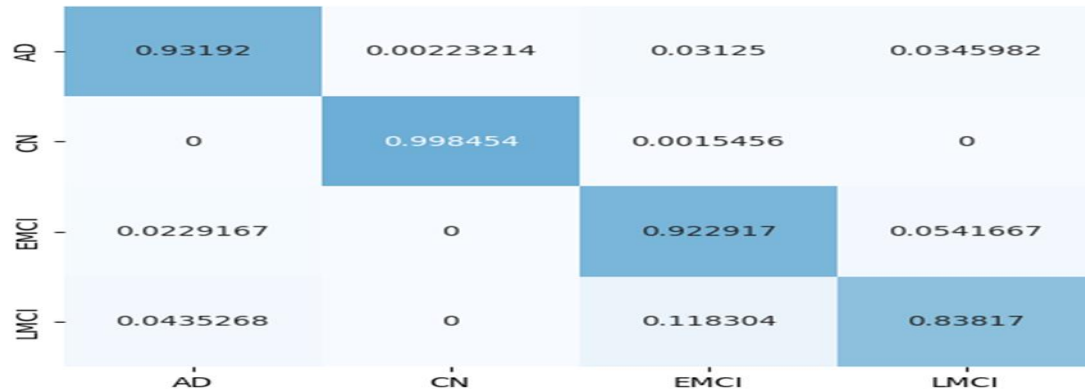


Figure (34) ResNet152V2 Confusion Matrix

5.4.6.2 Classification Report

```

[17]: 1 ClassificationReport = classification_report(y_test,y_pred,target_names=class_num)
      2 print('Classification Report is : ', ClassificationReport)

Classification Report is :

```

		precision	recall	f1-score	support
AD	0.93	0.93	0.93	0.93	896
CN	1.00	1.00	1.00	1.00	647
EMCI	0.87	0.92	0.89	0.89	960
LMCI	0.90	0.84	0.87	0.87	896
accuracy			0.92	0.92	3399
macro avg	0.92	0.92	0.92	0.92	3399
weighted avg	0.92	0.92	0.92	0.92	3399

5.5 MobileNetV2

MobileNetV2 is a deep learning model designed for computer vision tasks on mobile devices such as smartphones and tablets. It is an improvement over the original MobileNet model and was developed by researchers at Google. MobileNetV2 aims to enhance model efficiency so that it can run efficiently on resource-constrained devices like mobile phones without sacrificing performance or accuracy. It relies on the idea of using repeated building blocks for construction, making it robust and suitable for a variety of computer vision applications on resource-limited devices.

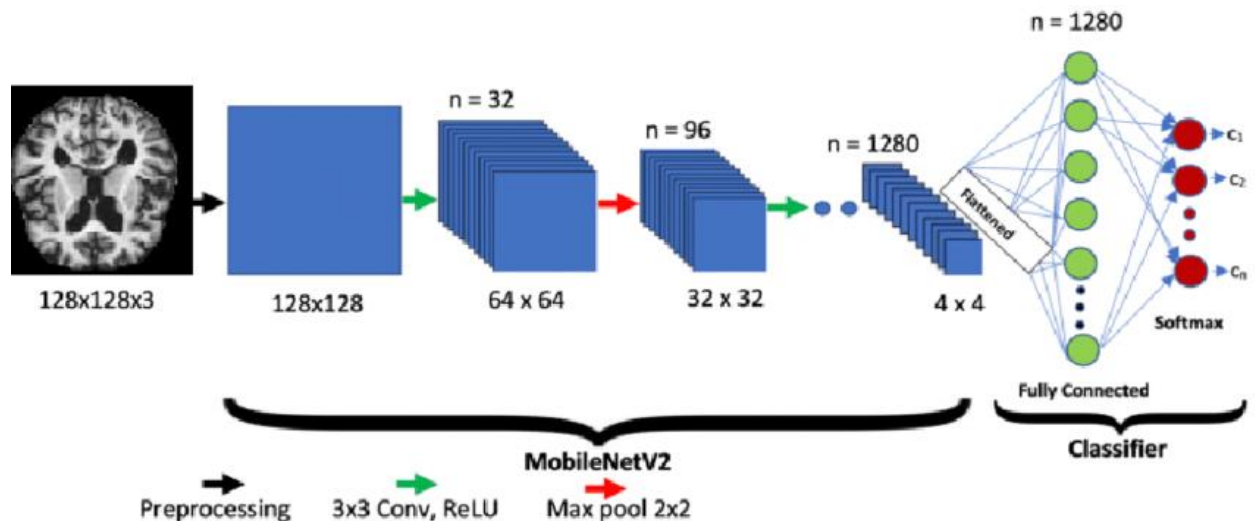


Figure (35) MobileNetV2 Model

5.5.1 Model Architecture

```
model = keras.models.Sequential()
model.add(tf.keras.applications.MobileNetV2(input_shape=(224,224,3),include_top=False, pooling='avg', weights='imagenet'))
model.add(keras.layers.Flatten())
model.add(keras.layers.BatchNormalization())
model.add(keras.layers.Dense(2048, activation='relu'))
model.add(keras.layers.BatchNormalization())
model.add(keras.layers.Dense(1024, activation='relu'))
model.add(keras.layers.BatchNormalization())
model.add(keras.layers.Dense(4, activation='softmax'))
model.layers[0].trainable = False
model.summary()
```

5.5.2 Model Scratch

Model: "sequential"

Layer (type)	Output Shape	Param #
=====		
mobilenetv2_1.00_224 (Functional)	(None, 1280)	2257984
flatten (Flatten)	(None, 1280)	0
batch_normalization (Batch Normalization)	(None, 1280)	5120
dense (Dense)	(None, 2048)	2623488
batch_normalization_1 (Batch Normalization)	(None, 2048)	8192
dense_1 (Dense)	(None, 1024)	2098176
batch_normalization_2 (Batch Normalization)	(None, 1024)	4096
dense_2 (Dense)	(None, 4)	4100
=====		
Total params: 7001156 (26.71 MB)		
Trainable params: 4734468 (18.06 MB)		
Non-trainable params: 3326688 (12.65 MB)		

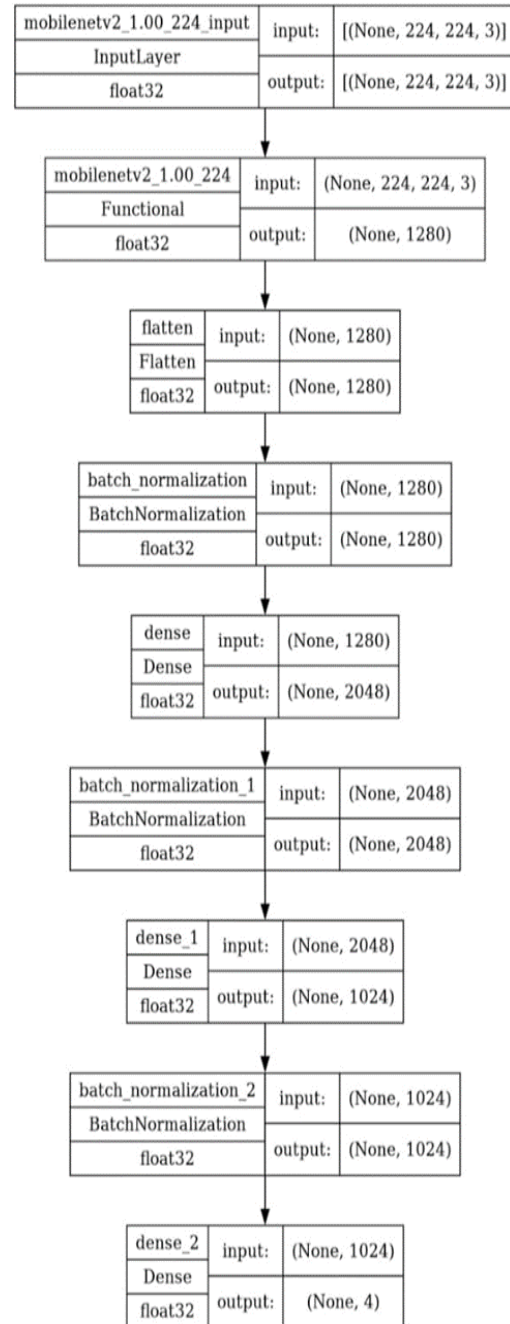


Figure (36) Model Scratch

5.5.3 Model Performance

```

Epoch 1/50
850/850 [=====] - ETA: 0s - loss: 0.7943 - accuracy: 0.6905
850/850 [=====] - 231s 261ms/step - loss: 0.7943 - accuracy:
0.6905 - val_loss: 0.5611 - val_accuracy: 0.7749
Epoch 2/50
850/850 [=====] - 93s 109ms/step - loss: 0.4840 - accuracy:
0.8049 - val_loss: 0.4064 - val_accuracy: 0.8361
Epoch 3/50
850/850 [=====] - 92s 108ms/step - loss: 0.3889 - accuracy:
0.8453 - val_loss: 0.3336 - val_accuracy: 0.8749
Epoch 4/50
850/850 [=====] - 92s 108ms/step - loss: 0.3047 - accuracy:
0.8812 - val_loss: 0.2857 - val_accuracy: 0.8908
Epoch 5/50
850/850 [=====] - 98s 116ms/step - loss: 0.2598 - accuracy:
0.8985 - val_loss: 0.2663 - val_accuracy: 0.9017
Epoch 6/50
850/850 [=====] - 97s 114ms/step - loss: 0.2276 - accuracy:
0.9149 - val_loss: 0.2472 - val_accuracy: 0.9035
Epoch 7/50
850/850 [=====] - 96s 113ms/step - loss: 0.1896 - accuracy:
0.9291 - val_loss: 0.2545 - val_accuracy: 0.9073
Epoch 8/50
850/850 [=====] - 95s 112ms/step - loss: 0.1666 - accuracy:
0.9379 - val_loss: 0.2525 - val_accuracy: 0.9114
Epoch 9/50
850/850 [=====] - 94s 111ms/step - loss: 0.1546 - accuracy:
0.9426 - val_loss: 0.2293 - val_accuracy: 0.9191
Epoch 10/50
850/850 [=====] - 107s 125ms/step - loss: 0.1383 - accuracy:
0.9482 - val_loss: 0.1977 - val_accuracy: 0.9338
Epoch 11/50
850/850 [=====] - 98s 115ms/step - loss: 0.1219 - accuracy:
0.9583 - val_loss: 0.2267 - val_accuracy: 0.9258
Epoch 12/50
850/850 [=====] - 100s 117ms/step - loss: 0.1145 - accuracy:
0.9586 - val_loss: 0.2099 - val_accuracy: 0.9335
Epoch 13/50
850/850 [=====] - 102s 120ms/step - loss: 0.1080 - accuracy:
0.9626 - val_loss: 0.2235 - val_accuracy: 0.9244
Epoch 14/50
850/850 [=====] - 99s 117ms/step - loss: 0.0996 - accuracy:
0.9642 - val_loss: 0.1921 - val_accuracy: 0.9350
Epoch 15/50
850/850 [=====] - 94s 110ms/step - loss: 0.0955 - accuracy:
0.9659 - val_loss: 0.1967 - val_accuracy: 0.9367
Epoch 16/50
850/850 [=====] - 95s 112ms/step - loss: 0.0845 - accuracy:
0.9697 - val_loss: 0.2215 - val_accuracy: 0.9367
Epoch 17/50
850/850 [=====] - 96s 113ms/step - loss: 0.0783 - accuracy:
0.9727 - val_loss: 0.1885 - val_accuracy: 0.9408
Epoch 18/50
850/850 [=====] - 94s 110ms/step - loss: 0.0681 - accuracy:
0.9753 - val_loss: 0.1996 - val_accuracy: 0.9429
Epoch 19/50
850/850 [=====] - 100s 118ms/step - loss: 0.0747 - accuracy:
0.9736 - val_loss: 0.1859 - val_accuracy: 0.9438
Epoch 20/50
850/850 [=====] - 93s 109ms/step - loss: 0.0677 - accuracy:
0.9758 - val_loss: 0.1918 - val_accuracy: 0.9420
Epoch 21/50
850/850 [=====] - 99s 117ms/step - loss: 0.0661 - accuracy:
0.9762 - val_loss: 0.1796 - val_accuracy: 0.9438
Epoch 22/50
850/850 [=====] - 119s 140ms/step - loss: 0.0675 - accuracy:
0.9766 - val_loss: 0.1689 - val_accuracy: 0.9506
Epoch 23/50
850/850 [=====] - 95s 112ms/step - loss: 0.0600 - accuracy:
0.9796 - val_loss: 0.1685 - val_accuracy: 0.9506
Epoch 24/50
850/850 [=====] - 91s 106ms/step - loss: 0.0566 - accuracy:
0.9811 - val_loss: 0.2039 - val_accuracy: 0.9438
Epoch 25/50
850/850 [=====] - 96s 112ms/step - loss: 0.0544 - accuracy:
0.9807 - val_loss: 0.2050 - val_accuracy: 0.9411
Epoch 26/50
850/850 [=====] - 93s 109ms/step - loss: 0.0546 - accuracy:
0.9814 - val_loss: 0.1952 - val_accuracy: 0.9532
Epoch 27/50
850/850 [=====] - 95s 111ms/step - loss: 0.0478 - accuracy:
0.9834 - val_loss: 0.2099 - val_accuracy: 0.9423
Epoch 28/50
850/850 [=====] - 99s 116ms/step - loss: 0.0519 - accuracy:
0.9816 - val_loss: 0.1575 - val_accuracy: 0.9509
Epoch 29/50
850/850 [=====] - 95s 111ms/step - loss: 0.0461 - accuracy:
0.9840 - val_loss: 0.1969 - val_accuracy: 0.9494
Epoch 30/50
850/850 [=====] - 92s 108ms/step - loss: 0.0468 - accuracy:
0.9828 - val_loss: 0.2117 - val_accuracy: 0.9461
Epoch 31/50
850/850 [=====] - 95s 112ms/step - loss: 0.0451 - accuracy:
0.9849 - val_loss: 0.1742 - val_accuracy: 0.9532
Epoch 32/50
850/850 [=====] - 92s 108ms/step - loss: 0.0458 - accuracy:
0.9843 - val_loss: 0.2028 - val_accuracy: 0.9473
Epoch 33/50
850/850 [=====] - 90s 106ms/step - loss: 0.0433 - accuracy:
0.9859 - val_loss: 0.1615 - val_accuracy: 0.9517
Epoch 34/50
850/850 [=====] - 94s 110ms/step - loss: 0.0439 - accuracy:
0.9858 - val_loss: 0.1732 - val_accuracy: 0.9567
Epoch 35/50
850/850 [=====] - 95s 111ms/step - loss: 0.0451 - accuracy:
0.9837 - val_loss: 0.1797 - val_accuracy: 0.9511
Epoch 36/50
850/850 [=====] - 93s 109ms/step - loss: 0.0356 - accuracy:
0.9886 - val_loss: 0.1848 - val_accuracy: 0.9494
Epoch 37/50
850/850 [=====] - 92s 108ms/step - loss: 0.0367 - accuracy:
0.9873 - val_loss: 0.2523 - val_accuracy: 0.9411

```

5.5.4 Evaluate the Model

	loss	accuracy	val_loss	val_accuracy
0	0.794263	0.690477	0.561089	0.774868
1	0.484034	0.804907	0.406399	0.836080
2	0.388852	0.845257	0.333598	0.874926
3	0.304652	0.881193	0.285740	0.890818
4	0.259773	0.898518	0.266288	0.901707
5	0.227642	0.914886	0.247242	0.903473
6	0.189606	0.929084	0.254533	0.907298
7	0.166628	0.937948	0.252487	0.911418
8	0.154564	0.942583	0.229285	0.919070
9	0.138296	0.948174	0.197686	0.933785
10	0.121883	0.955346	0.226654	0.925839
11	0.114534	0.958620	0.209883	0.933490
12	0.107991	0.962556	0.223492	0.924367
13	0.099640	0.964174	0.192056	0.934962
14	0.095511	0.965940	0.196684	0.936727
15	0.084515	0.969728	0.221460	0.936727
16	0.078295	0.972671	0.188527	0.940848
17	0.068078	0.975282	0.199569	0.942908
18	0.074690	0.973627	0.185887	0.943790
19	0.067662	0.975834	0.191842	0.942025
20	0.066126	0.976165	0.179637	0.943790
21	0.067497	0.976643	0.168861	0.950559
22	0.060031	0.979623	0.168476	0.950559
23	0.056608	0.981094	0.203856	0.943790
24	0.054372	0.980689	0.205036	0.941142
25	0.054591	0.981425	0.195181	0.953208
26	0.047770	0.983411	0.209859	0.942319
27	0.051903	0.981572	0.157466	0.950853
28	0.046117	0.984000	0.196940	0.949382
29	0.046802	0.982823	0.211651	0.946145
30	0.045084	0.984919	0.174235	0.953208
31	0.045826	0.984257	0.202751	0.947322

```
In [12]: plt.figure(figsize=(15,10))
plt.subplot(1,2,1)
plt.plot(hist_['loss'],label='Train_Loss')
plt.plot(hist_['val_loss'],label='Validation_Loss')
plt.title('Train_Loss & Validation_Loss',fontsize=20)
plt.legend()
plt.subplot(1,2,2)
plt.plot(hist_['accuracy'],label='Train_Accuracy')
plt.plot(hist_['val_accuracy'],label='Validation_Accuracy')
plt.title('Train_Accuracy & Validation_Accuracy',fontsize=20)
plt.legend()
plt.show()
```



Figure (37) MobileNetV2 Evaluation

5.5.5 Model Score

```

>]: 1 score, acc= model.evaluate(test_generator)
     2 print('Test Loss =', score)
     3 print('Test Accuracy =', acc)

107/107 [=====] - 28s 264ms/step - loss: 0.1583 - accuracy: 0.9482
Test Loss = 0.15833717584609985
Test Accuracy = 0.9482200741767883
    
```

5.5.6 Model Evaluation

5.5.6.1 Confusion Matrix

```

[16]: 1 CM = confusion_matrix(y_test,y_pred)
      2 CM_percent = CM.astype('float') / CM.sum(axis=1)[:, np.newaxis]
      3 sns.heatmap(CM_percent,fmt='g',center = True,cbar=False,annot=True,cmap='Blues',
      4             xticklabels=class_num, yticklabels=class_num)
      5 CM

t[16]: array([[862,  0, 12, 22],
              [ 1, 645,  0,  1],
              [12,  0, 904, 44],
              [25,  1, 54, 816]])
    
```

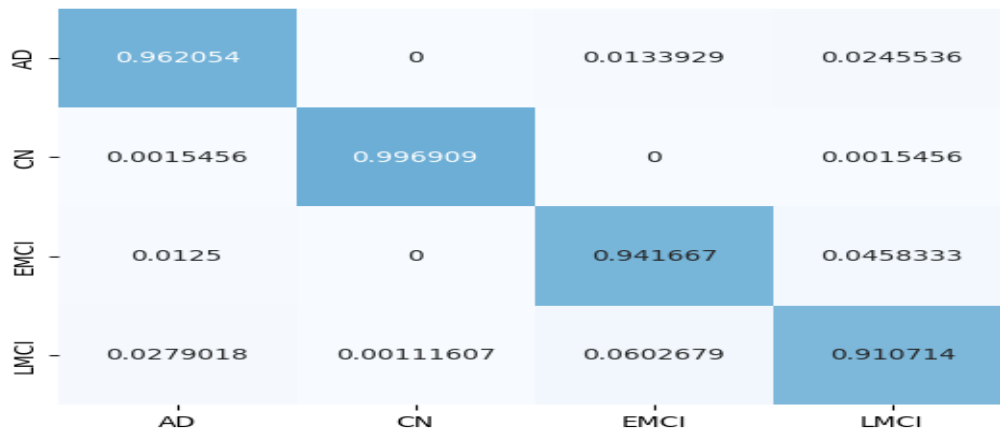


Figure (38) MobileNetV2 Confusion Matrix

5.5.6.2 Classification Report

```

[17]: 1 ClassificationReport = classification_report(y_test,y_pred,target_names=class_num)
      2 print('Classification Report is : ', ClassificationReport)
    
```

```

Classification Report is :              precision    recall  f1-score   support

      AD          0.96          0.96          0.96         896
      CN          1.00          1.00          1.00         647
      EMCI         0.93          0.94          0.94         960
      LMCI         0.92          0.91          0.92         896

 accuracy              0.95              3399
 macro avg           0.95           0.95           0.95              3399
 weighted avg        0.95           0.95           0.95              3399
    
```

5.6 Model predictions

```
y_test = test_generator.classes
predictions = model.predict(test_generator)
y_pred = np.argmax(predictions,axis=1)
y_test = np.ravel(y_test)
y_pred = np.ravel(y_pred)
df = pd.DataFrame({'Actual': y_test, 'Prediction': y_pred})
```

```
plt.figure(figsize=(30,70))
batch = next(test_generator)
images= batch[0]
for n in range(32):
    plt.subplot(8,4,n+1)
    plt.imshow(images[n])
    plt.axis('off')
    plt.title(f"Actual: {class_num[y_test[n]]}, \n Predicted: {class_num[y_pred[n]]}.\n Confidence: {round(predictions[n][np.argmax(predictions[n]),0]}%", fontsize=20)
```

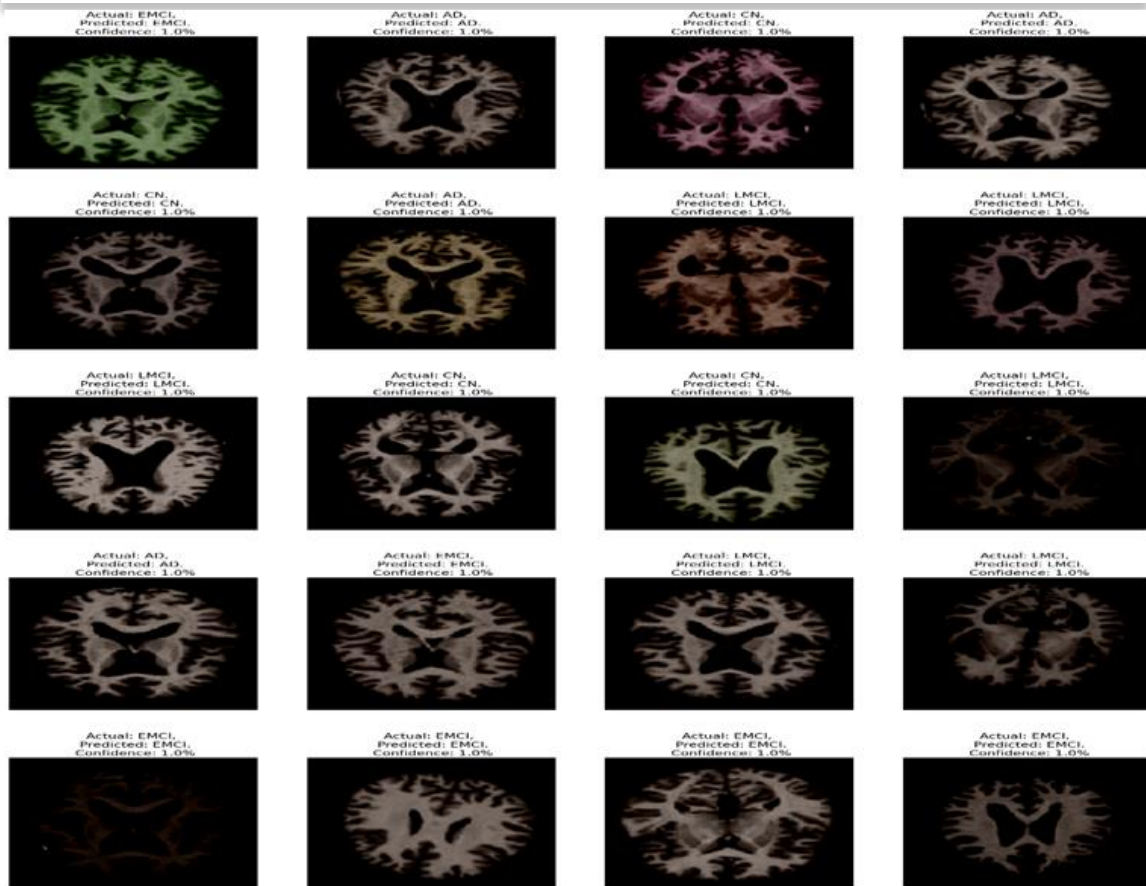


Figure (39) Model predictions.

5.7 Summary

Models					
	VGG 16	DenseNet 201	MobileNet	ResNet15 V2	CNN
Train loss	0.02	0.02	0.04	0.06	0.15
Test Loss	0.07	0.08	0.16	0.26	0.39
Validation Loss	0.08	0.07	0.19	0.29	0.48
Train Accuracy	99.1%	99.3%	98.8%	98.1%	94.3%
Test Accuracy	97.9%	97.9%	94.8%	91.7 %	85.0%
Validation Accuracy	98.3%	98.02%	95.1%	92.4%	85.7%

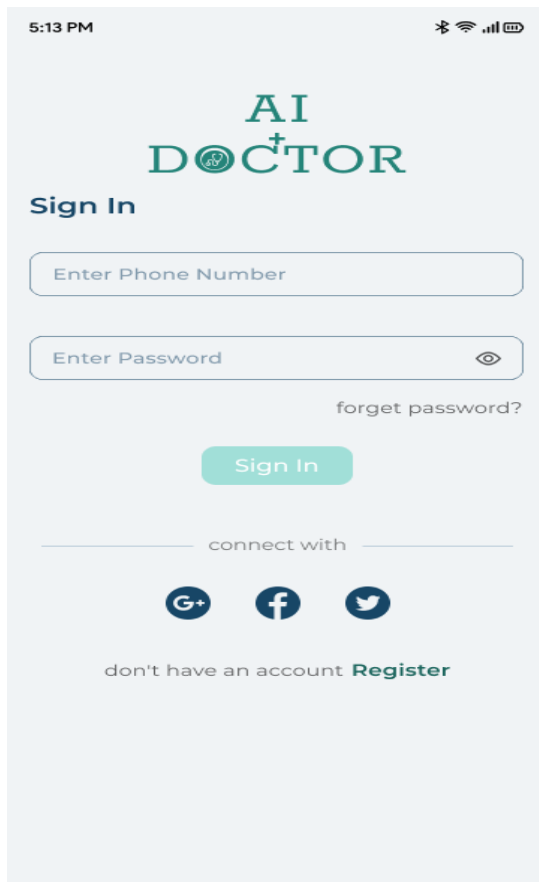
Figure (40) Models Summary

Chapter 4 → UI Design

1. Start page.

On the first page we can do:

- sign in.
- sign up.
- Initial registration for the application by Phone number, when registering for the first time, you will move to the sign-up page.



5:13 PM

AI
DOCTOR

Sign In

Enter Phone Number

Enter Password

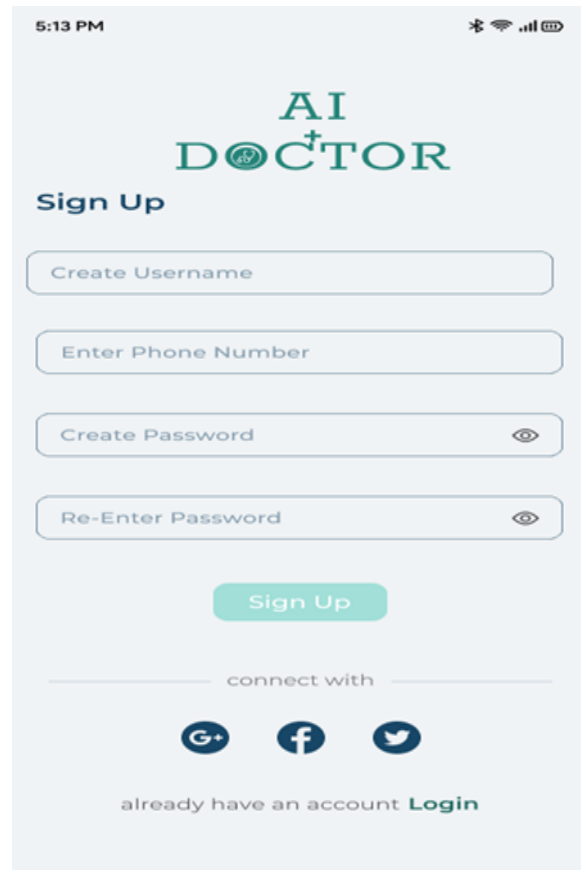
[forget password?](#)

Sign In

connect with

G+ f

don't have an account [Register](#)



5:13 PM

AI
DOCTOR

Sign Up

Create Username

Enter Phone Number

Create Password

Re-Enter Password

Sign Up

connect with

G+ f

already have an account [Login](#)

2. Home Page:

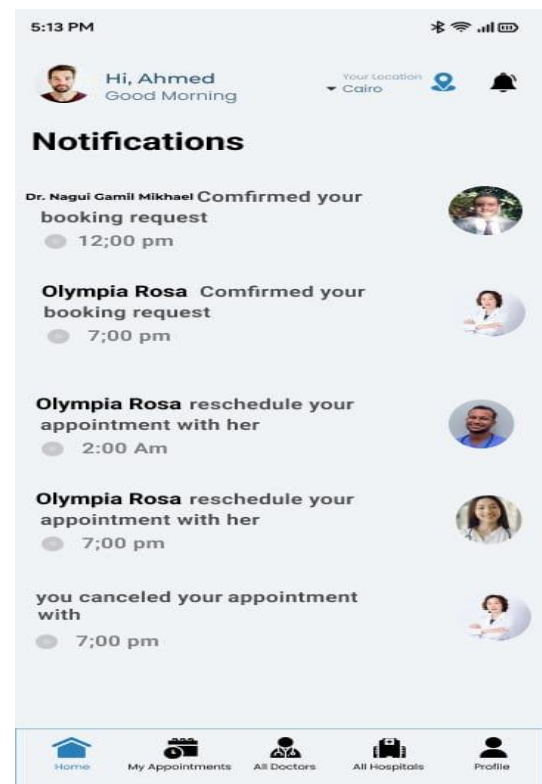
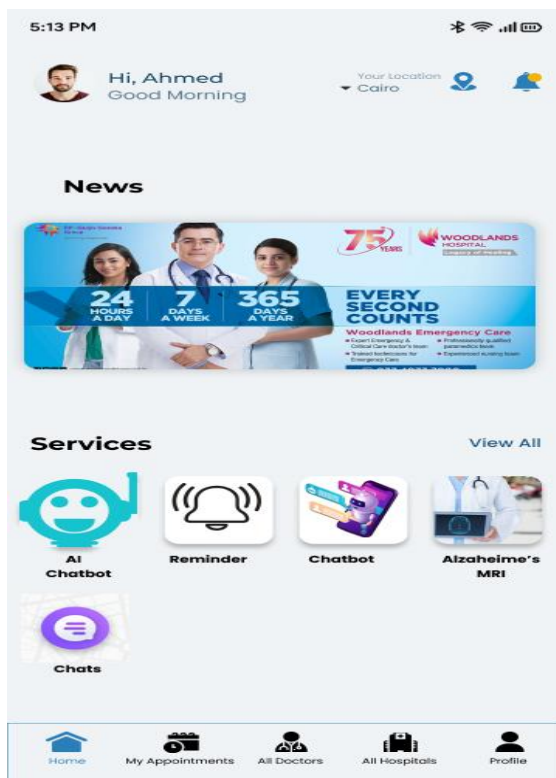
At the beginning of the application, the dashboard appears, And There are also notifications to confirm the appointment with the doctor.

® part One

News as API

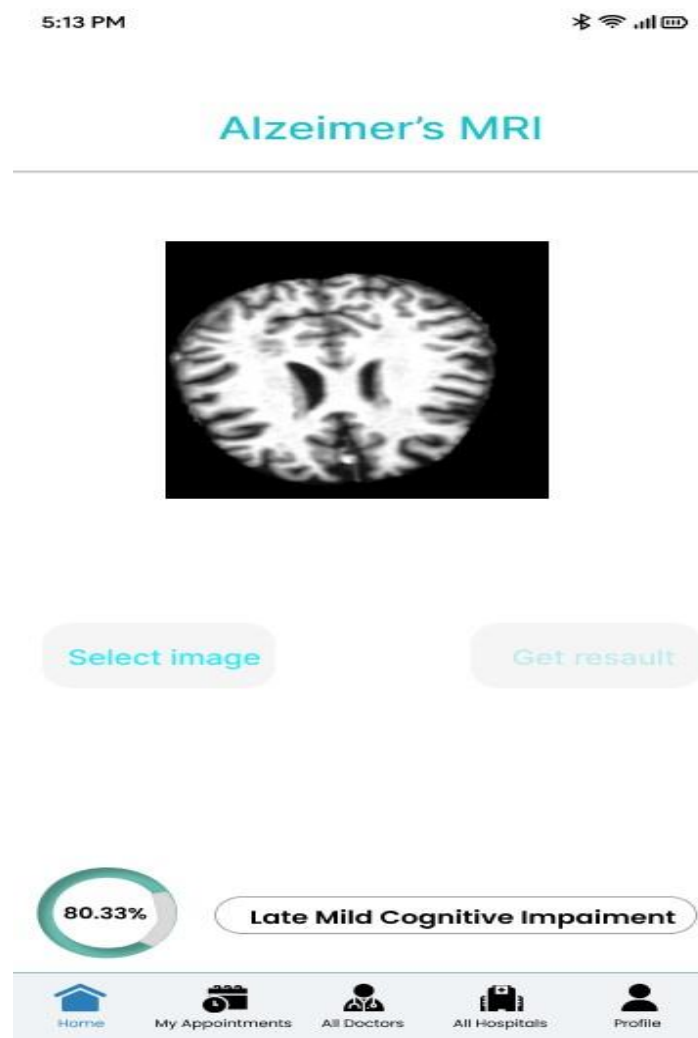
® Part two

It shows the Services to help the patient choose what he wants, these are the following: AI chatbot, chatbot, alarm, chats, Alzheimer's MRI



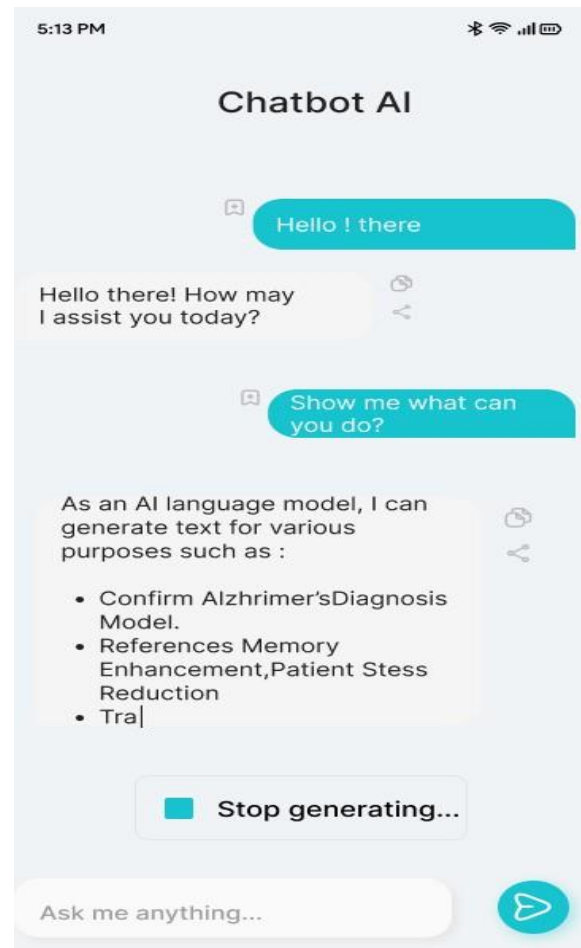
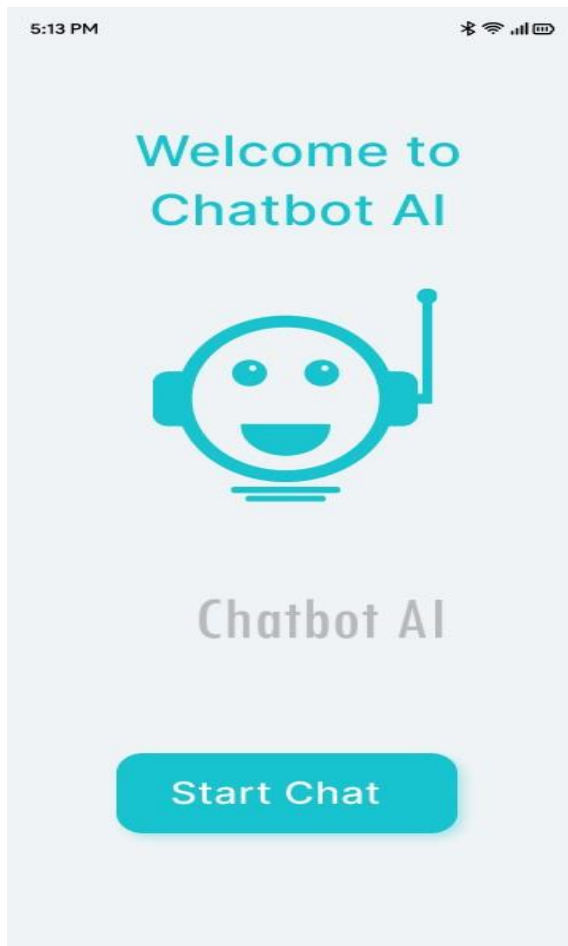
3. Models (Alzheimer's MRI)

In this section, the ray image is uploaded, and the models process this image and show us the type of disease with the percentage of its presence, and it shows the results.



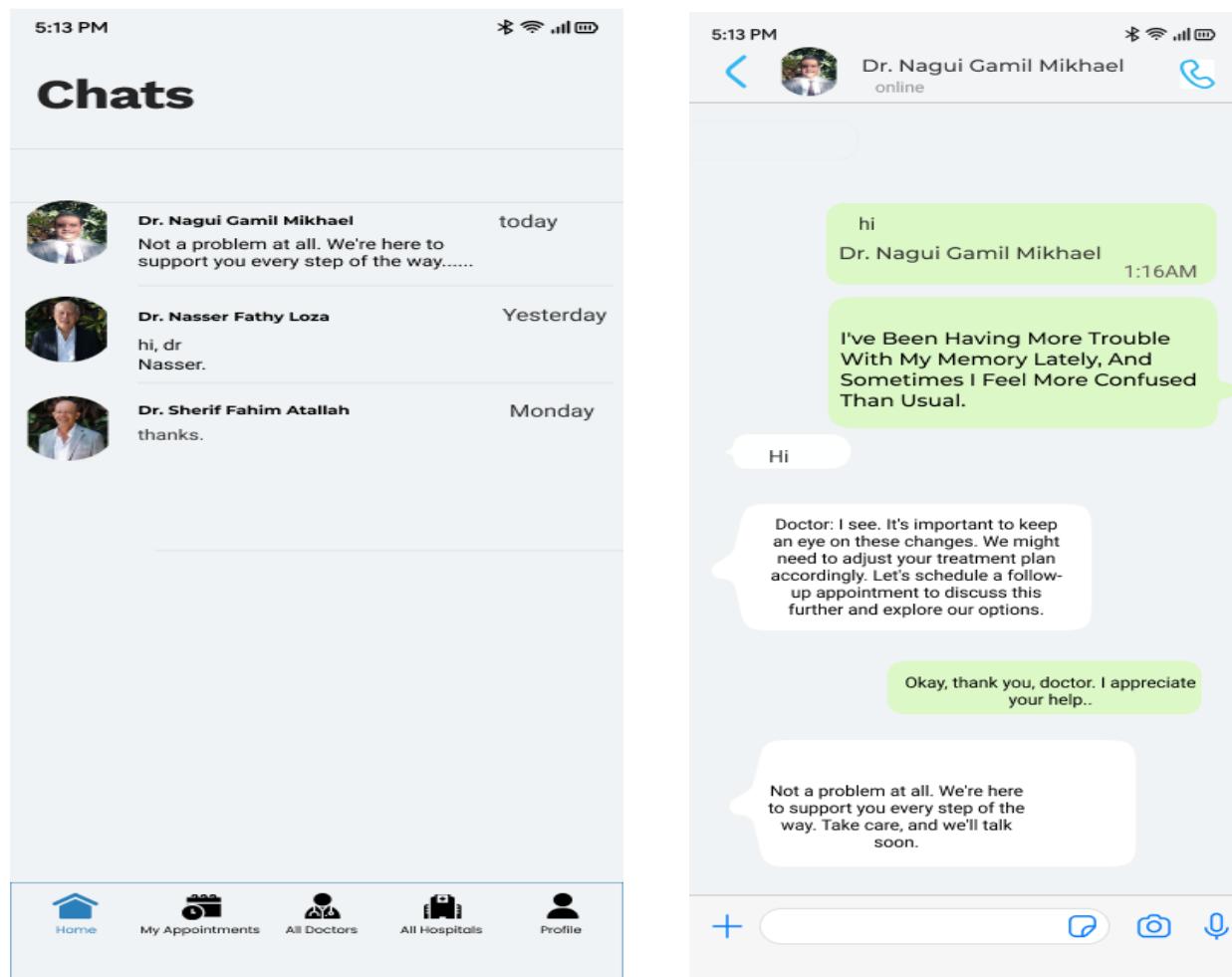
4. chatbot AI

In this section, its generation AI, the patient can confirm the result of the rays by the model. He can talk to it to learn a lot about his illness, treatment, and lifestyle.



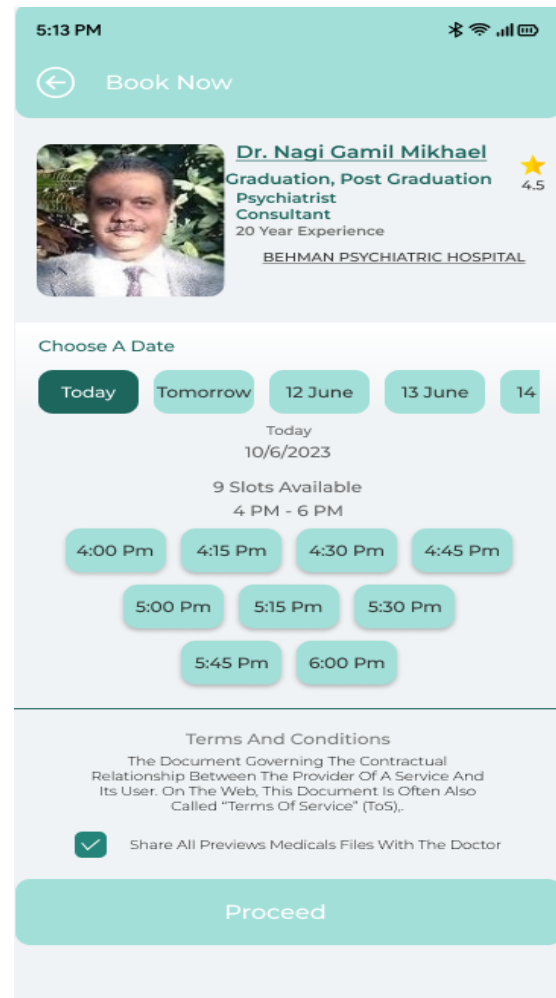
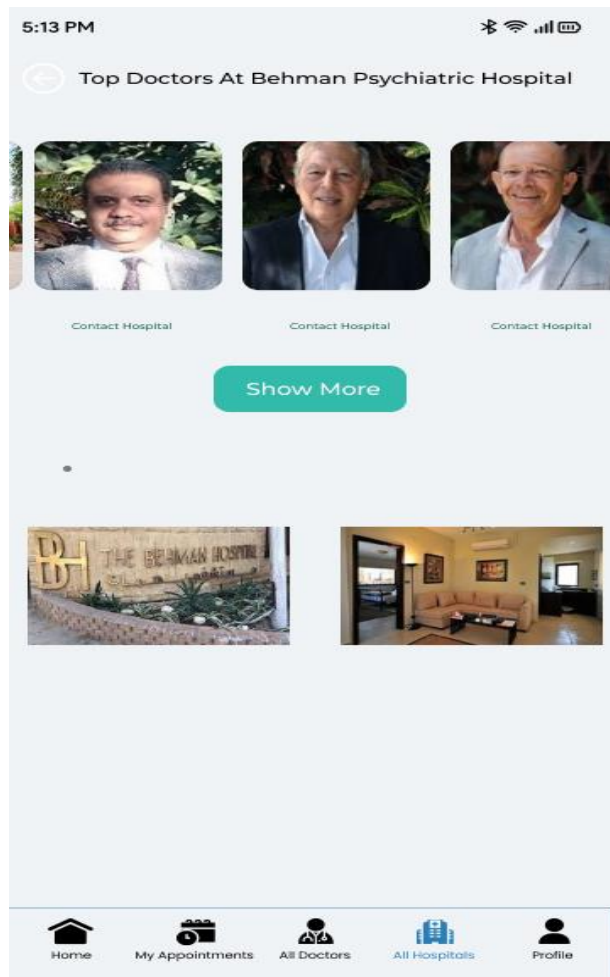
5- chats

In this section, the patient can talk to the doctor, book an appointment, and learn about the His illness and treatment easily, and this facilitates communication between him and the doctor.



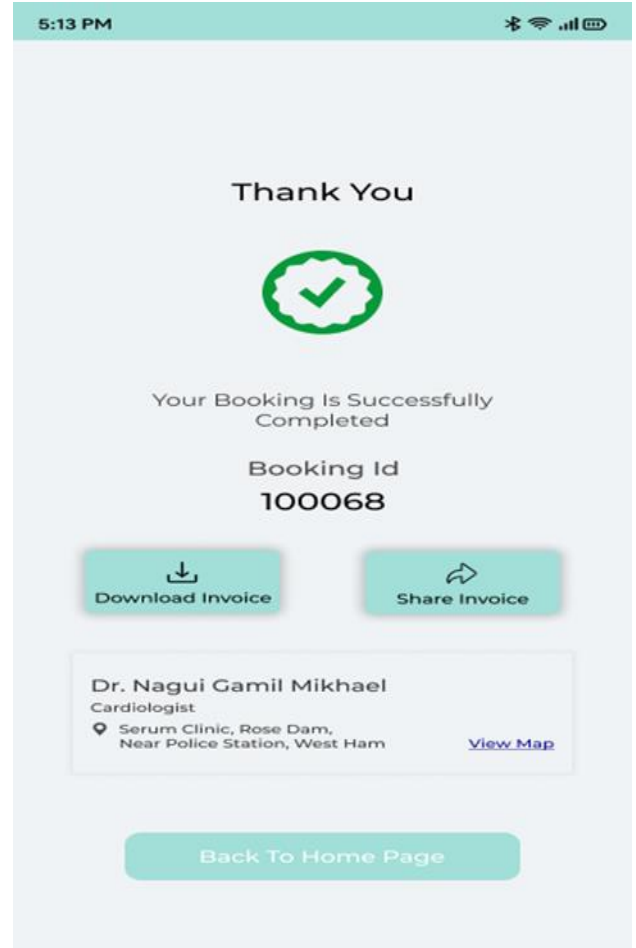
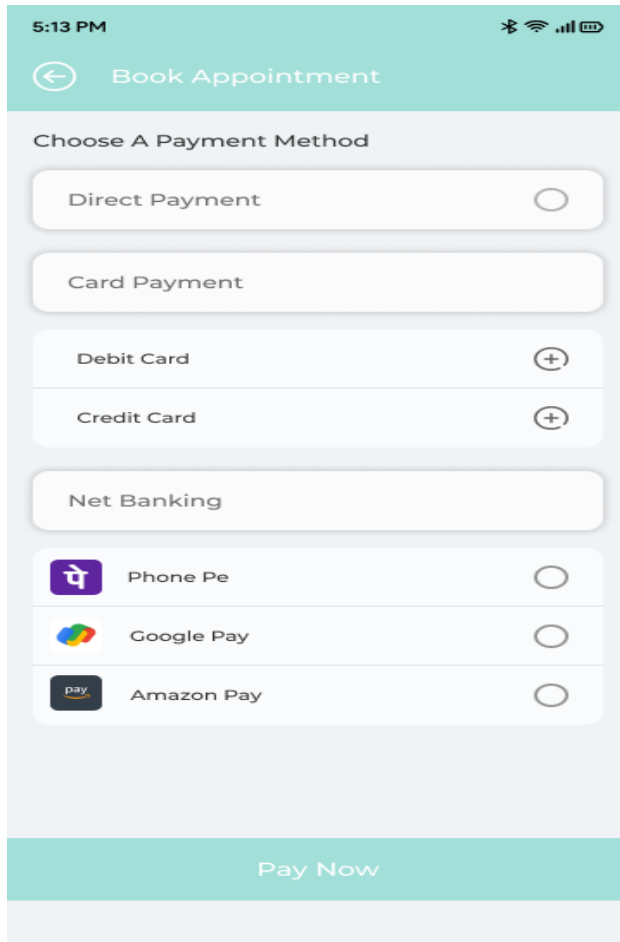
6. book Now

In this section, the patient can book an appointment. He can know the top doctors and hospitals and what hospitals can provide to the patient and choose what suits him.



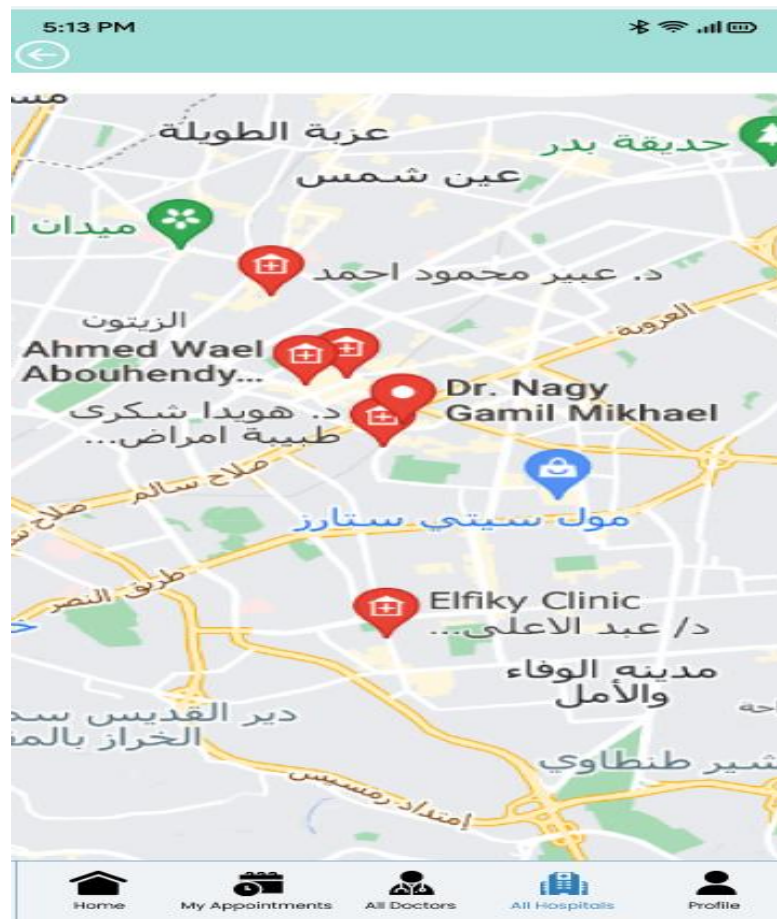
7.Payment

The patient can pay in most ways, such as cash or Visa. After your booking is successfully completed you can download invoice and can view doctor location in map.



8. Map

The map determines the Doctor closest to me. It also determines the time taken to go to it, either by walking or transportation. Displays the location of any hospital on the map when pressed. He shows me the hospitals in order of the closest to the person according to his location.



Chapter 5 → Generative AI Chatbot

5.1 Introduction

In the realm of healthcare, diagnosing medical conditions can induce significant stress for patients. Leveraging technology like generative models presents an opportunity to alleviate this burden. These models can simulate medical conditions and provide informative feedback, potentially reducing patient anxiety. By developing a generative model that confirms detection and minimizing stress.

5.2 Libraries

```
1 import os
2 import pandas as pd
3 import random
4 import matplotlib.pyplot as plt
5 import tensorflow as tf
6 import string
7 import re
8 import numpy as np
9 from transformers import (TextDataset, DataCollatorForLanguageModeling, GPT2Tokenizer, GPT2LMHeadModel, Trainer, TrainingArguments)
10 import matplotlib.pyplot as plt
11 import wandb
12 wandb.init(mode="disabled")
13 import warnings
14 warnings.filterwarnings(['ignore'])
```

5.2 Load Data

```
df=pd.read_csv('/kaggle/input/alzheimer-chat-leader/full_Chat_data.csv',usecols=[1,2])
df.columns=['source','target']
df=df.astype(str)
# display a few random samples
df.sample(5)
```

	source	target
13146	Are there any professional services available ...	Yes, professional services such as home health...
19484	Are there specific dietary recommendations to ...	A balanced diet rich in fruits, vegetables, wh...
4670	How might the potential cognitive benefits of ...	The potential cognitive benefits of coconut oi...
22432	How do beta-secretase and gamma-secretase enzy...	Beta-secretase and gamma-secretase enzymes cle...
23751	How does heart failure impact the development ...	Heart failure affects the pumping function of ...

5.3 Split Data

we split data into: -

- 90% train
- 5% test
- 5% validation

```
# shuffle the data
df = df.sample(frac=1).reset_index(drop=True)

# split the data into train, validation, and test sets
train_size = int(len(df) * 0.9)
val_size = int(len(df) * 0.05)
test_size = int(len(df) * 0.05)

train_df = df[:train_size]
val_df = df[train_size:train_size+val_size]
test_df = df[train_size+val_size:]

# display the data sets representations using a pie chart just to see the distribution of the data
labels = 'Train', 'Validation', 'Test'
sizes = [len(train_df), len(val_df), len(test_df)]
explode = (0.1, 0, 0)
fig1, ax1 = plt.subplots()
ax1.pie(sizes, explode=explode, labels=labels, autopct='%1.1f%%', shadow=True, startangle=90)
ax1.axis('equal')
plt.show()
```

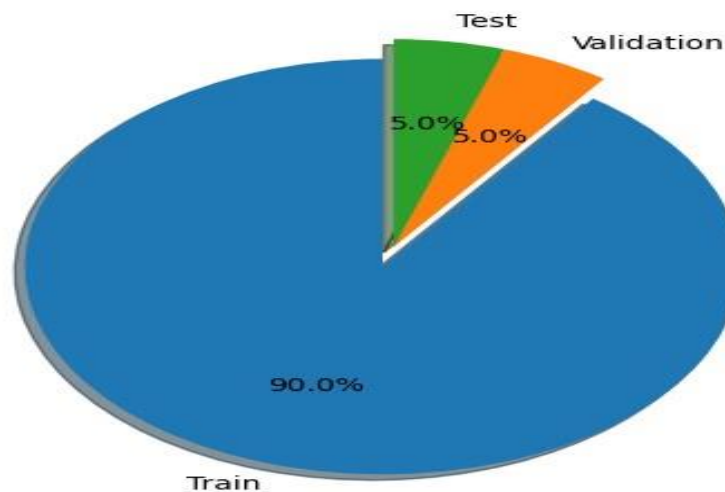


Figure (41) Transformer Data Split

5.4 Model training

- ® Transformer
- ® Pre-Trained GPT-2

5.4.1 Transformer

5.4.1.1 Text Vectorization

```
:
max_tokens = 25000
sequence_length = 200

# define a custom standardization function that convert to lowercase and strips all punctuations except "[" and "]" (so we can tell apart "start" from "[start]").
strip_chars = string.punctuation
strip_chars = strip_chars.replace("[", "")
strip_chars = strip_chars.replace("]", "")

def custom_standardization(input_string):
    lowercase = tf.strings.lower(input_string)
    return tf.strings.regex_replace(
        lowercase, f"[{re.escape(strip_chars)}]", "")

# tokenize the data using our custom standardization function
source_vectorization = tf.keras.layers.TextVectorization(
    max_tokens=max_tokens,
    output_mode="int",
    output_sequence_length=sequence_length,
)
target_vectorization = tf.keras.layers.TextVectorization(
    max_tokens=max_tokens,
    output_mode="int",
    output_sequence_length=sequence_length + 1, # add +1 token to our target sentences since they'll
    # be shifted right by 1 during training
    standardize=custom_standardization,
)

# index all tokens in the source and target sentences
train_source_texts = train_df['source'].values
train_target_texts = train_df['target'].values
source_vectorization.adapt(train_source_texts)
target_vectorization.adapt(train_target_texts)
```

5.4.1.2 Positional Embedding

```
class PositionalEmbedding(tf.keras.layers.Layer):
    def __init__(self, sequence_length, input_dim, output_dim, **kwargs):
        super().__init__(**kwargs)
        self.token_embeddings = tf.keras.layers.Embedding(input_dim=input_dim, output_dim=output_dim) # token embedding layer
        self.position_embeddings = tf.keras.layers.Embedding(input_dim=sequence_length, output_dim=output_dim) # position embedding layer
        self.sequence_length = sequence_length
        self.input_dim = input_dim
        self.output_dim = output_dim

    def call(self, inputs):
        embedded_tokens = self.token_embeddings(inputs) # embed the tokens
        length = tf.shape(inputs)[-1]
        positions = tf.range(start=0, limit=length, delta=1) # create the positional information
        embedded_positions = self.position_embeddings(positions) # embed the positions
        return embedded_tokens + embedded_positions # add the token and position embeddings to create the positional embeddings

    def compute_mask(self, inputs, mask=None):
        return tf.math.not_equal(inputs, 0)

    def get_config(self):
        config = super(PositionalEmbedding, self).get_config()
        config.update({
            "input_dim": self.input_dim,
            "output_dim": self.output_dim,
            "sequence_length": self.sequence_length,
        })
        return config
```

5.4.1.3 Scaled Dot-Product Attention

```
def scaled_dot_product_attention(q, k, v, use_causal_mask=False):
    d_k = tf.cast(tf.shape(k)[-1], tf.float32)
    scores = tf.matmul(q, k, transpose_b=True) # Matmul of Q and K
    scaled_scores = scores / tf.math.sqrt(d_k) # Scale
    if use_causal_mask:
        scaled_scores = mask_attn_weights(scaled_scores) # Mask (opt.)
    weights = tf.nn.softmax(scaled_scores, axis=-1) # SoftMax
    output = tf.matmul(weights, v) # Matmul of SoftMax and V
    return output
```

5.4.1.4 Causal Masking

```
def shape_list(x):
    """Deal with dynamic shape in tensorflow cleanly."""
    static = x.shape.as_list()
    dynamic = tf.shape(x)
    return [dynamic[i] if s is None else s for i, s in enumerate(static)]

def attention_mask(nd, ns, *, dtype):
    """1's in the lower triangle, counting from the lower right corner.
    Same as tf.matrix_band_part(tf.ones([nd, ns]), -1, ns-nd), but doesn't produce garbage on TPUs.
    """
    i = tf.range(nd)[:None]
    j = tf.range(ns)
    m = i >= j - ns + nd
    return tf.cast(m, dtype)

def mask_attn_weights(w):
    # w has shape [batch, heads, dst_sequence, src_sequence], where information flows from src to dst.
    _, _, nd, ns = shape_list(w)
    b = attention_mask(nd, ns, dtype=w.dtype)
    b = tf.reshape(b, [1, 1, nd, ns])
    w = w*b - tf.cast(1e10, w.dtype)*(1-b)
    return w
```

5.4.1.5 Multi-Head Attention

```
class MultiHeadAttention(tf.keras.layers.Layer):
    def __init__(self, embed_dim, h, **kwargs):
        super().__init__(**kwargs)
        self.embed_dim = embed_dim
        self.h = h
        if embed_dim % h != 0:
            raise ValueError(
                f"dimension of the embedding space = {embed_dim} should be divisible by number of heads = {h}"
            )
        self.q_linear = tf.keras.layers.Dense(embed_dim)
        self.k_linear = tf.keras.layers.Dense(embed_dim)
        self.v_linear = tf.keras.layers.Dense(embed_dim)
        self.concat_linear = tf.keras.layers.Dense(embed_dim)

    def split_heads(self, x, batch_size):
        x = tf.reshape(x, shape=(batch_size, -1, self.h, self.embed_dim // self.h))
        return tf.transpose(x, perm=[0, 2, 1, 3])

    def concat_heads(self, x, batch_size):
        x = tf.transpose(x, perm=[0, 2, 1, 3])
        return tf.reshape(x, (batch_size, -1, self.embed_dim))

    def call(self, q, k, v, use_causal_mask=False):
        batch_size = tf.shape(k)[0]
        q = self.q_linear(q)
        k = self.k_linear(k)
        v = self.v_linear(v)
        q = self.split_heads(q, batch_size)
        k = self.split_heads(k, batch_size)
        v = self.split_heads(v, batch_size)
        attention = scaled_dot_product_attention(q, k, v, use_causal_mask)

        concat = self.concat_heads(attention, batch_size)
        concat = self.concat_linear(concat)
        return concat

    def get_config(self):
        config = super(MultiHeadAttention, self).get_config()
        config.update({
            "embed_dim": self.embed_dim,
            "h": self.h,
        })
        return config
```

5.4.1.6 Encoder

```
class TransformerEncoder(tf.keras.layers.Layer):
    def __init__(self, embed_dim, dense_dim, num_heads, **kwargs):
        super().__init__(**kwargs)
        self.embed_dim = embed_dim
        self.dense_dim = dense_dim
        self.num_heads = num_heads
        self.layer_norm_1 = tf.keras.layers.LayerNormalization()
        self.layer_norm_2 = tf.keras.layers.LayerNormalization()
        self.global_self_attention = MultiHeadAttention(embed_dim=embed_dim, h=num_heads)
        self.feed_forward = tf.keras.Sequential(
            [tf.keras.layers.Dense(dense_dim, activation="relu"),
             tf.keras.layers.Dense(embed_dim),]
        )

    def call(self, x):
        # Post layer normalization + residual connections
        x = self.layer_norm_1(x + self.global_self_attention(q=x, k=x, v=x))
        x = self.layer_norm_2(x + self.feed_forward(x))
        return x

    def get_config(self):
        config = super().get_config()
        config.update({
            "embed_dim": self.embed_dim,
            "dense_dim": self.dense_dim,
            "num_heads": self.num_heads,
        })
        return config
```

5.4.1.7 Decoder

```
class TransformerDecoder(tf.keras.layers.Layer):
    def __init__(self, embed_dim, dense_dim, num_heads, **kwargs):
        super().__init__(**kwargs)
        self.embed_dim = embed_dim
        self.dense_dim = dense_dim
        self.num_heads = num_heads
        self.causal_self_attention = MultiHeadAttention(embed_dim=embed_dim, h=num_heads)
        self.cross_attention = MultiHeadAttention(embed_dim=embed_dim, h=num_heads)
        self.feed_forward = tf.keras.Sequential(
            [tf.keras.layers.Dense(dense_dim, activation="relu"),
             tf.keras.layers.Dense(embed_dim),]
        )
        self.layer_norm_1 = tf.keras.layers.LayerNormalization()
        self.layer_norm_2 = tf.keras.layers.LayerNormalization()
        self.layer_norm_3 = tf.keras.layers.LayerNormalization()

    def get_config(self):
        config = super().get_config()
        config.update({
            "embed_dim": self.embed_dim,
            "dense_dim": self.dense_dim,
            "num_heads": self.num_heads,
        })
        return config

    def call(self, x, context):
        # Post layer normalization + residual connections
        x = self.layer_norm_1(x + self.causal_self_attention(q=x, k=x, v=x, use_causal_mask=True))
        x = self.layer_norm_2(x + self.cross_attention(q=x, k=context, v=context))
        x = self.layer_norm_3(x + self.feed_forward(x))
        return x
```

5.4.1.8 Transformer Block for Encoder and Decoder

```
batch_size = 64

def format_dataset(source, target):
    source_vectors = source_vectorization(source)
    target_vectors = target_vectorization(target)
    return ({
        "source": source_vectors, # encoder_inputs
        "target": target_vectors[:, :-1], # decoder_inputs (truncate by 1 to keep it at the same length as decoder_outputs, which is shifted right by 1).
    }, target_vectors[:, 1:]) # decoder_outputs

def make_dataset(df):
    dataset = tf.data.Dataset.from_tensor_slices((df["source"].values, df["target"].values))
    dataset = dataset.batch(batch_size)
    dataset = dataset.map(format_dataset, num_parallel_calls=4)
    return dataset.shuffle(2048).prefetch(16).cache()

train_ds = make_dataset(train_df)
val_ds = make_dataset(val_df)
test_ds = make_dataset(test_df)
```

```
---
Layer (type)                Output Shape              Param #   Connected to
-----
===
source (InputLayer)         [(None, None)]           0        []
target (InputLayer)         [(None, None)]           0        []
positional_embedding (PositionalEmbedding)  (None, None, 256)        6451200   ['source[0][0]']
positional_embedding_1 (PositionalEmbedding)  (None, None, 256)        6451200   ['target[0][0]']
transformer_encoder (TransformerEncoder)      (None, None, 256)        1315072   ['positional_embedding[0]']
transformer_decoder (TransformerDecoder)      (None, None, 256)        1578752   ['positional_embedding_1[0]', 'transformer_encoder[0]']
dropout (Dropout)           (None, None, 256)        0         ['transformer_decoder[0]']
dense_16 (Dense)            (None, None, 25000)      6425000   ['dropout[0][0]']
===
Total params: 22221224 (84.77 MB)
Trainable params: 22221224 (84.77 MB)
Non-trainable params: 0 (0.00 Byte)
```

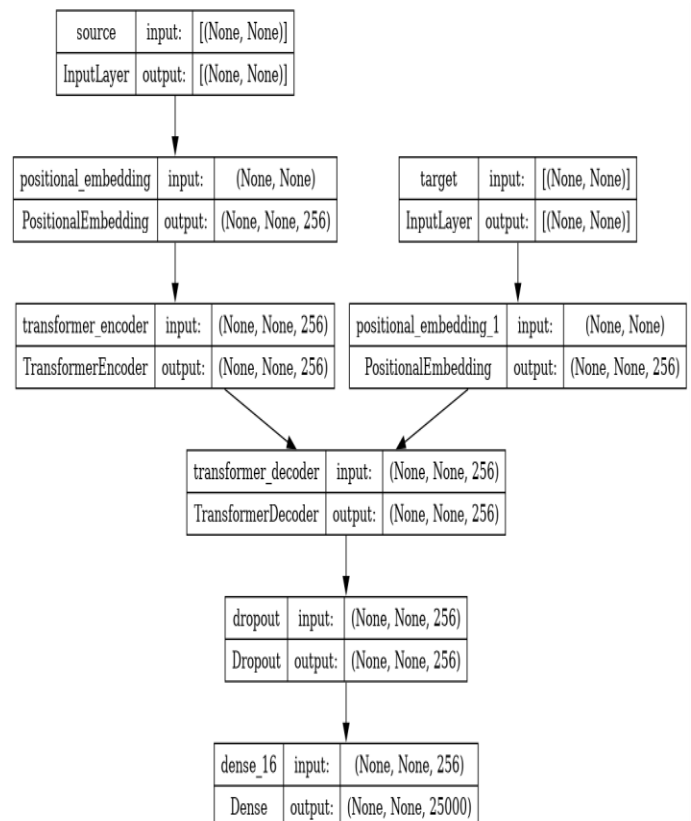


Figure (42) Transformer Block

5.4.1.9 Training the Transformer

```
transformer.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"])

EPOCHS = 50
checkpoint_filepath = '/tmp/checkpoint/'
callbacks_list = [
    tf.keras.callbacks.ReduceLROnPlateau(
        monitor='val_loss',
        factor=0.1,
        patience=3,
    ),
    tf.keras.callbacks.EarlyStopping(
        monitor='val_loss',
        patience=10,
    ),
    tf.keras.callbacks.ModelCheckpoint(
        filepath=checkpoint_filepath,
        save_weights_only=True,
        monitor='val_loss',
        save_best_only=True
    ),
]

hist= transformer.fit(train_ds,
                     epochs=EPOCHS,
                     callbacks=callbacks_list,
                     validation_data=val_ds)

transformer.load_weights(checkpoint_filepath)
```

```

Epoch 1/50

WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1710159547.055152      68 device_compiler.h:186] Compiled cluster using XLA! This
line is logged at most once for the lifetime of the process.

355/355 [=====] - 101s 257ms/step - loss: 1.1758 - accuracy: 0.8598 - v
al_loss: 0.6708 - val_accuracy: 0.8851 - lr: 0.0010
Epoch 2/50
355/355 [=====] - 87s 244ms/step - loss: 0.6174 - accuracy: 0.8902 - v
al_loss: 0.5500 - val_accuracy: 0.8979 - lr: 0.0010
Epoch 3/50
355/355 [=====] - 86s 243ms/step - loss: 0.5211 - accuracy: 0.9009 - v
al_loss: 0.4905 - val_accuracy: 0.9061 - lr: 0.0010
Epoch 4/50
355/355 [=====] - 86s 243ms/step - loss: 0.4582 - accuracy: 0.9090 - v
al_loss: 0.4457 - val_accuracy: 0.9137 - lr: 0.0010
Epoch 5/50
355/355 [=====] - 87s 244ms/step - loss: 0.4007 - accuracy: 0.9180 - v
al_loss: 0.4097 - val_accuracy: 0.9202 - lr: 0.0010
Epoch 6/50
355/355 [=====] - 87s 244ms/step - loss: 0.3540 - accuracy: 0.9255 - v
al_loss: 0.3912 - val_accuracy: 0.9240 - lr: 0.0010
Epoch 7/50
355/355 [=====] - 87s 245ms/step - loss: 0.3170 - accuracy: 0.9316 - v
al_loss: 0.3794 - val_accuracy: 0.9275 - lr: 0.0010
Epoch 8/50
355/355 [=====] - 86s 244ms/step - loss: 0.2865 - accuracy: 0.9367 - v
al_loss: 0.3779 - val_accuracy: 0.9290 - lr: 0.0010
Epoch 9/50
355/355 [=====] - 87s 244ms/step - loss: 0.2604 - accuracy: 0.9413 - v
al_loss: 0.3771 - val_accuracy: 0.9300 - lr: 0.0010
Epoch 10/50
355/355 [=====] - 86s 242ms/step - loss: 0.2381 - accuracy: 0.9453 - v
al_loss: 0.3795 - val_accuracy: 0.9313 - lr: 0.0010
Epoch 11/50
355/355 [=====] - 86s 242ms/step - loss: 0.2184 - accuracy: 0.9489 - v
al_loss: 0.3844 - val_accuracy: 0.9323 - lr: 0.0010
Epoch 12/50
355/355 [=====] - 86s 242ms/step - loss: 0.2017 - accuracy: 0.9518 - v
al_loss: 0.3891 - val_accuracy: 0.9325 - lr: 0.0010
Epoch 13/50
355/355 [=====] - 87s 245ms/step - loss: 0.1564 - accuracy: 0.9615 - v
al_loss: 0.3749 - val_accuracy: 0.9372 - lr: 1.0000e-04
Epoch 14/50
355/355 [=====] - 86s 242ms/step - loss: 0.1385 - accuracy: 0.9656 - v
al_loss: 0.3758 - val_accuracy: 0.9377 - lr: 1.0000e-04
Epoch 15/50
355/355 [=====] - 86s 242ms/step - loss: 0.1289 - accuracy: 0.9677 - v
al_loss: 0.3780 - val_accuracy: 0.9381 - lr: 1.0000e-04
Epoch 16/50
355/355 [=====] - 86s 242ms/step - loss: 0.1221 - accuracy: 0.9694 - v
al_loss: 0.3808 - val_accuracy: 0.9382 - lr: 1.0000e-04
Epoch 17/50
355/355 [=====] - 86s 242ms/step - loss: 0.1136 - accuracy: 0.9713 - v
al_loss: 0.3804 - val_accuracy: 0.9385 - lr: 1.0000e-05
Epoch 18/50
355/355 [=====] - 86s 242ms/step - loss: 0.1127 - accuracy: 0.9716 - v
al_loss: 0.3806 - val_accuracy: 0.9385 - lr: 1.0000e-05
Epoch 19/50
355/355 [=====] - 86s 242ms/step - loss: 0.1116 - accuracy: 0.9718 - v
al_loss: 0.3810 - val_accuracy: 0.9386 - lr: 1.0000e-05
Epoch 20/50
355/355 [=====] - 86s 242ms/step - loss: 0.1108 - accuracy: 0.9720 - v
al_loss: 0.3810 - val_accuracy: 0.9386 - lr: 1.0000e-06

```

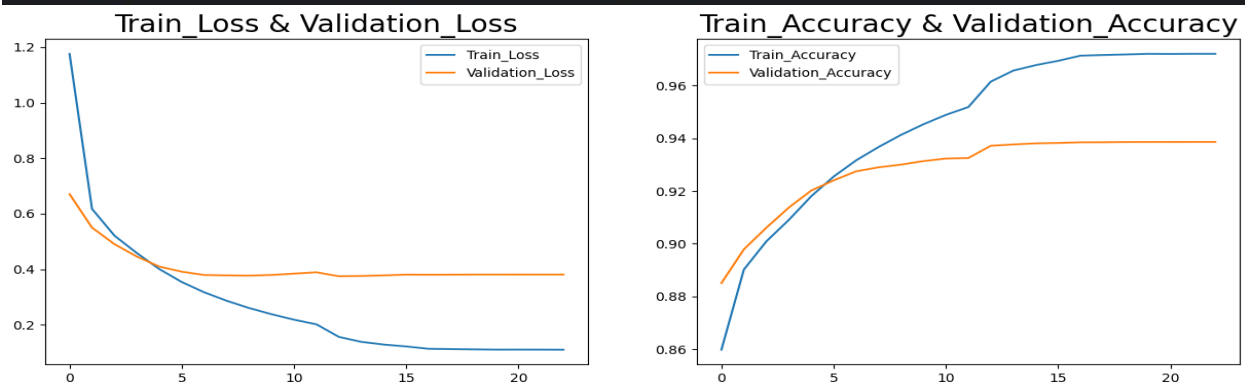


Figure (43) Training the Transformer

5.4.1.10 Model Evaluation

```
score, acc = transformer.evaluate(test_ds)
print('Test Loss =', score)
print('Test Accuracy =', acc)
```

```
20/20 [=====] - 3s 125ms/step - loss: 0.3853 - accuracy: 0.9362
Test Loss = 0.385326087474823
Test Accuracy = 0.9361587166786194
```

```
target_vocab = target_vectorization.get_vocabulary()
target_index_lookup = dict(zip(range(len(target_vocab)), target_vocab))
max_decoded_sentence_length = 200

def decode_sequence(input_sentence):
    tokenized_input_sentence = source_vectorization([input_sentence])
    decoded_sentence = "[start]"
    for i in range(max_decoded_sentence_length):
        tokenized_target_sentence = target_vectorization(
            [decoded_sentence])[0, :-1]
        predictions = model(
            [tokenized_input_sentence, tokenized_target_sentence])
        sampled_token_index = np.argmax(predictions[0, i, :])
        sampled_token = target_index_lookup[sampled_token_index]
        decoded_sentence += " " + sampled_token
        if sampled_token == "[end]":
            break
    return decoded_sentence

for i in range(50):
    random_index = np.random.randint(0, len(test_df))
    input_sentence = test_df["source"].iloc[random_index]
    print(input_sentence)
    print(decode_sequence(input_sentence))
    print()
```

How can caregivers address feelings of frustration or guilt associated with managing mind wandering in Alzheimer's patients?

[start] feelings of guilt or shame can be caregivers with others facing challenges with alzheimerers patients and their caregivers should prioritize selfcare seek support and support from healthcare providers and prioritize selfcare to manage feelings and halt the emotional progression

Can Alzheimer's disease affect a person's ability to recognize the needs of others?

[start] alzheimers disease can impair the ability to recognize familiar faces or places due to challenges in memory communication social interactions and social interactions

How does late-stage Alzheimer's disease affect communication?

[start] latestage alzheimers disease may affect language skills and communication skills leading to difficulty finding words understanding conversations and expressing thoughts

How can individuals with MCI maintain social connections?

[start] social activities provide opportunities for social interaction and emotional support helping individuals maintain cognitive function and resilience to maintain social connections with mci

How does maintaining a sense of purpose contribute to Alzheimer's disease prevention?

[start] a sense of purpose in life is associated with cognitive decline and alzheimers disease prevention understanding may provide opportunities for engagement and meaningful activities that challenge the mind may contribute to overall wellbeing and potentially reduce the risk of cognitive decline in later stages in life

Is medication used to treat mood swings in Alzheimer's patients?

[start] medication used for alzheimers patients can take their medication used to treat discomfort in medication side effects such as nausea vomiting and weight loss which can lead to mood swings

How does practicing relaxation techniques impact stress management and cognitive function?

[start] relaxation techniques such as deep breathing exercises and guided imagery can help reduce stress levels promote relaxation and reduce stress levels which may contribute to cognitive decline and promote overall wellbeing while more research is needed to confirm the effectiveness of interventions in managing stress through relaxation techniques is essential for reducing alzheimers risk

How does Vitamin D deficiency in individuals with a history of respiratory disorders impact Alzheimer's risk?

[start] vitamin d deficiency may be a risk of alzheimers for individuals with a history of respiratory disorders regular checkups and consultation with a healthcare provider are advisable

How does CLU interact with A β , and what determines the nature of this interaction?

[start] exposure to a β has been associated with nature sounds and its role to nature engaging in nature walks in outdoor activities like gardening or nature walks in the park or gardening contributes to mental wellbeing and cognitive health

How does Alzheimer's impact a person's sense of spatial awareness and navigation skills?

[start] a sense of spatial awareness and navigation can affect navigation and coordination in individuals with alzheimers may struggle with spatial awareness

How does Alzheimer's disease impact cognitive function?

[start] the cognitive domains affected by alzheimers disease leads to a gradual decline in memory thinking and behavior these cognitive function can lead to a decline in cognitive functions seen in alzheimers disease

Are there any clinical trials evaluating the effectiveness of Curcumin in Alzheimer's disease treatment?

[start] clinical trials are underway focusing on the effectiveness of curcumin in alzheimers disease while others do not reverse the effects of curcumin in clinical trials may provide access to cuttingedge treatments and contribute to the advancement of alzheimers research

Are there specific signs of dehydration to watch for in someone with Alzheimer's?

[start] signs of dehydration may include difficulty swallowing confusion about meals or snacks the time or place challenges in decisionmaking and challenges in problemsolving

5.4.2 Pre-Trained GPT-2

5.4.2.1 Helper Function for GPT-2

```
def load_dataset(file_path, tokenizer, block_size = 1024):
    dataset_train = TextDataset(
        tokenizer = tokenizer,
        file_path = file_path+'X_train.csv',
        block_size = block_size,
    )
    dataset_test = TextDataset(
        tokenizer = tokenizer,
        file_path = file_path+'X_test.csv',
        block_size = block_size,
    )
    dataset_val = TextDataset(
        tokenizer = tokenizer,
        file_path = file_path+'X_val.csv',
        block_size = block_size,
    )
    return dataset_train, dataset_test, dataset_val
```

```
def load_data_collator(tokenizer, mlm = False):
    data_collator = DataCollatorForLanguageModeling(
        tokenizer=tokenizer,
        mlm=mlm,
    )
    return data_collator
```

```
def train(train_file_path, model_name, output_dir, overwrite_output_dir,
          per_device_train_batch_size, per_device_val_batch_size, num_train_epochs):
    tokenizer = GPT2Tokenizer.from_pretrained(model_name)

    # Load datasets
    train_dataset, test_dataset, val_dataset = load_dataset(train_file_path, tokenizer)

    # Load data collator
    data_collator = load_data_collator(tokenizer)

    # Save tokenizer
    tokenizer.save_pretrained(output_dir)

    # Load or initialize model
    model = GPT2LMHeadModel.from_pretrained(model_name)

    # Save model
    model.save_pretrained(output_dir)

    training_args = TrainingArguments(
        output_dir=output_dir,
        overwrite_output_dir=overwrite_output_dir,
        num_train_epochs=num_train_epochs,
        per_device_train_batch_size=per_device_train_batch_size,
        per_device_eval_batch_size=per_device_val_batch_size,
        evaluation_strategy="epoch",
        logging_dir="./logs",
        logging_steps=100, # Log every 100 steps
        save_steps=500, # Save checkpoint every 500 steps
        logging_first_step=True,
        save_total_limit=2,
    )

    trainer = Trainer(
        model=model,
        args=training_args,
        data_collator=data_collator,
        train_dataset=train_dataset,
        eval_dataset=val_dataset
    )

    hist = trainer.train()
    trainer.save_model()
    return trainer, hist, test_dataset
```

5.4.2.2 Training GPT-2

```
train_file_path = '/kaggle/working/'
model_name = 'gpt2'
output_dir = '/kaggle/working/custom_model'
overwrite_output_dir = True
per_device_train_batch_size = 4
per_device_val_batch_size=8
num_train_epochs = 30

# Train
model, hist, test_dataset = train(
    train_file_path=train_file_path,
    model_name=model_name,
    output_dir=output_dir,
    overwrite_output_dir=overwrite_output_dir,
    per_device_train_batch_size=per_device_train_batch_size,
    per_device_val_batch_size=per_device_val_batch_size,
    num_train_epochs=num_train_epochs,
)
```

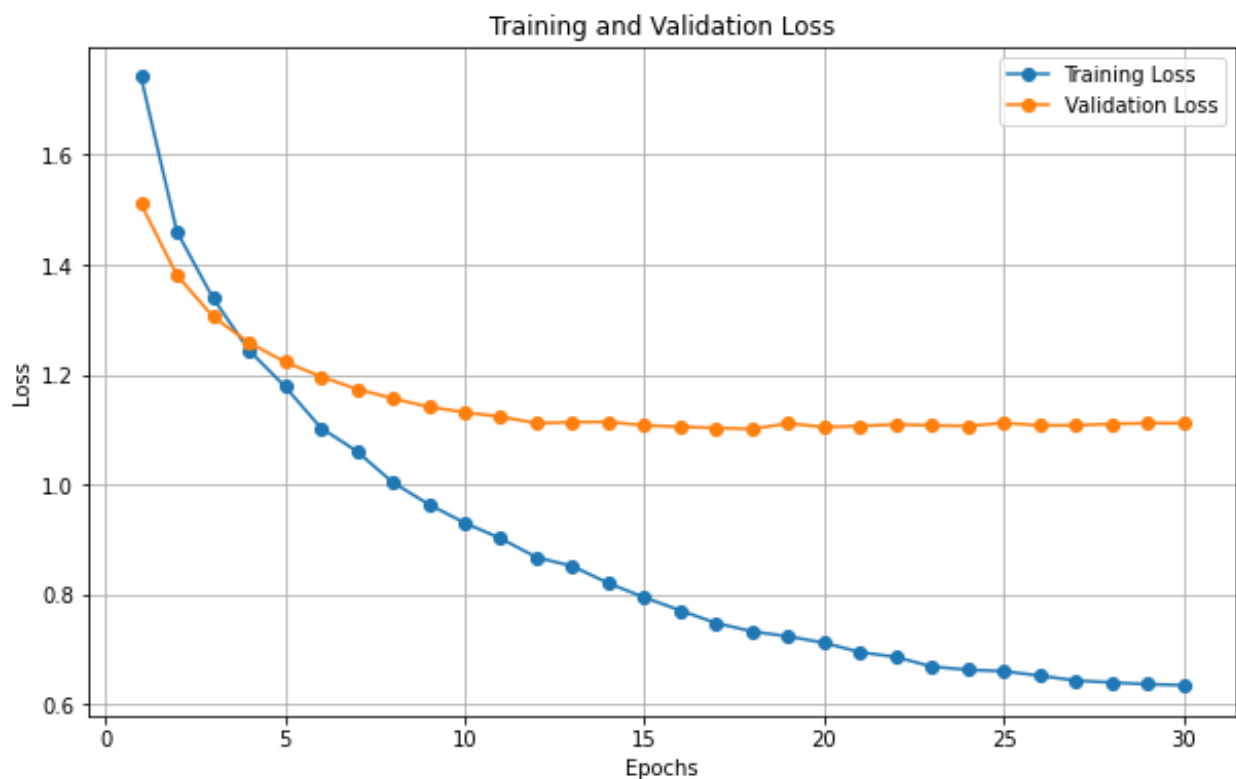


Figure (44) Training GPT-2

5.4.2.3 Evaluation GPT-2

```
print("Global Step:", hist.global_step)
print("Epoch:", hist.metrics['epoch'])
print("Train Runtime:", hist.metrics['train_runtime'])
print("Train Samples Per Second:", hist.metrics['train_samples_per_second'])
print("Train Steps Per Second:", hist.metrics['train_steps_per_second'])
print("Total FLOPS:", hist.metrics['total_flops'])
print("Train Loss:", hist.metrics['train_loss'])
```

```
Global Step: 8430
Epoch: 30.0
Train Runtime: 6174.3601
Train Samples Per Second: 5.456
Train Steps Per Second: 1.365
Total FLOPS: 1.760585711616e+16
Train Loss: 0.8861369677427118
```

```
eval_results = model.evaluate(eval_dataset=test_dataset)
print('Evaluation on Test Data:', eval_results)
```

[8/8 00:03]

```
Evaluation on Test Data: {'eval_loss': 1.111701488494873, 'eval_runtime': 3.5547, 'eval_samples_per_second': 17.442, 'eval_steps_per_second': 2.251, 'epoch': 30.0}
```

5.4.2.4 Generation Using Custom GPT-2

```
def load_model(model_path):
    model = GPT2LMHeadModel.from_pretrained(model_path)
    return model

def load_tokenizer(tokenizer_path):
    tokenizer = GPT2Tokenizer.from_pretrained(tokenizer_path)
    return tokenizer

def generate_text(model_path, sequence, max_length):

    model = load_model(model_path)
    tokenizer = load_tokenizer(model_path)
    ids = tokenizer.encode(f'{sequence}', return_tensors='pt')
    final_outputs = model.generate(
        ids,
        do_sample=True,
        max_length=max_length,
        pad_token_id=model.config.eos_token_id,
        top_k=50,
        top_p=0.95,
    )
    return tokenizer.decode(final_outputs[0], skip_special_tokens=True)
```

Generative AI Chatbot

```
model_path = "/kaggle/working/custom_model"
sequence = test_df['Questions'].iloc[100]
max_len = 100
print("Q : ",test_df['Questions'].iloc[100])
print()
print("A : ",test_df['Answers'].iloc[100])
print()
print("G : ",generate_text(model_path, sequence, max_len))
```

Q : How does Alzheimer's affect the ability to plan and cook meals?

A : Alzheimer's can impact planning and cooking meals due to challenges in sequential tasks. Simplifying recipes and providing assistance can support individuals in the kitchen.

G : How does Alzheimer's affect the ability to plan and cook meals?, "Alzheimer's impairs planning and cooking skills, making meal preparation and preparation more difficult. Preparing meals on a step-by-step basis involves intricate planning and hand-holding, while ensuring safety and familiarity."

```
sequence = test_df['Questions'].iloc[50]
max_len = 200
print("Q : ",test_df['Questions'].iloc[50])
print()
print("A : ",test_df['Answers'].iloc[50])
print()
print("G : ",generate_text(model_path, sequence, max_len))
```

Q : What role does nutrition play in the management of Alzheimer's disease?

A : Adequate nutrition is essential for overall health and well-being in Alzheimer's patients. Nurses can assess nutritional status, provide dietary recommendations, monitor food intake, and address barriers to proper nutrition.

G : What role does nutrition play in the management of Alzheimer's disease?, "Nutrition plays a crucial role in the management of Alzheimer's disease by offering essential nutrients, antioxidants, and anti-inflammatory compounds that support brain health, function, and overall well-being. Ensuring adequate nutrition through a balanced diet can support brain health, overall well-being, and cognitive function in individuals with Alzheimer's. Consultation with healthcare professionals can help tailor dietary recommendations and manage nutritional deficiencies in individual patients."

```
sequence = "what is alzheimer's disease"
max_len = 500
print("Q : ",sequence)
print()
print("G : ",generate_text(model_path, sequence, max_len))
```

Q : what is alzheimer's disease

G : what is alzheimer's disease and how does it differ from dementia?, "Alzheimer's disease is common, and while it is not considered a dementia, it shares some common risk factors. Alzheimer's is distinguished by characteristic brain changes visible during autopsy, such as shrinkage of the hippocampus, white matter lesions, or structural changes like perivascular folds and amyloid plaques."

References

- [1] <https://www.kaggle.com/ahmedashrafahmed>
- [2] <https://www.tensorflow.org/?hl=ar>
- [3] <https://adni.loni.usc.edu/>
- [4] <https://scikit-learn.org/stable/tutorial/index.html>
- [5] <https://sigma.software/services/ui-ux-design>
- [6] <https://arxiv.org/abs/1706.03762>
- [7] <https://www.tensorflow.org/text/tutorials/transformer>
- [8] <https://github.com/openai/gpt-2/blob/master/src/model.py>
- [9] <https://github.com/karpathy/minGPT/blob/master/minGPT/model.py>
- [10] <https://livebook.manning.com/book/deep-learning-with-python-second-edition/chapter-11>
- [11] https://github.com/fchollet/deep-learning-with-python-notebooks/blob/master/chapter11_part03_transformer.ipynb
- [12] https://github.com/fchollet/deep-learning-with-python-notebooks/blob/master/chapter11_part04_sequence-to-sequence-learning.ipynb
- [13] <https://www.oreilly.com/library/view/natural-language-processing/9781098136789/ch03.html>
- [14] https://github.com/nlp-with-transformers/notebooks/blob/main/03_transformer-anatomy.ipynb